Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Dinh Quoc Cuong – 523H0008

Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT

DEVOPS, CONTAINERIZATION & CLOUD SYSTEMS

Supervisor

MSc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, we would like to express our sincere gratitude to our supervisor, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and the academic rigour of this work.

We are also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for our work. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to our growth.

Our gratitude further extends to our classmates and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, we are profoundly grateful to our families for their unconditional support, patience, and encouragement. Their belief in us has always been our greatest source of strength and determination.

To all of you, we extend our heartfelt thanks.

Certification

This thesis was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(Title, Full Name and Signature)

This thesis was defended at the Undergraduate Thesis Examination Committee held at Ton Duc Thang University on . . . . . / . . . . . / . . . . . .

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the Faculty after receiving the revised thesis (if any).

CHAIRMAN **DEAN OF FACULTY**

. . . . . .

Declaration

This thesis was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this thesis are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this thesis. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this thesis. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

Author(s)

(Signature and full name)

Abstract

A video sharing platform must turn every upload, in any format, codec or resolution, into HTTP Live Streaming (HLS) renditions that a browser can play at adaptive quality. Transcoding is a bursty workload: a server cluster kept running to wait for uploads pays for idle capacity most of the time, yet the platform must still absorb many uploads arriving at once.

This project designs, builds and evaluates a complete video sharing platform whose transcoding runs on serverless containers driven by events. An upload to Amazon S3 emits an event that Amazon SQS buffers and an AWS Lambda function submits to AWS Batch on AWS Fargate, where a container packaging FFmpeg exists only while it works: it produces up to six renditions from 144p to 1080p, a master playlist and a thumbnail, and then terminates. Videos of five minutes or more are split into chunks encoded in parallel. Amazon CloudFront delivers the streams, and Origin Access Control together with CloudFront Signed Cookies keeps a private video unreachable even to someone who knows its manifest URL. Uploads are screened with Amazon Rekognition, and staff review doubtful videos in a console whose recorded decisions make the error rates of the screening measurable. The infrastructure is described in thirteen Terraform modules, and seven GitHub Actions workflows run the tests, the security scans and the deployment.

Measured on the deployed system, a four-hour video was ready 135 minutes after its upload at the production quota of 8 vCPU, and one hundred clips submitted at once were all processed without a failure. Measuring the time to first frame exposed a content delivery configuration that served Vietnam from Europe; correcting it cut the median from 727.69 ms to 110.32 ms. For the part of the bill that the architecture controls, the estimated monthly cost is 3.40 USD against 133.35 USD for an always-on instance. The saving comes from removing idle capacity, and it narrows as sustained utilisation rises.

List of Abbreviations

ABR	Adaptive Bitrate Streaming
ACM	AWS Certificate Manager
API	Application Programming Interface
ARN	Amazon Resource Name
AWS	Amazon Web Services
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DLQ	Dead Letter Queue
DNS	Domain Name System
DRM	Digital Rights Management
EBS	Amazon Elastic Block Store
ECR	Amazon Elastic Container Registry
ECS	Amazon Elastic Container Service
ERD	Entity Relationship Diagram
FinOps	Cloud Financial Operations
FR	Functional Requirement
GPU	Graphics Processing Unit
HEVC	High Efficiency Video Coding
HLS	HTTP Live Streaming
IaC	Infrastructure as Code
IAM	AWS Identity and Access Management
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
NFR	Non-Functional Requirement
OAC	Origin Access Control

QoE	Quality of Experience
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SPA	Single Page Application
SQS	Amazon Simple Queue Service
S3	Amazon Simple Storage Service
TLS	Transport Layer Security
TTFF	Time-to-First-Frame
TTL	Time To Live
UC	Use Case
vCPU	Virtual Central Processing Unit
VMAF	Video Multi-Method Assessment Fusion
VOD	Video on Demand
VPC	Amazon Virtual Private Cloud

Contents

Acknowledgment i

Certification ii

Declaration iii

Abstract iv

List of Abbreviations v

1 Introduction 1

1.1 Background and Rationale 1
1.2 Project Objectives 2
1.3 Scope and Deliverables 2
1.4 Research Methodology 3
1.5 Practical Significance 3
1.6 Main Contributions 4
1.7 Structure of the Report 4

2 Literature Review and Technologies 5

2.1 Related Platforms and Comparison 5
2.2 Core Models and Principles 7
2.2.1 Serverless Containers 7
2.2.2 Event-Driven Architecture 9
2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder 9
2.3 Technology Stack Justification 11

3 Requirements Analysis 12

3.1 Stakeholders and User Requirements 12
3.2 Functional Requirements 13

3.3 Non-Functional Requirements . . . 15
3.4 Use Case Model . . . 16
3.4.1 Use Case Diagrams . . . 16
3.4.2 Use Case Specifications . . . 18

4 System Design **27**
4.1 High-Level Architecture . . . 27
4.2 Transcoding Pipeline Design . . . 29
4.2.1 Upload of Large Files . . . 29
4.2.2 Chunk-Parallel Transcoding of Long Videos . . . 31
4.2.3 HLS Transcoding Pipeline and Multi-Variant Segmentation . . . 32
4.3 Database Design . . . 33
4.4 Content Protection Design . . . 37
4.5 Content Moderation and Operations Design . . . 39
4.5.1 Automatic Screening . . . 41
4.5.2 Moderation States and Visibility . . . 42
4.5.3 Review, Appeals and Audit . . . 42
4.5.4 Measuring the Screening . . . 43
4.5.5 Roles and Permissions . . . 44
4.5.6 Verifying the Owner of an Account . . . 44
4.6 Interface Design . . . 46
4.7 API Design . . . 48
4.8 Performance and Auto-Scaling Architecture . . . 49
4.8.1 Event-Driven Serverless Compute Elasticity . . . 50
4.8.2 Chunk-Parallel Speedup for Long-Duration Media . . . 50
4.8.3 Edge Offloading and Delivery Scale . . . 50
4.8.4 Application Layer Scaling and Serverless Evolution . . . 51

5 Development and Testing **52**
5.1 Backend Implementation . . . 52
5.1.1 Layered Services and Access Controls . . . 52
5.1.2 Serverless Execution Adapter and State Management . . . 54
5.2 Transcoder Implementation . . . 54
5.3 Infrastructure as Code . . . 56
5.4 CI/CD Pipeline and DevSecOps . . . 57
5.5 Challenges During Development . . . 59
5.5.1 CloudFront in a Second Account . . . 60

5.5.2 Retry and Alerting on the Deployed Path 61
5.6 Testing 61

6 Deployment and Evaluation **64**
6.1 The Deployed Environment 64
6.2 User Interface Showcase 65
6.3 Quality of Experience Evaluation 68
6.3.1 Time to First Frame Through the CDN 69
6.3.2 Delivery Latency by Client Location 70
6.3.3 Playback Under Constrained Bandwidth 71
6.3.4 Effect of Content Complexity 73
6.4 Transcoding Pipeline Performance 75
6.5 Chunked Transcoding of Long Videos 76
6.6 Load Testing 81
6.6.1 Queue and Admission Under Burst 81
6.6.2 Throughput With Real Video 82
6.6.3 Serverless Backend Concurrency and Latency Evaluation 83
6.7 Cost Analysis 84
6.8 Requirements Against the Evidence 86

7 Conclusion and Future Work **87**
7.1 Summary of Achievements 87
7.2 Strengths of the System 88
7.3 Limitations 89
7.4 Future Work 90
7.5 Task Distribution 91

A Supplementary Material **95**
A.1 Repository Structure 95
A.2 FFmpeg Transcoding Command 95
A.3 Signed Cookie Custom Policy 97
A.4 Reproducing the Measurements 98
A.5 Deploying the Infrastructure 99

B Measurement Tables **101**
B.1 Time-to-First-Frame 101
B.2 Playback Quality 103

B.3 Transcoding Latency and Capacity 103

List of Figures

2.1 *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers* 8
2.2 *HLS artefacts produced from one source file and the order in which a player retrieves them* 10

3.1 *Use case diagram of the video sharing platform* 17
3.2 *Use case diagram of moderation and operation* 18

4.1 *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery; the key at the lower left names each step* 28
4.2 *Sequence diagram of the event-driven transcoding pipeline for one container; a video of five minutes or more takes the chunked path of Table 4.1* 30
4.3 *Dedicated HLS Transcoding Pipeline, illustrating stream probing, multivariant filtergraph scaling, synchronized GOP keyframe alignment, and AES-128 encrypted segment packaging (bitrates denote maximum rendition ceilings)* 32
4.4 *Entity relationship diagram of the core MongoDB collections* 34
4.5 *Entity relationship diagram of the collections that support moderation and operation; every video field refers to* `Video` *and every user field to* `User` 36
4.6 *Sequence diagram of playback authorisation: the backend issues Signed Cookies for one video, and CloudFront checks them on every request* . . 38
4.7 *Sequence diagram of moderation: automatic screening, a viewer's report, an operator's decision and the owner's appeal* 40
4.8 *Sequence diagram of account verification: following the e-mailed link, or signing in with Google at the same address* 45
4.9 *User flow through the application, from an anonymous visit to a published video* 47

5.1 *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate* 58

6.1 *Catalogue page, the entry point for a visitor* 65
6.2 *Watch page with custom player, rendition selector and related videos sidebar* 66
6.3 *Upload page, showing direct-to-storage transfer with live telemetry* . . . 66
6.4 *Channel management view, including videos still being processed* . . . 67
6.5 *Operations console, Removed tab, with the labels the machine detected and who removed each video* 67
6.6 *Operations console, Accuracy tab, each rate shown with its interval and sample size* 68
6.7 *Median time to first frame in three delivery configurations, and the median of each stage where it was measured* 69
6.8 *Median time to first frame by client location, before and after* `PriceClass_All` 70
6.9 *Rebuffering ratio and average bitrate for three sources of differing visual complexity under three network profiles* 74
6.10 *A 25-minute video on the chunked path: time to READY and compute by chunk configuration and capacity* 78
6.11 *A four-hour video: time from the end of the upload to READY, by step, at 16 and at 8 vCPU* 79
6.12 *Four 25-minute videos submitted together: when each waited, encoded and finalised, at 8 and at 16 vCPU* 80

List of Tables

2.1 Comparison of transcoding approaches across platforms 6

3.1 Stakeholders, their needs and the requirements that serve them 13
3.2 Functional requirements of the system 14
3.3 Functional requirements of moderation and operation 15
3.4 Non-functional requirements of the system 16
3.5 Use case UC-01: register and verify the e-mail address 19
3.6 Use case UC-02: log in 20
3.7 Use case UC-03: reset a forgotten password 20
3.8 Use case UC-04: browse and search the catalogue 21
3.9 Use case UC-05: watch a video 21
3.10 Use case UC-06: like, comment on and share a video 22
3.11 Use case UC-07: upload a video 22
3.12 Use case UC-08: manage one's own videos 23
3.13 Use case UC-09: transcode a video automatically 23
3.14 Use case UC-10: report a video 24
3.15 Use case UC-11: decide on a video 24
3.16 Use case UC-12: appeal a removal 25
3.17 Use case UC-13: audit a random sample and read the error rates 25
3.18 Use case UC-14: assign a role 26

4.1 Jobs of the chunked path 31
4.2 Keys, indexes and constraints of the core collections 33
4.3 Collections that support moderation and operation 35
4.4 Actions assigned to Rekognition labels by the screening policy 41
4.5 Outcome class of a human decision, given the machine's conclusion . . 43
4.6 Permissions of the four roles 44
4.7 Endpoints of the REST API, grouped by purpose 49

5.1 Terraform modules and their responsibilities 57
5.2 Problems met during deployment, their causes and resolutions 60
5.3 Automated backend tests by area, 492 tests in 35 suites 62
5.4 Automated transcoder tests by area, 361 tests in 17 suites 63

6.1 Playback quality by emulated network profile 72
6.2 Declared against actual bitrate for each rendition 72
6.3 Playback quality before and after the manifest correction 73
6.4 Unthrottled playback at device pixel ratio one against two 75
6.5 Transcoding pipeline latency by input size 76
6.6 Single-container path against chunked path for videos of five to fifteen minutes 77
6.7 Simultaneous submissions at 16 vCPU 80
6.8 Cross-validation of the load test across two independent tools 82
6.9 Backlog drain under synthetic and real payloads 83
6.10 Architectural and performance comparison between monolithic EC2 and serverless Lambda 84
6.11 Monthly operating cost comparison for the part of the bill that the architecture controls (ap-southeast-1) 85
6.12 Non-functional requirements against the evidence 86

7.1 Objectives of the project against their results 88
7.2 Task distribution between the members of the team 91

A.1 Provenance of each measurement table 99

B.1 Time-to-First-Frame measurement results 101
B.2 Time-to-First-Frame through CloudFront after the price class correction 102
B.3 Time-to-First-Frame by client location, before and after `PriceClass_All` 102
B.4 Playback quality across three sources of differing visual complexity . . 103
B.5 A 25-minute video on the chunked path, by chunk configuration and capacity 104
B.6 A four-hour video at two capacities 104
B.7 Four 25-minute videos submitted together, by capacity 105

Chapter 1

Introduction

1.1 Background and Rationale

Video has become the dominant form of content on the Internet, and platforms such as YouTube, Vimeo, and TikTok have made uploading and sharing video an everyday activity. Behind that apparent simplicity lies a demanding engineering problem. When a user uploads a source file, that file may use any container format, any codec, and any resolution, yet the platform must deliver a stream that plays smoothly on every device and adapts to whatever bandwidth the viewer happens to have at that moment.

The industry solution to this problem is HTTP Live Streaming (HLS), a protocol that splits a video into short segments, typically between two and ten seconds long, and produces several quality renditions of the same content. A playlist file written in the `.m3u8` format enumerates the available renditions, and the player selects among them in real time according to measured network throughput. This mechanism, known as Adaptive Bitrate Streaming (ABR), is what allows playback to degrade gracefully rather than stall when a connection weakens.

Producing those renditions is computationally expensive, and this is where the architectural difficulty arises. Transcoding is a bursty workload: it consumes a large amount of CPU while a video is being processed and nothing at all when no one is uploading. A platform that keeps a dedicated server cluster or a Kubernetes system running continuously simply to wait for work therefore pays for a substantial amount of idle capacity. The opposite failure is equally undesirable: a system provisioned only for the average case cannot absorb a sudden burst when many users upload at the same time, and uploads either queue for a long time or are dropped.

This tension between idle cost and burst capacity motivates the project. Rather than keeping compute permanently allocated, the system starts a container only when there

is a video to process and removes it immediately afterwards, so that cost accrues only while transcoding actually runs.

1.2 Project Objectives

The project aims to build a complete video sharing platform for end users, together with the automated transcoding subsystem that supports it, and has five objectives. The first is a web platform on which users register an account, upload videos, watch them at several quality levels, and share them as public, unlisted or private. The second is to automate the whole processing pipeline, from the moment a source file lands in Amazon S3, through transcoding and storage, to delivery through a content delivery network, until the video is ready to play in a browser. The third is to transcode with FFmpeg inside a Docker container on AWS Batch and AWS Fargate, created on demand and terminated automatically, so that idle resources cost nothing. The fourth is to automate deployment and infrastructure management through a CI/CD pipeline and Infrastructure as Code. The fifth is to evaluate the result empirically, in terms of quality of experience, transcoding performance, scalability under load and operating cost.

The first two objectives define what the platform does for its users, and the third how the processing is carried out; on-demand containers are the central hypothesis of the project. The fourth makes the system reproducible and safe to change, and the fifth turns the claims of the other four into measurements. Table 7.1 returns to each objective with its result.

1.3 Scope and Deliverables

The scope of this project covers the full vertical slice of a video sharing service: the browser-based user interface, the backend API, the transcoding engine, the cloud infrastructure that hosts them, and the automation that deploys them.

Several areas are deliberately placed outside the scope. The project does not implement digital rights management systems such as Widevine or FairPlay, which would be required to prevent an authorised viewer from redistributing content they can legitimately access. It does not implement live streaming, since the architecture is designed around video on demand. It also does not attempt per-title bitrate ladder optimisation, using instead a fixed ladder of six rungs from 144p to 1080p, of which each video receives those that its source resolution can fill; the trade-off involved is discussed in Chapter 7.

1.4 Research Methodology

The work combines a review of existing practice with an iterative build-and-measure cycle.

The theoretical component established how the problem is solved at industrial scale before any code was written. Published engineering accounts from Netflix describe how a large platform decomposes video processing into microservices and optimises a bitrate ladder per title, and the AWS Video on Demand reference architecture describes the managed-service equivalent. The protocol was taken from its specification, RFC 8216, and the encoding parameters from the FFmpeg HLS muxer documentation, rather than from secondary sources. Peer-reviewed studies of serverless video processing supplied the motivation for the architecture and the vocabulary of the evaluation, and the multi-threaded H.264 literature supplied the bound against which the vCPU scaling results of Chapter 6 are read.

The practical component followed an incremental development model. The backend API, the transcoding container, the frontend and the cloud infrastructure were built and integrated in turn, with the whole environment described as Terraform code so that every change to it is reviewable and reproducible rather than applied by hand. An automated test suite ran on every push through a continuous integration pipeline, which also runs dependency, container and infrastructure security scans.

Evaluation was empirical. Every performance figure in Chapter 6 was produced by a measurement script committed with the source code and run against the deployed system, with the raw output stored as JSON under `docs/results/`; Table A.1 maps each table to the run that produced it. Where a result could be an artefact of the measuring tool, it was reproduced with a second, independently implemented tool before being reported.

1.5 Practical Significance

The immediate beneficiaries of this design are small teams and individual creators who need a video platform but cannot justify a permanently running transcoding cluster. The cost comparison of Section 6.7 estimates that processing three hundred videos a month costs about $3.40 on the architecture built here against $133.35 on an always-on instance sized for the same work, because compute exists only while a job runs. That difference makes video hosting feasible for a service whose upload volume is intermittent, which describes most platforms before they become large.

The design also meets a requirement that is easy to state and easy to implement

incorrectly. Marking a video private means nothing if the underlying storage stays publicly addressable, and Chapter 4 shows that restricting the API alone does not achieve it. The two-layer scheme used here, a private bucket with authorisation enforced at the content delivery network, can be reused by any application that serves user-owned media from object storage.

1.6 Main Contributions

The principal contribution of this work is an end-to-end demonstration that serverless containers can serve as the transcoding backbone of a video platform, with a measured account of what that choice costs and what it saves. Six parts of it can be reused on their own. An upload event travels through Amazon S3, Amazon SQS and AWS Lambda to AWS Batch with no polling service in between. Conditional database writes make a duplicate delivery harmless, so that each job takes effect once without relying on exactly-once delivery. Origin Access Control and CloudFront Signed Cookies keep a private video unreachable even to a viewer who holds its manifest URL. A chunk-parallel path brings a four-hour source to playback in 135 minutes at the production quota. A moderation pipeline and a role-based console record every decision, so that the false positive and false negative rates of the screening can be measured. Finally, the environment is described in fourteen Terraform modules, and the DevSecOps pipeline blocks a merge on a fixable critical finding rather than only reporting it.

1.7 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 reviews comparable platforms and the models behind the design, and justifies the technology choices. Chapter 3 identifies the stakeholders, states the functional and non-functional requirements, and specifies the use cases. Chapter 4 describes the architecture, the transcoding pipeline, the database, content protection, moderation, the interface and the API. Chapter 5 covers the implementation of each subsystem, the infrastructure code and the delivery pipeline, the problems met during deployment, and the automated tests. Chapter 6 presents the deployed system and measures playback quality, transcoding performance, behaviour under load and operating cost, then sets the non-functional requirements against this evidence. Chapter 7 compares the results with the objectives and states the strengths, limitations and future work.

Chapter 2

Literature Review and Technologies

2.1 Related Platforms and Comparison

Understanding how established platforms solve the transcoding problem provides the necessary context for the design decisions taken in this project.

Netflix offers the most thoroughly documented case. Its original video processing system, internally known as Reloaded, was a monolithic pipeline that proved difficult to evolve as requirements changed. Netflix subsequently rebuilt it as a set of independent microservices running on a platform called Cosmos, decomposing the workload into a Video Inspection Service, a Video Encoding Service, and a Video Quality Service [1]. Cosmos itself is layered: an API tier receives requests, a workflow tier orchestrates the processing steps, and a serverless compute tier performs the actual media work, with the three tiers communicating asynchronously through a priority-based messaging system [2]. Two aspects of this design are directly relevant here. First, Netflix also relies on asynchronous messaging rather than synchronous calls to decouple job submission from job execution, which is precisely the role played by Amazon SQS in this project. Second, Netflix moved from linear encoding to distributed chunk-based encoding, allowing hundreds of machines to encode segments of the same title in parallel and reducing processing latency from days to hours. This project applies the same principle at a far smaller scale: for videos of five minutes or longer, a planner job divides the source into chunks that an array of AWS Batch jobs encodes concurrently and a finalizer job merges, with the degree of parallelism bounded by the Fargate vCPU quota rather than by a fleet of dedicated machines (Section 6.5).

Amazon Web Services publishes a reference architecture for video on demand that provides a useful point of comparison at the level of managed services [3]. That architecture routes an upload from Amazon S3 through an AWS Lambda job submitter into AWS Elemental MediaConvert, stores the output in a second S3 bucket, and delivers it through Amazon CloudFront, with AWS Step Functions orchestrating the workflow. The architecture presented in this report follows the same overall shape but substitutes a self-packaged FFmpeg container on AWS Batch for MediaConvert. This substitution is the central engineering trade-off of the project: MediaConvert is a fully managed service billed per minute of output and requires almost no operational effort, whereas a self-managed container is considerably cheaper per unit of work but transfers the responsibility for image maintenance, dependency updates, and failure handling onto the development team. AWS documentation also confirms that Origin Access Control is the currently recommended pattern for securing an S3 origin, superseding the legacy Origin Access Identity mechanism, which validates the approach adopted in Section 4.4.

Table 2.1 summarises the comparison across the dimensions most relevant to this work.

Table 2.1: Comparison of transcoding approaches across platforms

ASPECT	**NETFLIX (COSMOS)**	**AWS VOD REFERENCE**	**THIS PROJECT**
Compute model	Dedicated microservice platform with serverless compute tier	Managed service (MediaConvert)	Serverless container on AWS Batch / Fargate
Parallelism	Chunk-based, distributed across many nodes	Managed internally by the service	Chunk-based on AWS Batch array jobs from five minutes of video, one container below that
Bitrate ladder	Per-shot optimisation driven by VMAF	Configurable job template	Fixed ladder: six rungs from 144p to 1080p, limited by the source
Queueing	Priority-based messaging system	AWS Step Functions	Amazon SQS with dead letter queue
Content protection	DRM	Signed URLs or signed cookies	OAC plus CloudFront Signed Cookies
Operational effort	Very high	Very low	Moderate

2.2 Core Models and Principles

2.2.1 Serverless Containers

The term serverless container describes a model in which a container image is executed without the developer provisioning or managing the underlying virtual machines, and in which billing is based on execution time rather than allocated capacity. AWS Fargate is the implementation used in this project.

Figure 2.1 contrasts this model with the conventional alternative for a transcoding workload of the size assumed in Section 6.7. In the always-on arrangement on one side of the figure, a cluster is sized for the busiest moment the service expects and then runs continuously, so it is billed for all 730 hours in a month while performing roughly 105 vCPU-hours of actual encoding out of the 2,920 that a four-vCPU instance offers, a utilisation of under four percent. Its capacity is also a hard ceiling: an upload burst larger than the cluster was provisioned for must either wait or be rejected, and the only remedy is to provision for a still larger peak and waste correspondingly more. In the serverless arrangement on the other side, no compute exists between uploads. An upload event enters a queue, a container is started to serve it, and that container terminates when the job finishes, so charges accrue per vCPU-second of real work. The burst is absorbed by the queue rather than by spare capacity, and the degree of parallelism is a configured number rather than a fixed fleet size. The two figures shown, $124.10 against $0.05 of monthly compute, are the same workload priced under the two models.

The choice of Fargate over a function-as-a-service platform such as AWS Lambda deserves justification, because Lambda is the more common answer to serverless workloads. Video transcoding is unsuitable for Lambda in this context for two reasons. The first is duration: transcoding a large file routinely exceeds the maximum execution time of a Lambda invocation. The second concerns resource configuration. A measurement study of serverless video processing found that selecting a cost-efficient memory configuration for transcoding is far from trivial, because the optimal setting depends not only on the type of task but even on the content of the video itself, and the cost difference between the least and most generous memory configuration reached 498% for sequential workloads [4]. Fargate avoids this difficulty by allowing vCPU and memory to be specified explicitly and independently.

A known weakness of the container model is cold start latency. A systematic review of the subject reports that more than half of function invocations spend at least 80% of their startup time pulling the container image [5]. This finding directly motivates the use of a multi-stage Docker build in this project, described in Section 5.2, whose purpose is

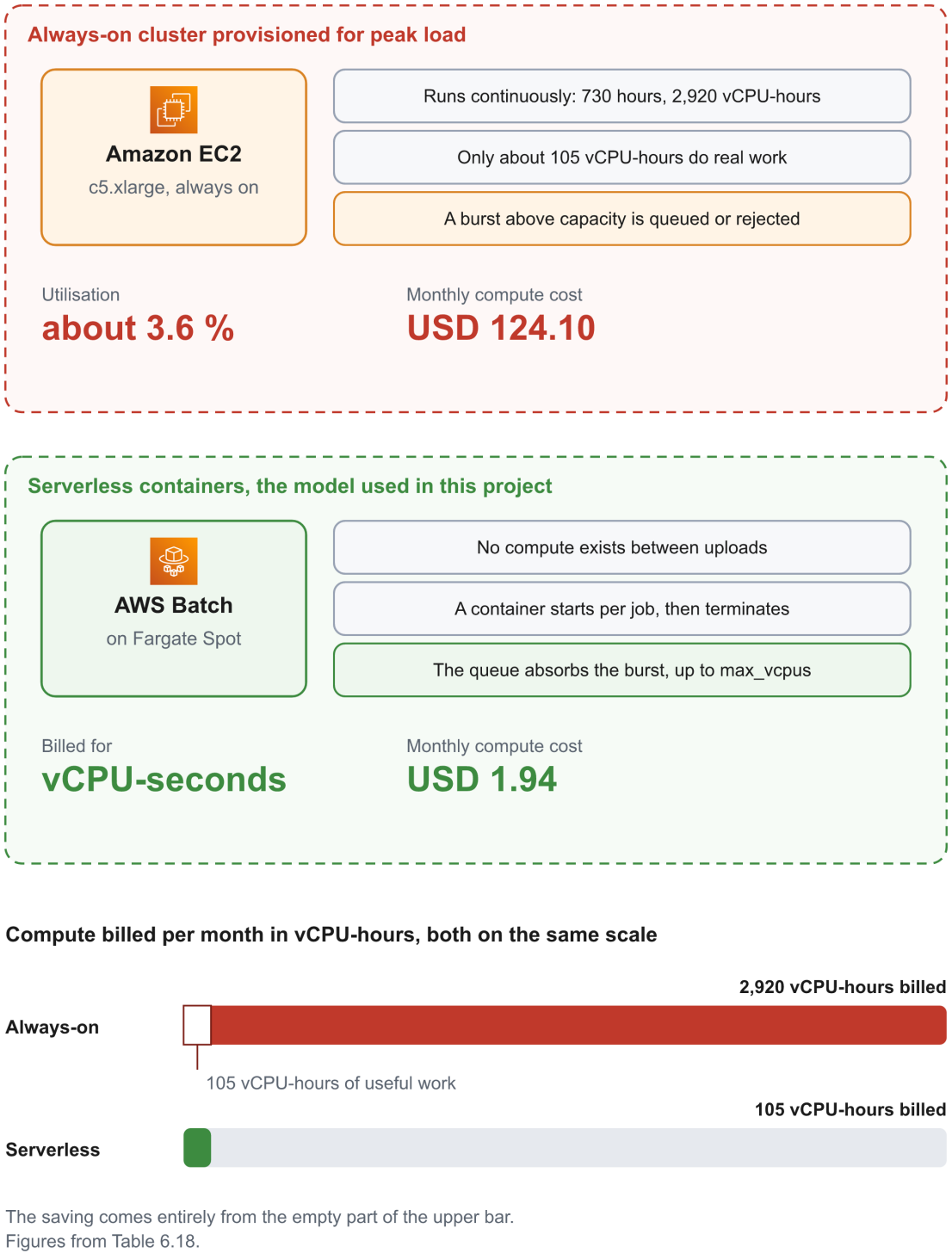


Figure 2.1: *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers*

to minimise the size of the image that must be transferred before work can begin.

2.2.2 Event-Driven Architecture

An event-driven architecture propagates work through the emission and consumption of events rather than through direct synchronous invocation. In this project, the completion of an upload to Amazon S3 emits an event notification that is placed on an Amazon SQS queue, from which an AWS Lambda function submits a job to AWS Batch.

The queue is not merely a routing mechanism; it is the component that makes burst absorption possible. If a hundred users upload simultaneously, the queue retains all hundred messages while AWS Batch works through them at whatever rate the compute environment allows. Without this buffer, the system would have to either reject work or provision for peak load permanently. The queue is paired with a dead letter queue so that messages which repeatedly fail processing are isolated for inspection rather than being retried indefinitely.

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder

HLS delivers a video as a master playlist referencing several rendition playlists, each of which lists the media segments for one quality level. The player measures throughput and switches between renditions at segment boundaries.

Figure 2.2 shows the artefacts this produces and the order in which a player retrieves them. A single source file of arbitrary container, codec and resolution is encoded once per rung of the ladder, and each rung yields both a rendition playlist and the sequence of transport stream segments it indexes. The master playlist sits above them and carries, for each rung, the bandwidth the client should expect to need in order to play it. A player therefore begins by fetching only the master playlist, which is a few hundred bytes, chooses a rung from the declared bandwidths and its own throughput estimate, and only then requests that rung's playlist and its segments. Because every rung is segmented on the same boundaries, the player may change its choice at any segment boundary without interrupting playback, which is the mechanism that lets quality fall rather than the video stall when a connection weakens. The consequence for the transcoding subsystem is that the cost of preparing a video scales with the number of rungs, since each rung is encoded separately even though the source itself is decoded only once. The implementation exploits exactly that distinction, as Section 5.2 describes.

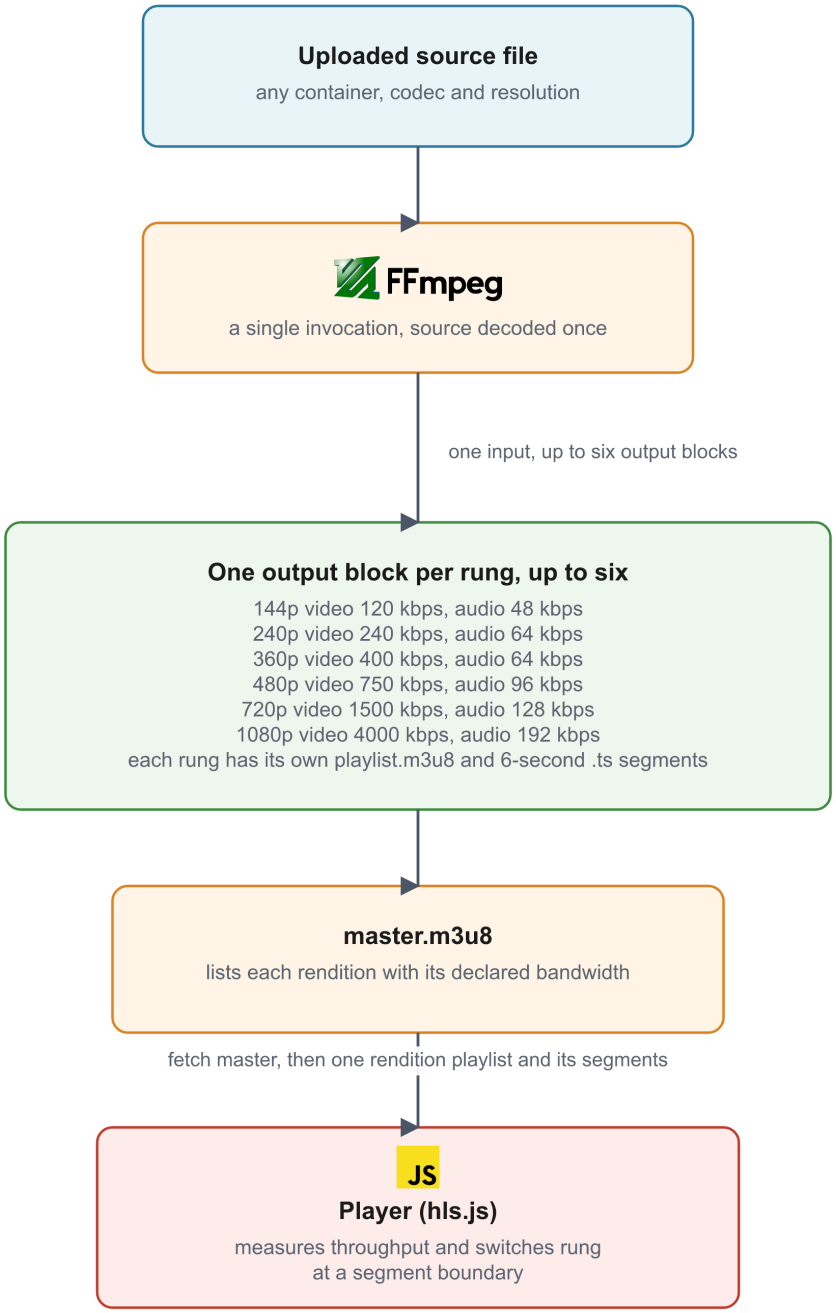


Figure 2.2: *HLS artefacts produced from one source file and the order in which a player retrieves them*

The set of resolution and bitrate pairs offered is called the bitrate ladder, and its construction is an active research area. Netflix demonstrated that the optimal ladder is not fixed but content-dependent, and that the ideal operating points lie on the convex hull of the quality-versus-bitrate curve [6]. By encoding each shot separately under the control of the VMAF perceptual quality metric, Netflix reported bitrate reductions of

between 28% and 38% at equivalent visual quality [7]. The present project uses a fixed ladder of six rungs, which is simpler to implement and reason about; the implications of this simplification are revisited in Section 7.4.

2.3 Technology Stack Justification

The frontend is implemented as a React single page application built with Vite, using HLS.js for playback. HLS.js is necessary because most desktop browsers do not support HLS natively; it parses the manifest in JavaScript and feeds segments to the browser through the Media Source Extensions API. Safari, which does support HLS natively, is handled through a fallback path.

The backend is built on Node.js with Express. The decisive factor is that the backend's workload is almost entirely input/output bound rather than CPU bound: it issues pre-signed URLs, queries MongoDB, and returns JSON, while the heavy computation is deliberately delegated to the transcoder containers. Node.js's event loop is well suited to this profile.

MongoDB Atlas provides the persistence layer. Video metadata is naturally document-shaped and its schema evolves as features are added, which suits a document store better than a rigid relational schema. Using the managed Atlas service also removes the need to operate a database cluster inside the project's own infrastructure.

Terraform was selected for Infrastructure as Code over alternatives such as AWS CloudFormation because its module system allows the infrastructure to be decomposed into reusable units with explicit inputs and outputs, and because its plan step makes the consequences of a change visible before it is applied. GitHub Actions was selected for CI/CD because the source code is already hosted on GitHub, which removes the need for a separate automation server.

Chapter 3

Requirements Analysis

This chapter identifies who uses the system and what each group needs, states the functional and non-functional requirements that follow, and models them as use cases, each specified step by step.

3.1 Stakeholders and User Requirements

The system serves three kinds of people: those who watch videos, those who upload them, and the staff who moderate and run the platform. Table 3.1 lists each group, what it needs, and the requirements of Sections 3.2 and 3.3 that answer those needs.

Table 3.1: Stakeholders, their needs and the requirements that serve them

STAKEHOLDER	NEEDS	REQUIREMENTS
Anonymous visitor	Browse, search and filter public videos and start watching at once, without registering, on any bandwidth	FR-05, FR-08, FR-10, FR-11, NFR-01
Registered user	Upload large files without waiting for processing; decide who can see each video; manage the channel; like, comment and report; appeal a removal	FR-01 to FR-04, FR-06, FR-07, FR-09, FR-13, FR-15, NFR-05
Operator	A queue of the videos that screening held back or viewers reported; a way to decide with a reason and to resolve appeals	FR-14, FR-15
Auditor	The error rates of the screening, with the uncertainty of each, and a log of decisions that cannot be altered	FR-16
Administrator	Assign the staff roles and draw random audit samples	FR-16, FR-17
System operator (the development team)	A platform that is observable, reproducible and deployable without manual steps, and that costs nothing while idle	NFR-02 to NFR-04, NFR-07, NFR-08

The visitor's need to watch without an account is deliberate: requiring registration before any content can be seen would deter adoption, so authentication is asked for only where an action needs an identity. The registered user plays two parts, viewer and uploader, and the staff roles are kept apart so that the person who reads the error rates is not the person whose decisions produce them (Section 4.5.5).

3.2 Functional Requirements

Table 3.2 lists the functional requirements of the platform. Each has an identifier so that it can be traced through the use cases, the design and the evaluation.

Table 3.2: Functional requirements of the system

ID	REQUIREMENT	DESCRIPTION
FR-01	Registration and login	Users register with a username, email address, and password. Authentication uses JSON Web Tokens carried in the Authorization header. A new account must prove that it can receive mail at its address before it may upload videos; signing in with Google counts as that proof. A new password, at registration, reset or change, is refused when it is a commonly used one, contains the user's own details or appears in a known breach.
FR-02	Password recovery	Users request a reset link by email; the token is single-use and expires after fifteen minutes. A completed reset also confirms the address.
FR-03	Direct upload to S3	The client requests a pre-signed URL from the backend and transfers the file directly to Amazon S3 without routing the payload through the application server. A file above 512 MiB is transferred in parts of 32 MiB, so that a part that fails is retried without repeating the whole file.
FR-04	Automatic transcoding	Once the upload completes, the system transcodes the source into HLS renditions without any manual trigger.
FR-05	Adaptive playback	The player switches automatically among the renditions produced for the video, chosen from six rungs between 144p and 1080p according to the source resolution, as the available bandwidth changes.
FR-06	Visibility control	Each video is public, unlisted, or private. A public video is listed in the catalogue; an unlisted one is reachable only by its direct link and appears in no listing; a private one is visible to its owner alone. The owner can change the setting after upload.
FR-07	Channel management	Users view their own uploads, manage visibility, update video details, and delete a video together with all of its stored artefacts.
FR-08	Search and filtering	Users search by title, description, or tags, and filter by category.
FR-09	Engagement	Users express likes and dislikes and post comments; a comment may be removed by its author or by the video owner.
FR-10	View counting	The system records a view only after playback has genuinely started, and does not count repeated views from the same viewer within a short window.
FR-11	Related videos	The watch page suggests further videos drawn from the same category and the same uploader. A suggestion is subject to the same visibility rule as the video itself, so the list never reveals one the viewer could not open directly.

Table 3.3 continues the list with the requirements of moderation and operation, which Section 4.5 designs.

Table 3.3: Functional requirements of moderation and operation

ID	REQUIREMENT	DESCRIPTION
FR-12	Automatic screening	Before a video is published, the system samples frames from it and classifies them; a video with clearly violating content is removed automatically, and one with doubtful content is held back for review. Screening that cannot be completed holds the video back and does not publish it.
FR-13	Viewer reports	A signed-in viewer can report a video for a reason chosen from a short list, and has at most one open report per video.
FR-14	Staff review	Operators approve or remove the held-back and reported videos, and every decision carries a reason from a closed list.
FR-15	Appeals	The owner of a removed video can appeal once per decision, an operator resolves the appeal with a note, and a person other than the owner cannot learn that the video exists.
FR-16	Accountability	Every decision is recorded together with what the automatic screening had concluded, so that the rates of false positives and false negatives can be estimated, the latter from a random audit sample, and an auditor can read them without being able to change a decision.
FR-17	Role management	An administrator assigns the roles operator, auditor and administrator, cannot change their own role and cannot demote the last administrator; every change is recorded.

Requirement FR-03 shapes the architecture more than any other. Routing a multi-gigabyte upload through the backend would consume server memory and bandwidth in proportion to the number of concurrent uploads and make the API server the bottleneck of the platform. A time-limited pre-signed URL lets the browser send the file directly to Amazon S3, so the backend handles only a small JSON request whatever the size of the file.

3.3 Non-Functional Requirements

The non-functional requirements are listed in Table 3.4. Section 6.8 sets each of them against the evidence of Chapter 6.

Table 3.4: Non-functional requirements of the system

ID	ATTRIBUTE	REQUIREMENT
NFR-01	Latency	Time-to-First-Frame should remain low enough that playback feels immediate; the measurement methodology is defined in Section 6.3.
NFR-02	Scalability	The pipeline must absorb a burst of concurrent uploads without losing work, processing them as capacity permits rather than rejecting them.
NFR-03	Reliability	A failed transcoding job must not silently disappear: it must be retried where retrying could succeed, and must raise an alert carrying the reason when it cannot. The retry and alerting mechanisms that satisfy this requirement on the deployed path are described in Section 5.5.
NFR-04	Cost efficiency	Compute cost must accrue only during actual transcoding, with no charge for idle capacity.
NFR-05	Security of content	Private videos must be inaccessible to unauthorised viewers even when the exact manifest URL is known.
NFR-06	Security of the API	Authentication endpoints must resist brute-force attempts, and the origin of cross-origin requests must be restricted to an explicit allow-list.
NFR-07	Reproducibility	The entire infrastructure must be reproducible from source control through a single command.
NFR-08	Maintainability	Changes must pass automated tests and security scanning before reaching the main branch.

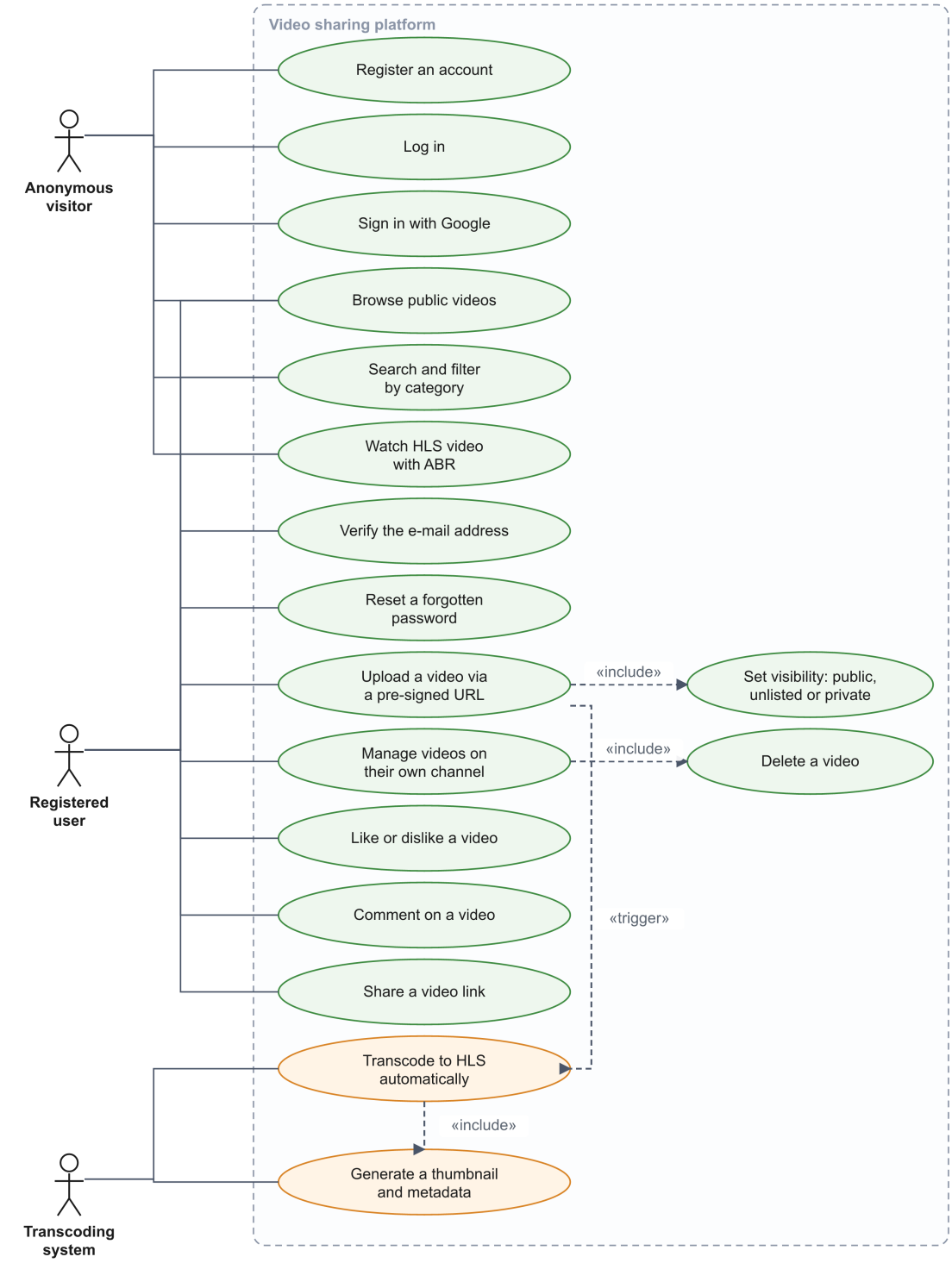
NFR-05 goes beyond ordinary access control. Restricting an API response is straightforward, but a video is not delivered by the API: a content delivery network delivers it as hundreds of segment files. Access control that protects only the API therefore leaves the content itself unprotected, which is the gap Section 4.4 closes.

3.4 Use Case Model

3.4.1 Use Case Diagrams

Figure 3.1 presents the use cases of the people who use the platform. Three actors appear: the anonymous visitor, the registered user, and the transcoding system, which is modelled as an actor because it starts processing in response to events rather than to a user's action.

The transcoding use cases are not invoked by the user at all. The upload triggers them indirectly through the chain of events, which is why they attach to the system actor, and it is also what lets the browser return control to the user as soon as the upload ends.



Association lines carry no arrowhead, as UML specifies. Only «include» and «trigger» relationships are directed. The transcoding use cases attach to the system actor because events start them, not a human action.

Figure 3.1: *Use case diagram of the video sharing platform*

Figure 3.2 adds the use cases of moderation and operation, requirements FR-12 to FR-17, designed in Section 4.5. They are drawn apart because adding them to Figure 3.1 would make it taller than a page. Three more actors appear, the roles operator, auditor and administrator; an administrator can also do everything an operator or an auditor can, so each association is drawn once. The transcoding system stays an actor because the screening runs inside the transcoder. Three things lead to a decision that only staff can take: what the screening holds back, a report from a viewer, and an appeal from the owner of a removed video.

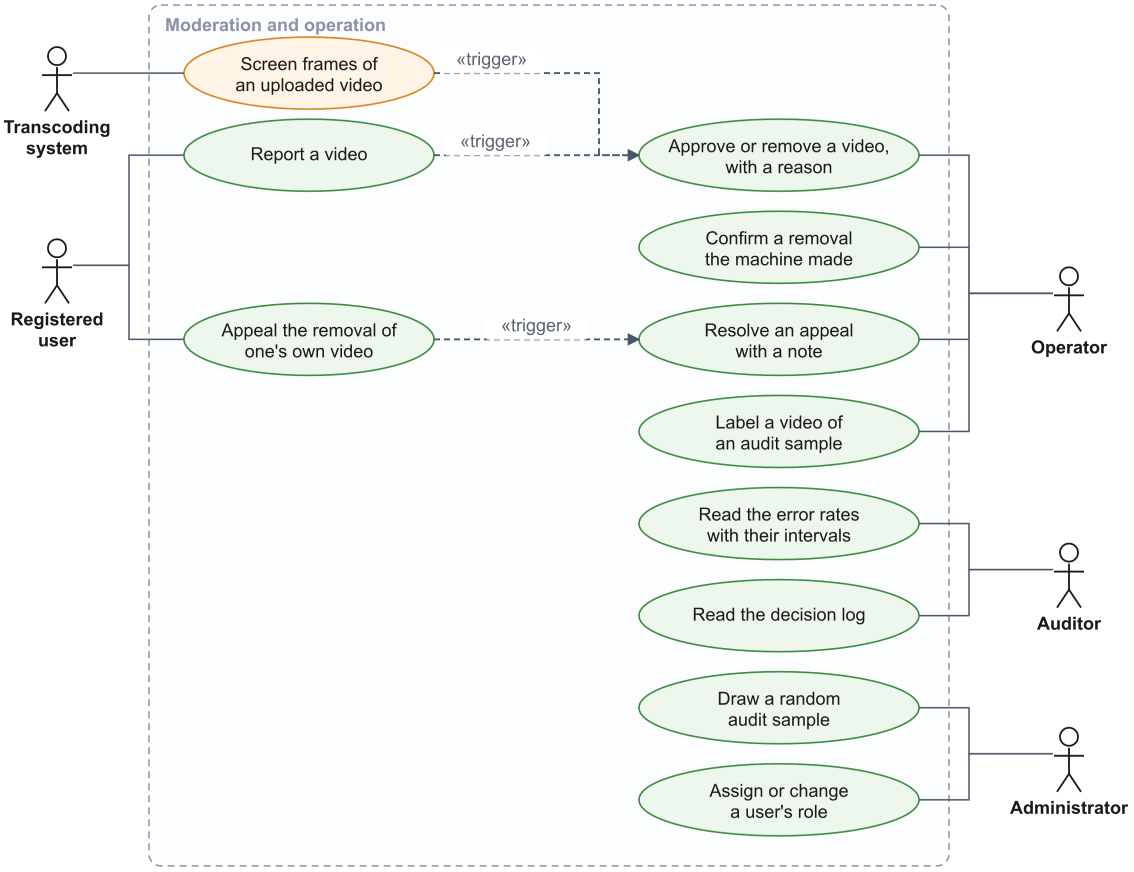


Figure 3.2: *Use case diagram of moderation and operation*

3.4.2 Use Case Specifications

The use cases of the two diagrams are specified in Tables 3.5 to 3.18. Closely related use cases of the diagrams are specified together, for example liking, commenting and sharing as one interaction, so that fourteen specifications cover the twenty-eight use

cases. Each lists its actor, the requirements it serves, the main flow and the alternative flows, numbered after the step they branch from, and was checked against the code of the routes and services it describes.

Table 3.5: Use case UC-01: register and verify the e-mail address

FIELD	**CONTENT**
Actor	Anonymous visitor
Requirements	FR-01
Precondition	No account uses the chosen username or e-mail address.
Main flow	1. The visitor enters a username, an e-mail address and a password. 2. The server checks the password against common passwords, the visitor's own details and the breach service. 3. The server creates an unverified account, stores the hash of a verification token valid for 24 hours, and returns a session token. 4. The server e-mails a link that carries the token. 5. The visitor opens the link; the page posts the token, and the server marks the address verified in one conditional update.
Alternative flows	1a. The username or the address is taken: the server answers 409 and creates nothing. 2a. The password is refused: the server answers 400 with a reason that the form shows. 4a. Five mails already went to the address this hour: the account is created, the mail is skipped, and the link can be sent again later. 5a. The link is malformed, used or expired: the server answers 400 and the user can ask for a new link.
Postcondition	The account exists; once the address is verified, it may upload.

Signing in, with a password or with Google, is specified in Table 3.6.

Table 3.6: Use case UC-02: log in

FIELD	CONTENT
Actor	Registered user
Requirements	FR-01, NFR-06
Precondition	The user has an account.
Main flow	1. The user enters the e-mail address and the password. 2. The server compares the password with the stored hash. 3. The server returns a JSON Web Token, which the client sends in the `Authorization` header of later requests.
Alternative flows	1a. The user signs in with Google instead: the server verifies the Google identity token; an unknown address creates a verified account, an unverified local account with the same address is taken over, and a verified one is refused with `EMAIL_IN_USE` (Figure 4.8). 1b. More than ten attempts in fifteen minutes from one address: the server answers 429. 2a. The address or the password is wrong: the server answers 401 with the same message for both, so the reply does not reveal which one was wrong.
Postcondition	The user holds a session token.

Table 3.7 covers a forgotten password; its reply never reveals whether an address has an account.

Table 3.7: Use case UC-03: reset a forgotten password

FIELD	CONTENT
Actor	Registered user who is not signed in
Requirements	FR-02
Precondition	None.
Main flow	1. The user enters the e-mail address. 2. The server gives the same generic answer whether or not an account exists; if one does, it stores the hash of a single-use token valid for fifteen minutes and e-mails a link. 3. The user opens the link and enters a new password. 4. The server checks the token and the password policy, stores the new password, marks the address verified, and returns a session token.
Alternative flows	2a. Five mails already went to the address this hour: the same generic answer, and no new token. 4a. The token is invalid or expired: the server answers 400 and the password is unchanged. 4b. The new password is refused by the policy: the server answers 400.
Postcondition	The password is changed, and tokens issued before the change are no longer accepted.

Two use cases are open to every visitor: browsing the catalogue (Table 3.8) and watching a video (Table 3.9).

Table 3.8: Use case UC-04: browse and search the catalogue

FIELD	CONTENT
Actor	Anonymous visitor or registered user
Requirements	FR-06, FR-08
Precondition	None.
Main flow	1. The visitor opens the catalogue. 2. The server returns, newest first, the first twelve videos that are public, `READY` and not held back or removed by moderation. 3. As the visitor scrolls near the end of the list, the page fetches the next twelve and appends them, so the catalogue scrolls without page numbers. 4. The visitor types a query or chooses a category tab, and the list starts again from the first twelve. 5. The server matches the query against the title, description and tags without regard to case, within the chosen category.
Alternative flows	3a. Loading the next videos fails: the page shows a button to try again. 5a. Nothing matches: the page shows an empty result with a message.
Postcondition	None; the catalogue is read only.

Table 3.9: Use case UC-05: watch a video

FIELD	CONTENT
Actor	Viewer, signed in or not
Requirements	FR-05, FR-06, FR-10, FR-11, NFR-01, NFR-05
Precondition	The viewer has the link of the video.
Main flow	1. The viewer opens the watch page. 2. The server returns the video, applying the access rule of Section 5.1. 3. The page requests playback authorisation and receives Signed Cookies for that video (Figure 4.6). 4. The player loads the master playlist from CloudFront and switches renditions as the bandwidth changes. 5. After thirty seconds of actual playback, or at the end of a shorter video, the page reports a view, which the server counts unless the same viewer was counted within thirty minutes or the viewer owns the video. 6. A sidebar lists related videos from the same category and uploader.
Alternative flows	2a. The video is private and the viewer is not its owner: the server answers not-found. 2b. The video was removed, or is held back for review: the server answers 403 with a distinct code, and the page shows a screen that explains it. 4a. The cookies expire during playback: the player asks for new cookies and reloads the source.
Postcondition	The view count may have grown by one.

Table 3.10 groups the interactions of a signed-in viewer.

Table 3.10: Use case UC-06: like, comment on and share a video

FIELD	CONTENT
Actor	Registered user
Requirements	FR-09
Precondition	The user is signed in and may see the video.
Main flow	1. The user presses like or dislike. 2. The server adds the user to that list and removes the user from the other in one update; pressing again removes the reaction. 3. The user writes a comment of at most 1000 characters. 4. The server stores it, and the page shows it at the top of the list. 5. The user presses Share, and the page copies the link of the video.
Alternative flows	3a. The comment is empty or too long: the server refuses it. 4a. The author of a comment, or the owner of the video, deletes it; anyone else is refused with 403.
Postcondition	The reaction counts and the comments are updated.

Uploading, the use case that shapes the architecture most, is specified in Table 3.11, and managing the uploaded videos in Table 3.12.

Table 3.11: Use case UC-07: upload a video

FIELD	CONTENT
Actor	Registered user
Requirements	FR-01, FR-03, FR-04
Precondition	The user holds a valid JSON Web Token and a verified e-mail address.
Main flow	1. The user selects a video file and submits the upload form; the client checks the file type and size. 2. The client calls the initiate-upload endpoint with the file metadata. 3. The server validates the metadata, creates a database record with status `UPLOADING`, and returns a pre-signed URL valid for fifteen minutes. 4. The browser transfers the file directly to the raw S3 bucket. 5. The client confirms completion and the record moves to `PROCESSING`. 6. The transcoding pipeline runs asynchronously and eventually sets the record to `READY`.
Alternative flows	3a. The file type is not an accepted video format or the size exceeds the limit: the server refuses before issuing a URL, and no orphan record is created. 3b. The file is larger than 512 MiB: step 3 returns a URL for each part of 32 MiB, and the server checks the stored parts before the object is assembled. 3c. The e-mail address is not verified: the server refuses with `EMAIL_NOT_VERIFIED`, and the interface offers to send the link again.
Postcondition	The video appears on the user's channel, first as processing and then as ready to play.

Table 3.12: Use case UC-08: manage one's own videos

FIELD	CONTENT
Actor	Registered user who owns the videos
Requirements	FR-06, FR-07
Precondition	The user is signed in.
Main flow	1. The user opens the channel management view, which lists every own video with its status, including those still processing. 2. The user edits the title, description, category, tags or visibility of a video, and the server applies the change. 3. The user deletes a video. 4. The server removes the source file, the renditions and the thumbnail, the comments and the reports, and then the record.
Alternative flows	2a. The video does not belong to the user: the server answers not-found. 4a. The video is still uploading: its multipart upload is aborted first.
Postcondition	The catalogue and the watch page reflect the change.

Table 3.13 specifies the use case of the transcoding system, which no person starts.

Table 3.13: Use case UC-09: transcode a video automatically

FIELD	CONTENT
Actor	Transcoding system
Requirements	FR-04, FR-12, NFR-02, NFR-03
Precondition	An upload has been confirmed and the record is `PROCESSING`.
Main flow	1. Amazon S3 emits an event, which Amazon SQS buffers. 2. The Lambda submitter submits an AWS Batch job and deletes the message. 3. The job checks that the video is not already `READY` and reads the source. 4. A video shorter than five minutes is encoded in one container; a longer one is split into chunks encoded in parallel (Section 4.2.2). 5. The job writes the renditions, the master playlist and the thumbnail, screens the frames, and sets `READY` with the verdict in one conditional write. 6. The owner receives an e-mail saying that the video is ready, under review or removed, according to the verdict.
Alternative flows	3a. A duplicate job finds the video `READY` and stops. 5a. FFmpeg rejects the content: the record becomes `ERROR` and the owner is notified. 5b. An infrastructure fault occurs: AWS Batch retries the job up to three attempts, after which an alert carries the reason.
Postcondition	The video is `READY`, and public if screening approved it, or `ERROR`.

The remaining use cases belong to moderation. A viewer's report (Table 3.14) and the screening both lead to an operator's decision (Table 3.15).

Table 3.14: Use case UC-10: report a video

FIELD	CONTENT
Actor	Registered user
Requirements	FR-13
Precondition	The user is signed in and may see the video.
Main flow	1. The user chooses Report and one of eight reasons, such as sexual, violent, hateful or spam, with optional details. 2. The server records the report and increases the video's count of open reports, which places the video in the review queue.
Alternative flows	1a. The video is the user's own: the server answers 400. 1b. Too many reports in a short time: the server answers 429. 2a. The user already has an open report on this video: no second report is created.
Postcondition	An open report waits for an operator.

Table 3.15: Use case UC-11: decide on a video

FIELD	CONTENT
Actor	Operator or administrator
Requirements	FR-14, FR-16
Precondition	The person holds the permission to review.
Main flow	1. The operator opens the review queue of held-back and reported videos, most reported first, and may play a video and read its reports. 2. The operator approves or removes the video with a reason code from the closed list for that decision. 3. The server copies the machine's verdict into `autoStatus` if no person has decided before, sets the verdict, closes the open reports and appends an event with its outcome class (Table 4.5).
Alternative flows	2a. For a video that the machine removed, the operator confirms the removal, which counts as a true positive. 2b. The reason code does not belong to the decision: the server answers 400. 3a. An operator tries to change an administrator's decision: the server answers 403.
Postcondition	The video is shown or hidden accordingly, and the decision is in the log.

The owner of a removed video can then appeal (Table 3.16).

Table 3.16: Use case UC-12: appeal a removal

FIELD	CONTENT
Actors	Owner of the video; operator
Requirements	FR-15
Precondition	The video has been removed, and its owner has not yet appealed this decision.
Main flow	1. The owner writes an appeal of at most 500 characters on the removed video. 2. The server records it, one per decision. 3. An operator reads the appeal, watches the video, and upholds or rejects it with a note. 4. Upholding approves the video with the reason “appeal upheld”, which restores it; rejecting keeps the removal.
Alternative flows	1a. The requester does not own the video: the server answers not-found. 1b. The video is not removed: the server answers 400. 2a. The decision has already been appealed: the server answers 409.
Postcondition	The appeal is resolved, and the owner sees the outcome on the video.

Table 3.17 specifies the random audit that estimates what the screening misses, and Table 3.18 the assignment of roles.

Table 3.17: Use case UC-13: audit a random sample and read the error rates

FIELD	CONTENT
Actors	Administrator; operator; auditor
Requirements	FR-16
Precondition	Each person holds the permission for their step.
Main flow	1. The administrator draws a sample of 1 to 100 videos among those that the machine approved, that have no open report and that were never sampled. 2. An operator watches each sampled video and labels it pass or fail. 3. A video labelled fail is removed with the reason “audit failed”. 4. The auditor reads the four rates of Section 4.5, each with its 95% interval and sample size, and the decision log.
Alternative flows	1a. The size is outside 1 to 100: the server answers 400. 2a. The sample was already labelled: the server answers 404.
Postcondition	The false negative rate includes the new labels.

Table 3.18: Use case UC-14: assign a role

FIELD	**CONTENT**
Actor	Administrator
Requirements	FR-17
Precondition	The person is signed in as an administrator.
Main flow	1. The administrator opens the Users tab and finds the user. 2. The administrator chooses user, operator, auditor or administrator. 3. The server changes the role and appends a record of who changed whom, from which role to which. 4. The change takes effect on the user's next request, since the role is read from the database every time.
Alternative flows	2a. The administrator chooses their own account: the server answers 403. 2b. The change would demote the last administrator: the server answers 409.
Postcondition	The role is changed and the change is logged.

Chapter 4

System Design

This chapter describes the architecture of the system, the transcoding pipeline, the database, the protection of private content, the moderation of uploads, the interface and the API.

4.1 High-Level Architecture

The system is organised into four tiers, shown in Figure 4.1: a client tier running in the browser, a delivery tier built on Amazon CloudFront, an application tier comprising the backend API and the database, and a processing tier that performs transcoding.

The separation between the application tier and the processing tier is the defining characteristic of this design. The backend API never touches video content: it issues pre-signed URLs, records metadata, and answers queries, while every byte of video flows directly between the browser, Amazon S3, and CloudFront. This separation has three consequences that together justify the architecture. The API server's resource consumption becomes independent of video size and upload concurrency. The processing tier can scale according to transcoding demand without affecting the responsiveness of the interface. And a failure in the processing tier degrades the system gracefully, leaving already-published videos playable while newly uploaded ones remain in a pending state.

Two later additions sit inside this structure and do not change it. Content moderation runs in the processing tier, as the last step before a video is published, so that the check neither delays the upload nor adds a component to operate (Section 4.5). And the console that human reviewers use is a role-restricted part of the same single page application and the same backend API, not a separate system.

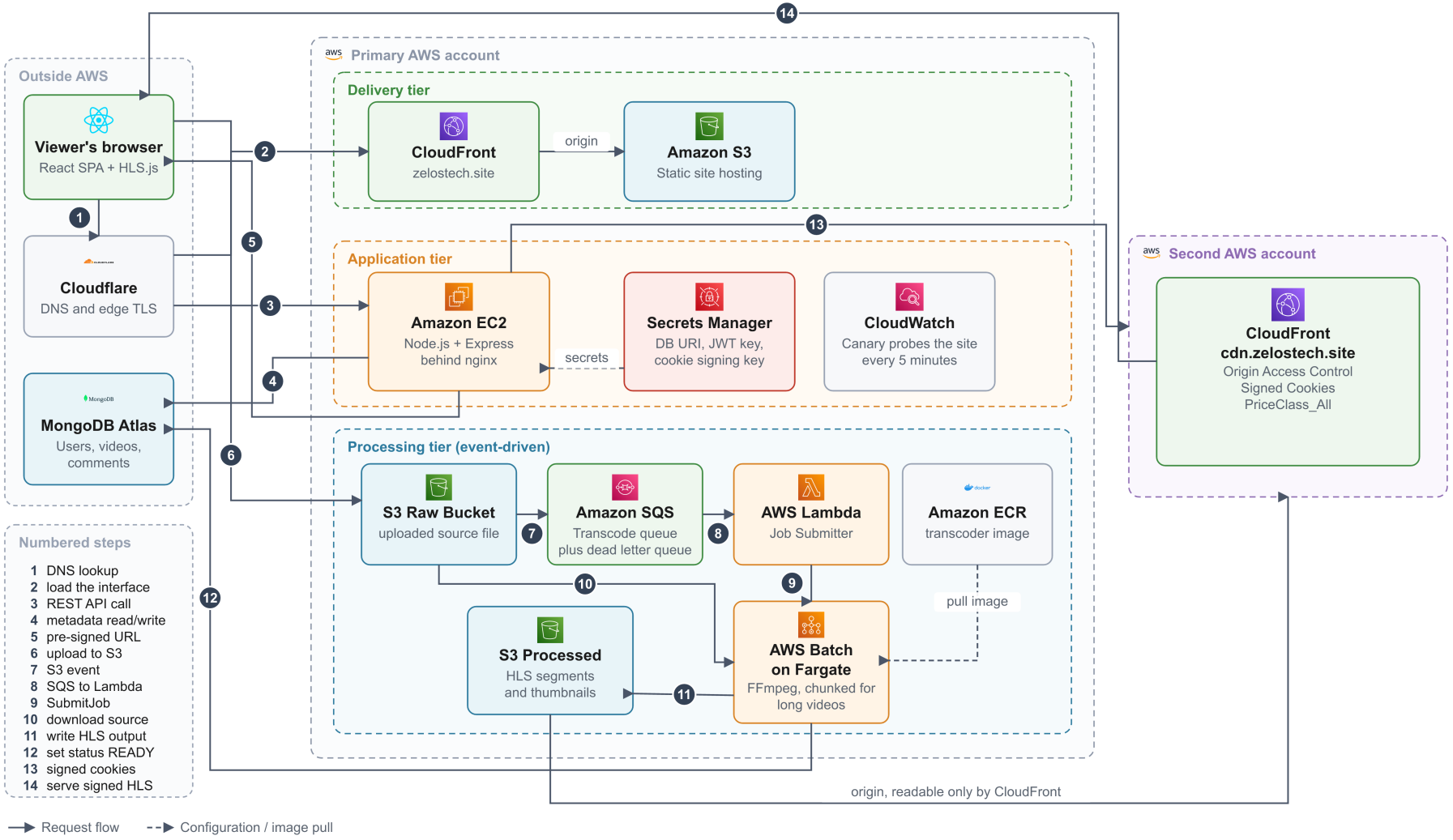


Figure 4.1: *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery; the key at the lower left names each step*

4.2 Transcoding Pipeline Design

Figure 4.2 traces the sequence of interactions from the moment the user initiates an upload until the video becomes available.

Two design decisions in this sequence need explanation. The first is the order of record creation and URL issuance. The backend creates the database record *before* it generates the pre-signed URL, and derives the S3 object key from the record's identifier. Generating the key first would create a race: the S3 event could start transcoding before the record exists, and the job would have no row to update. Creating the record first removes this failure entirely.

The second is protection against processing a video twice. The transcoder has two modes. In `worker` mode, used during development, the container reads the SQS queue itself and, while FFmpeg runs, keeps extending the visibility timeout of its message, a heartbeat, so that SQS does not hand the same message to another consumer; the message is deleted only after success. The deployed system runs `batch` mode: the Lambda submitter reads the message, submits an AWS Batch job and deletes the message at once, and the container receives the video identifier as an environment variable. On the deployed path there is therefore no heartbeat. Protection against a duplicate rests on conditional database writes, which is where it belongs in any case, since a renewal timer alone cannot guarantee mutual exclusion (Section 5.2), and retry is provided by AWS Batch (Section 5.5). Worker mode is kept because it lets the transcoder run against the queue without provisioning Batch.

4.2.1 Upload of Large Files

The sequence of Figure 4.2 uses one pre-signed request, which suits files up to a few hundred megabytes. For larger files the design switches to a multipart upload, because a single transfer that fails near its end must be repeated in full and Amazon S3 accepts at most 5 GiB in one request. The backend chooses the path from the declared size: a file of up to 512 MiB is sent with one pre-signed POST, and a larger file is divided into parts of 32 MiB.

Both paths keep the payload away from the application server, and both let the storage service, not the client, enforce the limits. For the single request, the signed policy fixes the object key, the content type and the size range. For a multipart upload the backend signs one URL per part for exactly that part's length, so S3 itself refuses a part of any other size, and when the client reports completion the backend lists the parts that S3 holds before asking S3 to assemble the object. The ceiling on file size is a configuration

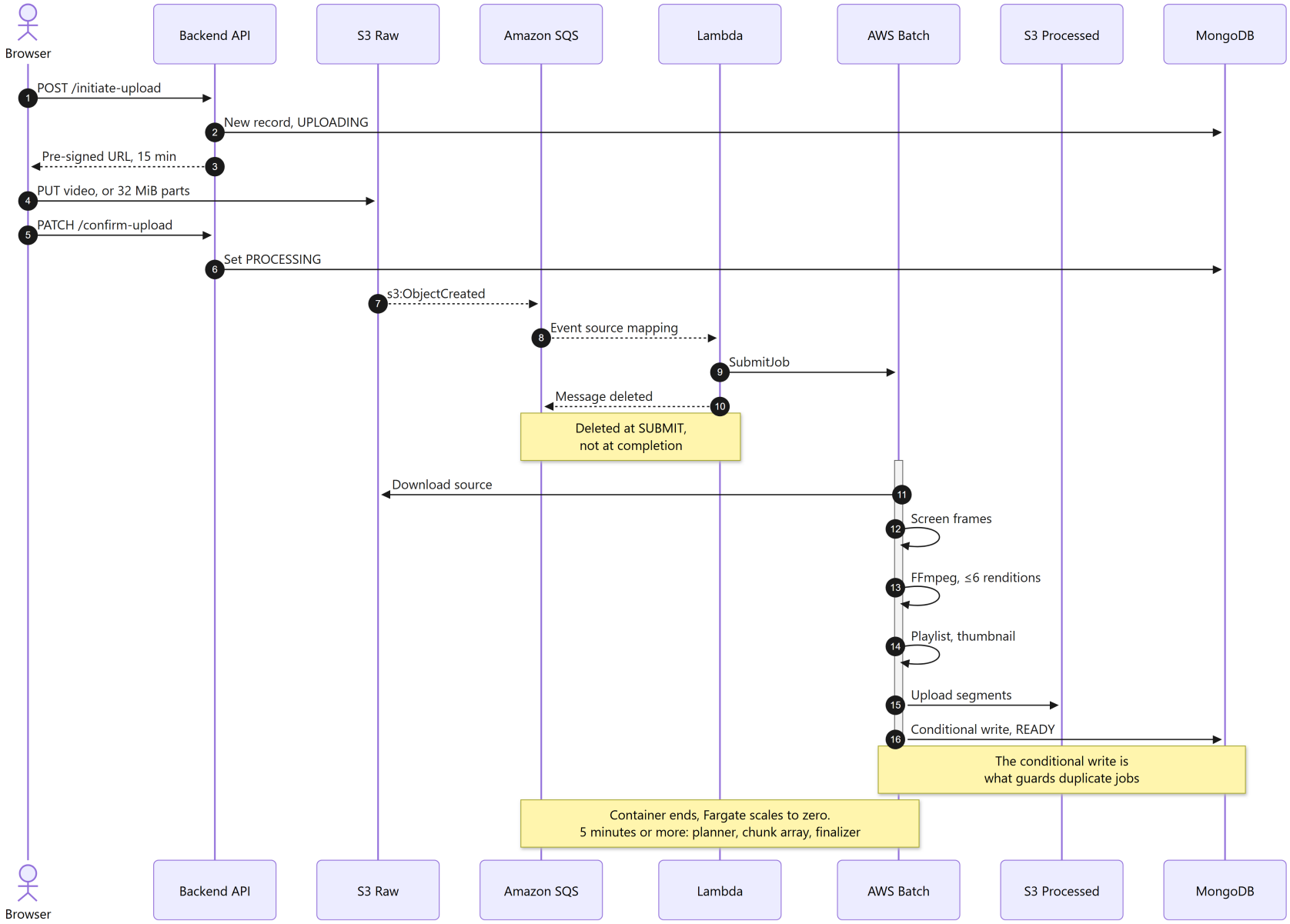


Figure 4.2: *Sequence diagram of the event-driven transcoding pipeline for one container; a video of five minutes or more takes the chunked path of Table 4.1*

value with a hard limit of 20 GiB, which is 640 parts of 32 MiB, and the deployed system uses that limit. An abandoned upload is cleaned up twice over: a lifecycle rule aborts incomplete multipart uploads after two days, and a periodic reconciler removes a draft that has stayed in the `UPLOADING` state for more than a day.

4.2.2 Chunk-Parallel Transcoding of Long Videos

The pipeline of Figure 4.2 gives each video to one container. That suits short videos and fails for long ones: on a single vCPU, FFmpeg ran four to five times slower than real time (Sections 6.4 and 6.5), so a long source would exceed the two-hour limit of a job however many vCPUs the account holds. The design therefore divides a video of five minutes or more into chunks that are encoded concurrently, and keeps the single-container path for shorter videos, where splitting costs more than it saves.

The job that the Lambda submitter starts becomes a planner. It reads the structure of the source over HTTP without downloading it and divides the video at boundaries that are whole multiples of a six-second group of pictures, counted in frames so that a rate such as 29.97 frames per second does not accumulate a fractional error. It then submits the jobs of Table 4.1 to the same AWS Batch service.

Table 4.1: Jobs of the chunked path

JOB	COUNT	TASK SIZE	RESPONSIBILITY
Planner	1	Default	Probes the source, computes the chunk boundaries, and submits the other jobs
Audio	2	Default	Encodes the audio once for the whole video, one job per bitrate (64 and 128 kbit/s)
Chunk	One per chunk, as an array job	4 vCPU, 8 GiB	Reads its own range of the source over HTTP and encodes every rendition of the ladder for that range
Finalizer	1	Default	Runs after every chunk has ended; joins the playlists, measures the bandwidth of each rendition from the segments, extracts the thumbnail, screens the content, and sets `READY`

Three properties of this design are deliberate. The chunk array goes to a second, lower-priority job queue that shares the Fargate Spot environment, so a short video submitted while a long one is encoding runs ahead of the remaining chunks. The number of chunks follows the number of tasks that can run at once, about three chunks per slot and none shorter than about a minute, so a larger quota produces smaller chunks. And

the planner does not split a source that cannot be split safely, meaning a variable frame rate, an unknown duration, or audio and video that differ by more than 0.25 seconds; such a video falls back to the single-container path and is not failed.

While a chunked video is encoding, its child jobs periodically touch the `updatedAt` field of the video, because the reconciler that resets a video stuck in `PROCESSING` for more than six hours would otherwise mistake a long, healthy encode for a stuck one. How this path performs is measured in Section 6.5.

4.2.3 HLS Transcoding Pipeline and Multi-Variant Segmentation

The generation of HTTP Live Streaming (HLS) artefacts represents the core computational burden of the platform. To deliver seamless Adaptive Bitrate Streaming (ABR) under changing client bandwidth, the transcoding worker decomposes the source media into a strictly aligned ladder of synchronized video variants and audio tracks, illustrated in Figure 4.3.

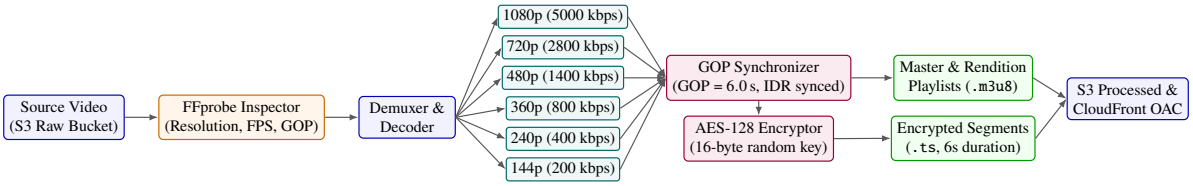


Figure 4.3: *Dedicated HLS Transcoding Pipeline, illustrating stream probing, multi-variant filtergraph scaling, synchronized GOP keyframe alignment, and AES-128 encrypted segment packaging (bitrates denote maximum rendition ceilings)*

The pipeline executes sequentially through five distinct transformation stages. First, in the stream inspection and probing stage, the worker invokes `ffprobe` to extract video dimensions, frame rate, duration, and audio track layouts. If the source resolution is lower than 1080p, the ladder is automatically bounded at the source's native resolution, preventing wasteful upscaling. Second, during single-decode multi-scale scaling, the input video stream is decoded exactly once into uncompressed raw frames; these frames are piped through a multi-branch filtergraph that scales them concurrently into the standard resolution tiers (from 1080p down to 144p).

Third, to ensure seamless Adaptive Bitrate Switching (ABR), strict keyframe (GOP) synchronization is enforced: all renditions must place instantaneous decoder refresh (IDR) keyframes at identical frame timestamps. FFmpeg is configured with `-g (fps*6) -keyint_min (fps*6) -sc_threshold 0`, enforcing identical six-second segment boundaries across all ladder rungs. Fourth, for copyright protection, envelope encryption generates a cryptographically secure 16-byte symmetric key via

crypto.randomBytes(16) and supplies it through FFmpeg's -hls_key_info_file. Every .ts transport stream segment is encrypted using AES-128 in Cipher Block Chaining (CBC) mode, and the key delivery URI embedded in the variant playlists points client players to the authorized licensing endpoint (/api/videos/:id/key). Finally, the HLS muxer assembles the hierarchical playlists: variant playlists index the six-second segments with precise #EXTINF:6.000000 duration tags, and a master playlist (master.m3u8) is written with #EXT-X-STREAM-INF descriptors declaring the resolution, codecs, and bandwidth ceilings of each rendition.

4.3 Database Design

The persistence layer uses MongoDB Atlas with eight collections. Three of them, User, Video and Comment, model the platform itself and are shown in Figure 4.4; Table 4.2 lists their keys, indexes and constraints. The other five exist to moderate and operate the platform and are described after the state machine of the video.

Table 4.2: Keys, indexes and constraints of the core collections

COLLECTION	KEYS AND REFERENCES	INDEXES	CONSTRAINTS
User	_id; username and email unique; googleId unique where present (sparse)	The three unique indexes	Username of 3 to 30 characters; e-mail stored in lower case and checked for form; password of at least 8 characters, required unless the account has a Google identity; role one of four, user by default
Video	_id; user refers to User and is required; likes and dislikes hold User identifiers	Visibility, status and creation time for the catalogue; owner with creation time, and owner with views, for channel pages; status; category; tags; moderation state, and open reports, for the review queue	Title required, at most 100 characters; description at most 5000; category one of seven fixed values; status one of four; visibility public, unlisted or private
Comment	_id; video refers to Video and user to User, both required	The video, to list a video's comments	Content required, at most 1000 characters

The Video document carries a status field that acts as a state machine with four values. A record begins as UPLOADING when the pre-signed URL is issued, moves to

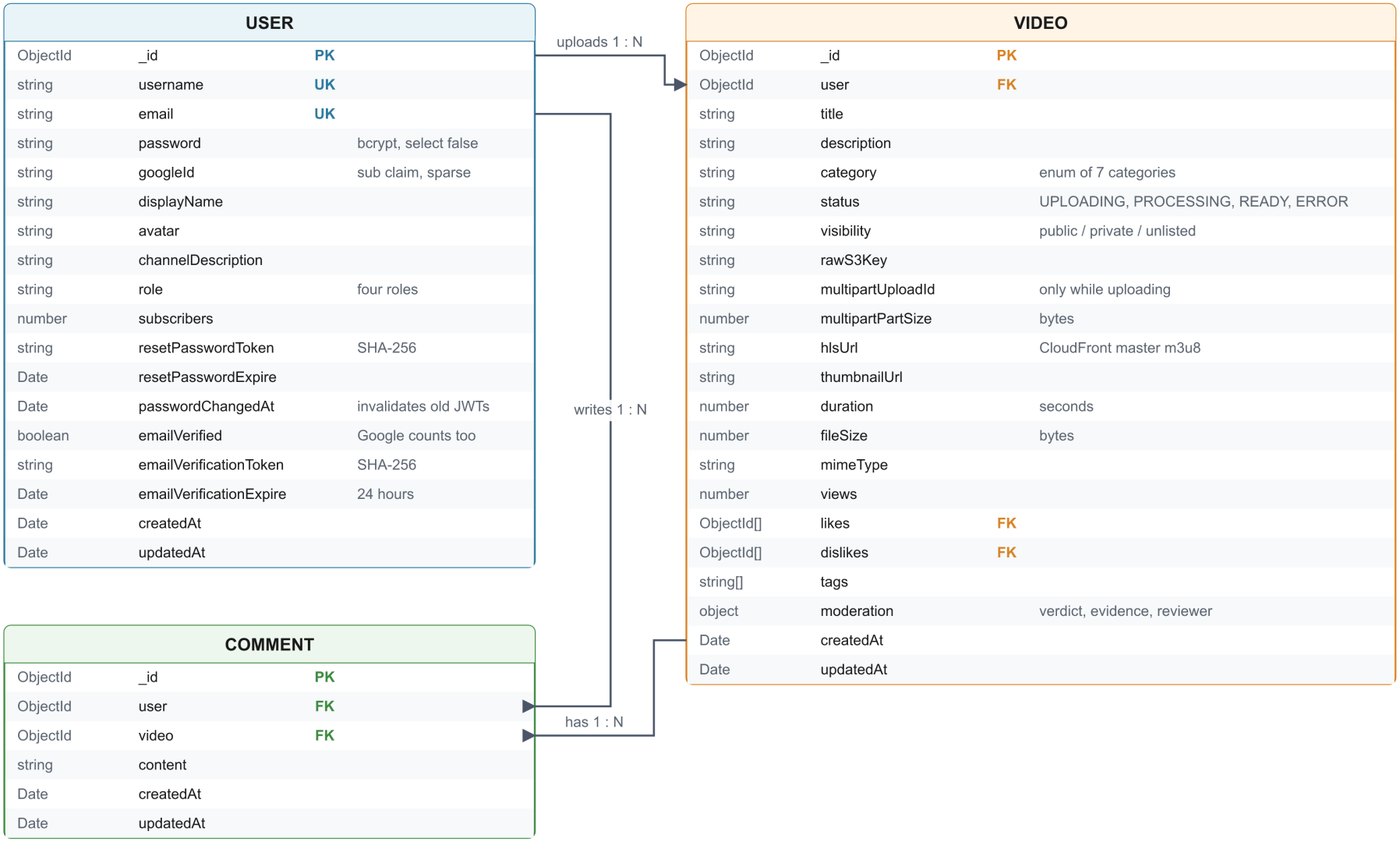


Figure 4.4: *Entity relationship diagram of the core MongoDB collections*

PROCESSING once the upload is confirmed and the job is submitted, and finally reaches either READY or ERROR. Modelling the lifecycle explicitly, rather than inferring it from other fields, makes the interface simple: the client renders one state per value, and a video that failed to transcode is distinguishable from one that is merely slow.

A second state, independent of the first, lives in the moderation sub-document of Video: the verdict (approved, flagged or blocked), whether it came from the machine or from a person, the labels that were its evidence, and autoStatus, the machine's own conclusion, written exactly once before any person first changes the verdict (Section 4.5). The five collections that support moderation are summarised in Table 4.3 and shown in Figure 4.5.

Table 4.3: Collections that support moderation and operation

COLLECTION	ONE DOCUMENT IS	CONSTRAINT
Report	A viewer's report of a video	At most one open report per viewer and video, by a partial unique index
ModerationEvent	One decision by a person	Append only: no route edits or deletes it
Appeal	An owner's appeal against a removal	One per video and decision time, by a unique index
AuditSample	A video drawn into a random audit	A video enters at most once, by a unique index on the video
RoleChange	A change of one user's role	Append only: who changed whom, from which role to which

Likes and dislikes are stored as arrays of user identifiers inside the video document rather than as a separate collection, because the counts are always read with the video and a user can appear at most once in each array. This holds at the scale of the project only. On a widely viewed video the arrays grow towards MongoDB's document size limit, and a listing of twelve videos reads every identifier in all twenty-four arrays merely to show counts. The usual remedy, a separate collection with a compound unique index on video and user plus a counter on the video, is the right structure for popular content; it was not adopted because of scale, not because the embedded form is better.

Search matches a case-insensitive regular expression against the title, description and tags. No index can serve an unanchored regular expression, so every search scans the collection, which is acceptable at this scale. A text index would be faster but would change the behaviour, since it matches neither parts of words nor the tags; an unused text index that the schema once declared was therefore removed.

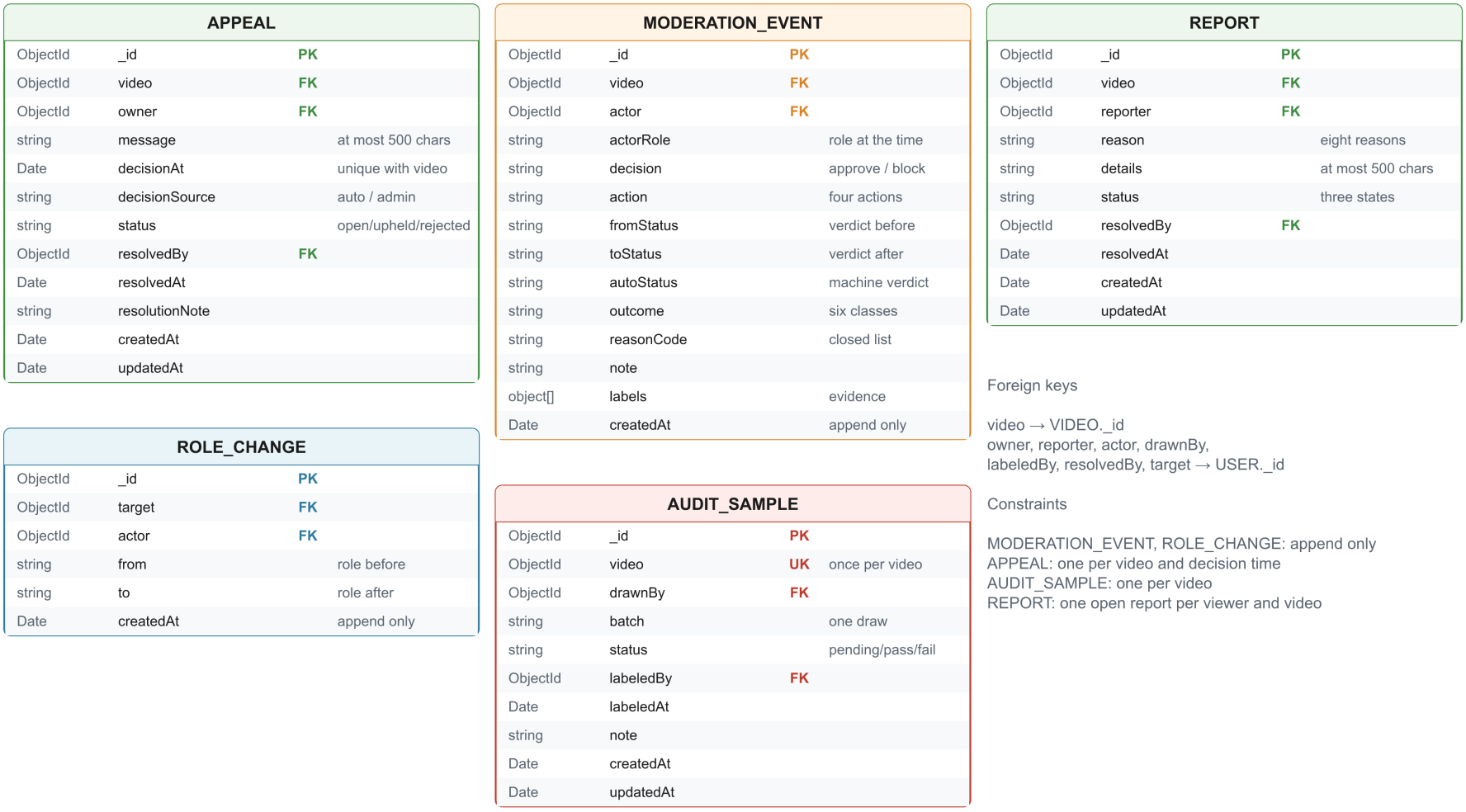


Figure 4.5: *Entity relationship diagram of the collections that support moderation and operation; every video field refers to* `Video` *and every user field to* `User`

4.4 Content Protection Design

Protecting private content requires two mechanisms, because each addresses a different threat.

Origin Access Control makes the processed S3 bucket private and grants read access only to the CloudFront distribution through a bucket policy. A viewer therefore cannot bypass the content delivery network and fetch objects from S3 directly, which would both escape any access control at the edge and incur higher transfer charges.

Origin Access Control alone does not decide *who* may retrieve content through CloudFront. Without a second mechanism, anyone holding a manifest URL could download a video whatever its visibility, which would make the private mode ineffective. The system therefore adds CloudFront Signed Cookies backed by a trusted key group [8]. The backend checks the requester's permission with the same service function that governs the API response, signs a policy scoped to the directory of that one video, and sets the cookies on the response; CloudFront refuses any request without a valid signature with HTTP 403 at the edge. Figure 4.6 shows the exchange.

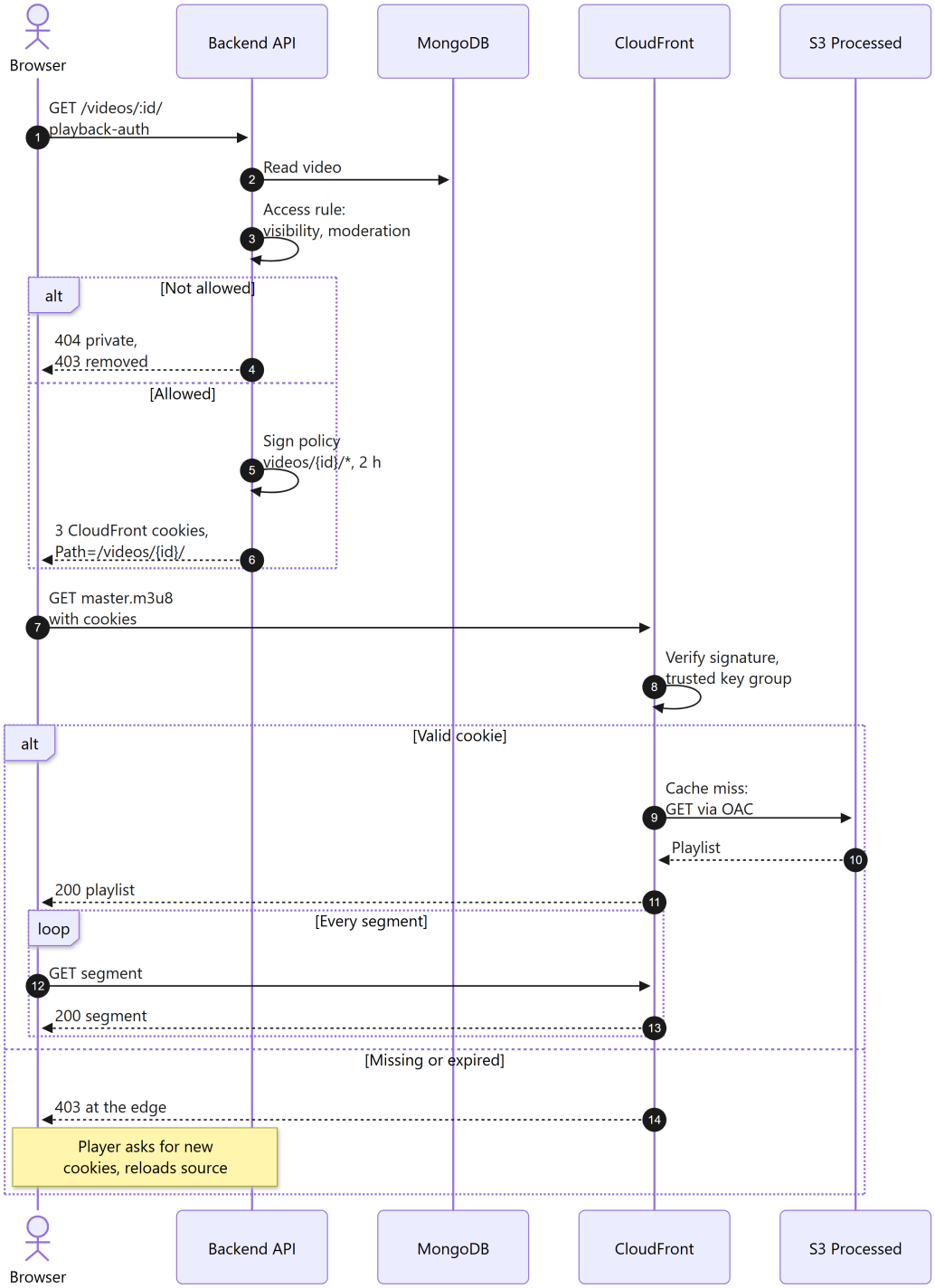


Figure 4.6: *Sequence diagram of playback authorisation: the backend issues Signed Cookies for one video, and CloudFront checks them on every request*

Signed cookies were chosen over signed URLs because of the structure of HLS. A playback session fetches a manifest and then hundreds of segments, each as a separate request. Signing each URL would require rewriting every entry of the manifest, typically in an edge function, which adds complexity and cost, whereas one set of cookies covers the whole directory and the browser attaches it to every request. The policy is a custom

policy rather than a canned one, because a canned policy does not allow a wildcard in the resource path, and the required scope is the directory of one video: its manifest and all its segments, and nothing more.

The edge does not distinguish one visibility level from another. Every manifest and segment requires a valid signature, and visibility decides only whether the backend issues the cookies at all. A cookie covers one video, while a catalogue page shows the thumbnails of many, so the thumbnail path has its own cache behaviour that needs no signature. A thumbnail is a single low-resolution still already shown in listings; the protected unit is the stream, not every byte that belongs to a video.

4.5 Content Moderation and Operations Design

A platform that publishes what its users upload has to decide, for each upload, whether it may be shown. No single check can be trusted with that decision, because an automatic classifier errs in both directions: it removes harmless videos, which are false positives, and it lets harmful ones through, which are false negatives. The design therefore stacks four layers, each of which can correct the one before it: automatic screening, reports from viewers, review by staff, and a random audit of what was approved. It also records every human decision together with what the machine had concluded, so that the error rates of the earlier layers can be estimated and not assumed. Figure 4.7 follows one video through screening, a report, a decision and an appeal; the subsections below explain each step.

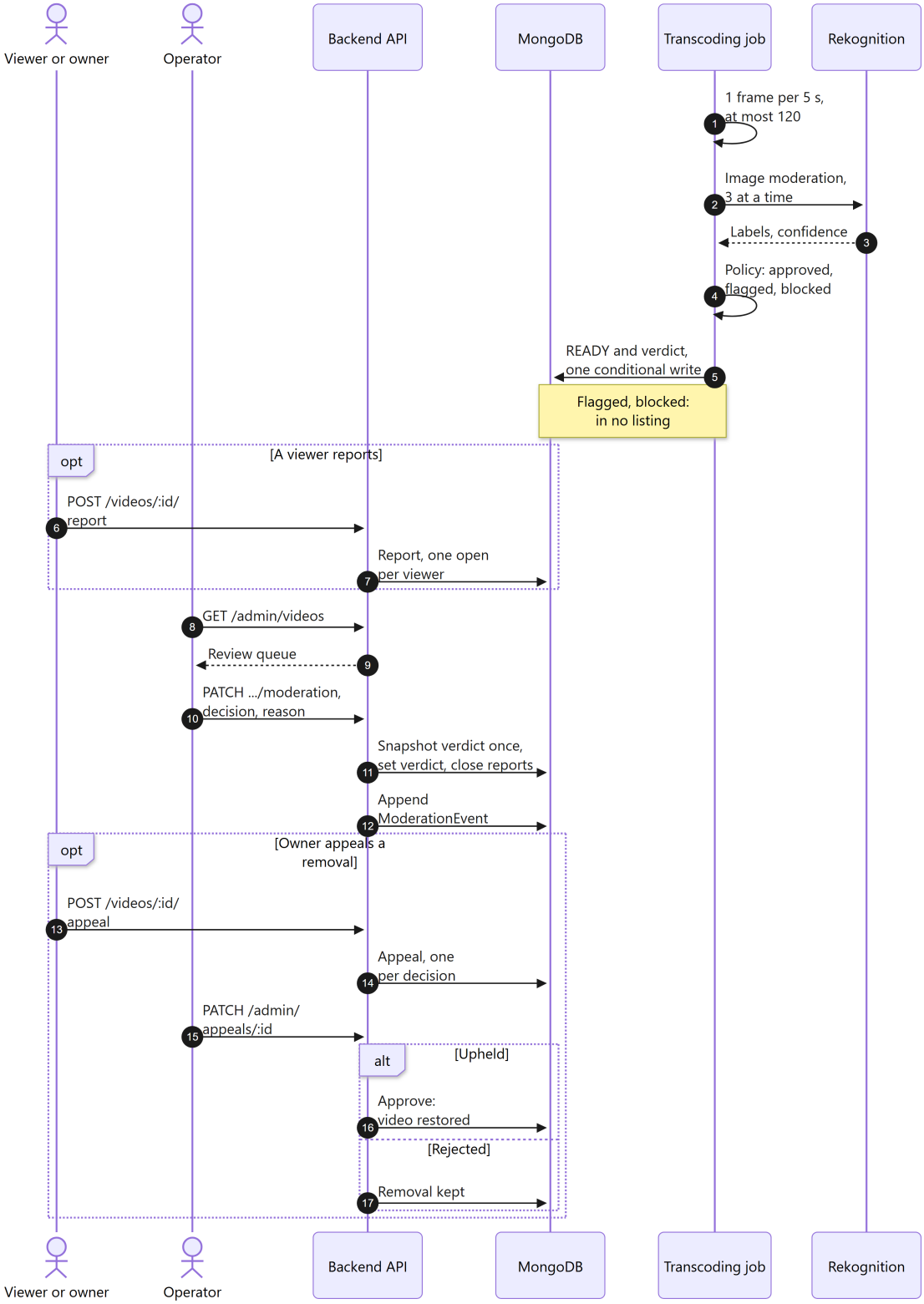


Figure 4.7: *Sequence diagram of moderation: automatic screening, a viewer's report, an operator's decision and the owner's appeal*

4.5.1 Automatic Screening

Screening runs inside the job that finishes the video and writes its verdict in the same conditional write that marks the video `READY`, so a video is never published without a verdict unless screening was switched off explicitly. On the single-container path it runs before transcoding, and on the chunked path in the finalizer. A video that is blocked is still transcoded, so that staff can watch it when its owner appeals.

The job samples one frame every five seconds, at most 120 frames, scaled to at most 720 pixels wide, and sends each to the image moderation operation of Amazon Rekognition [9]. The image operation was preferred to the video operation for a concrete reason: the video operation accepts stored videos only if they are encoded with H.264 in an MPEG-4 or MOV container and are at most 10 GB [10], whereas the platform accepts MKV, WebM and AVI uploads of up to 20 GiB, and FFmpeg in the container can decode all of them. The image operation also runs synchronously inside a job that is already running, so it needs no notification topic and no polling, and its cost has a ceiling: at the Singapore list price of 0.00125 USD per image, for the first million images of the month, a video costs at most 0.15 USD. The price of this choice is that sparse sampling can miss content that appears for less than the interval between two samples, which is why screening is the first layer and not the only one.

Each label returned for a frame is looked up in a policy table that maps it to one of three actions (Table 4.4). The table sets policy at the coarse levels of the label taxonomy and uses the finest level only to exclude a specific concept, so that a bare male chest, for example, does not send a video to review merely because its parent label is non-explicit nudity.

Table 4.4: Actions assigned to Rekognition labels by the screening policy

ACTION	LABELS	EFFECT
block	Explicit nudity and sexual activity, sex toys, graphic violence, death and emaciation, hate symbols and extremism	The video is removed automatically when a label reaches 90% confidence
review	Non-explicit nudity, obstructed intimate parts, violence, explosions, visually disturbing content, crashes	The video is held back for review when a label reaches 60% confidence
ignore	Weapons, bare back, exposed male nipple, and any label that is not in the table	No effect

Two refinements keep the policy from over-reaching. A frame that Rekognition classifies as animated or illustrated with at least 80% confidence lowers graphic violence and death from *block* to *review*, since violence in a game or a cartoon is not the violence

the rule is about; explicit sexual content and hate symbols are not lowered, because a drawing violates the rule as much as a photograph does. And screening fails closed: a video for which no frame could be analysed, or fewer than 80% of the planned frames, is held back for review and is not published, so that an unavailable service produces a delay and not a publication.

4.5.2 Moderation States and Visibility

The verdict is stored as `approved`, `flagged` or `blocked`, independently of the transcoding status of Section 4.3: a video can be `READY` and still hidden. A flagged video appears in no listing and is visible only to its owner and to staff. A blocked video can be played by staff alone, and its owner can appeal the removal. The filter that implements this is one function used by every public query path, for the reason given for authorisation in Section 5.1: copies of a rule drift apart. A video uploaded before moderation existed carries no verdict and is treated as approved.

4.5.3 Review, Appeals and Audit

Amazon's own guidance expects people to review a small share of the content, typically one to five per cent of the total volume, already flagged by the model [9]. Staff clear the review queue by approving or blocking each video, and every decision must carry a reason code from a short, closed list: for an approval, for example, a model error, fiction or a game, or an acceptable context; for a removal, for example content missed by the model, an upheld report or a failed audit; and, for a video that the machine has already removed, a confirmation that it was right. The list is closed because free text cannot be counted, whereas a count of how many removals were reversed because the content was a game points to the threshold that needs adjusting.

Each decision is appended to an event log that records who decided and in which role, the verdict before and after, the conclusion the machine had reached, the reason code, and the outcome class of Table 4.5. The machine's conclusion is copied onto the video once, immediately before a person first changes the verdict; without that snapshot the first human decision would overwrite the very fact needed to say whether the machine was right. No route edits or deletes an event, because the auditor's work depends on the log being intact.

Table 4.5: Outcome class of a human decision, given the machine's conclusion

OUTCOME	MACHINE	PERSON	MEANING
False positive	blocked	approves	The machine removed a video that was acceptable
Flag noise	flagged	approves	The flag did not need a person; it is not counted as a wrongful removal
False negative	approved	blocks	A violating video was published
True positive	blocked or flagged	blocks	The machine was right
True negative	approved	approves	The machine was right, for example a report found unfounded or an audit sample that passed
Unclassified	not known	either	A video from before the snapshot existed; it enters no rate

The owner of a removed video can appeal, once per decision, with a message of at most 500 characters; a new decision, such as the video being restored and later removed again, allows a new appeal. A person who does not own the video receives a not-found response, for the same reason as for visibility, and an operator resolves the appeal as upheld or rejected with a note. This follows the Santa Clara Principles, which ask a platform to notify the person affected and to offer an appeal that is reviewed by a person who took no part in the first decision [11]. Appeals are also the most reliable source of false positives, since only the owner has a reason to say that the machine was wrong.

False negatives are harder to find, because a missed video is one among thousands that were approved and viewer reports see only the most conspicuous of them. The design therefore draws a random sample of approved videos, which staff label *pass* or *fail*; the proportion of failures in the sample estimates the rate at which violating videos get through. A video can enter a sample only once, so its label is never counted twice.

4.5.4 Measuring the Screening

The console reports four figures, each with its denominator and a 95% Wilson confidence interval [12]: the share of automatic removals that staff reversed, which is the false positive rate; the share of flags that staff confirmed, which is the precision of the flag; the share of each group that has been reviewed at all, which is coverage; and the false negative rate. The Wilson interval suits small counts, because unlike the normal approximation it stays between zero and one and remains usable when the observed rate is near zero. The false negative rate comes only from the random audit and never from viewer reports, which would bias it low, and a video that no one has reviewed enters no rate. Confirming

a removal that the machine made is a separate action, because such a video goes straight to the Removed list, where the only other action is to restore it. Without the confirmation, the reviewed removals would be only those that staff reversed and the false positive rate would be computed over reversals alone; with it, a confirmed removal is a true positive and stays in the denominator.

4.5.5 Roles and Permissions

Four roles divide the work (Table 4.6). Operators decide, auditors read the error rates and the log without being able to change a decision, and administrators assign roles and draw audit samples. The separation of operator and auditor is deliberate: the person who reads the error rates is not the person whose decisions produce them, so the monitoring does not depend on the monitored.

Table 4.6: Permissions of the four roles

PERMISSION	ADMIN	OPERATOR	AUDITOR	USER
Approve or block a video, with a reason	yes	yes	–	–
Resolve an appeal	yes	yes	–	–
Label a video of an audit sample	yes	yes	–	–
Play a removed or held-back video	yes	yes	–	–
Draw a new audit sample	yes	–	–	–
Read the rates and the event log	yes	–	yes	–
See the owner's e-mail address in the queue	yes	–	–	–
Change a user's role	yes	–	–	–
Appeal the removal of one's own video	yes	yes	yes	yes

Authorisation is expressed as permissions and not as role names, so that adding a role means editing one table. A role change is itself restricted: only an administrator may make it, nobody may change their own role, the last administrator cannot be demoted, and every change is appended to a log that the auditor can read.

4.5.6 Verifying the Owner of an Account

Registration checks only that an e-mail address is well formed, so anyone can register with an address that belongs to someone else. Two harms follow. The registrant can upload, the most expensive operation of the system, under another person's address, which makes uploads anonymous in practice. And when the real owner later signs in with Google, the account is already held by someone who knows its password. The

design therefore asks each new account to prove that it receives mail at its address, and lets only verified accounts upload. Figure 4.8 shows both ways of proving it.

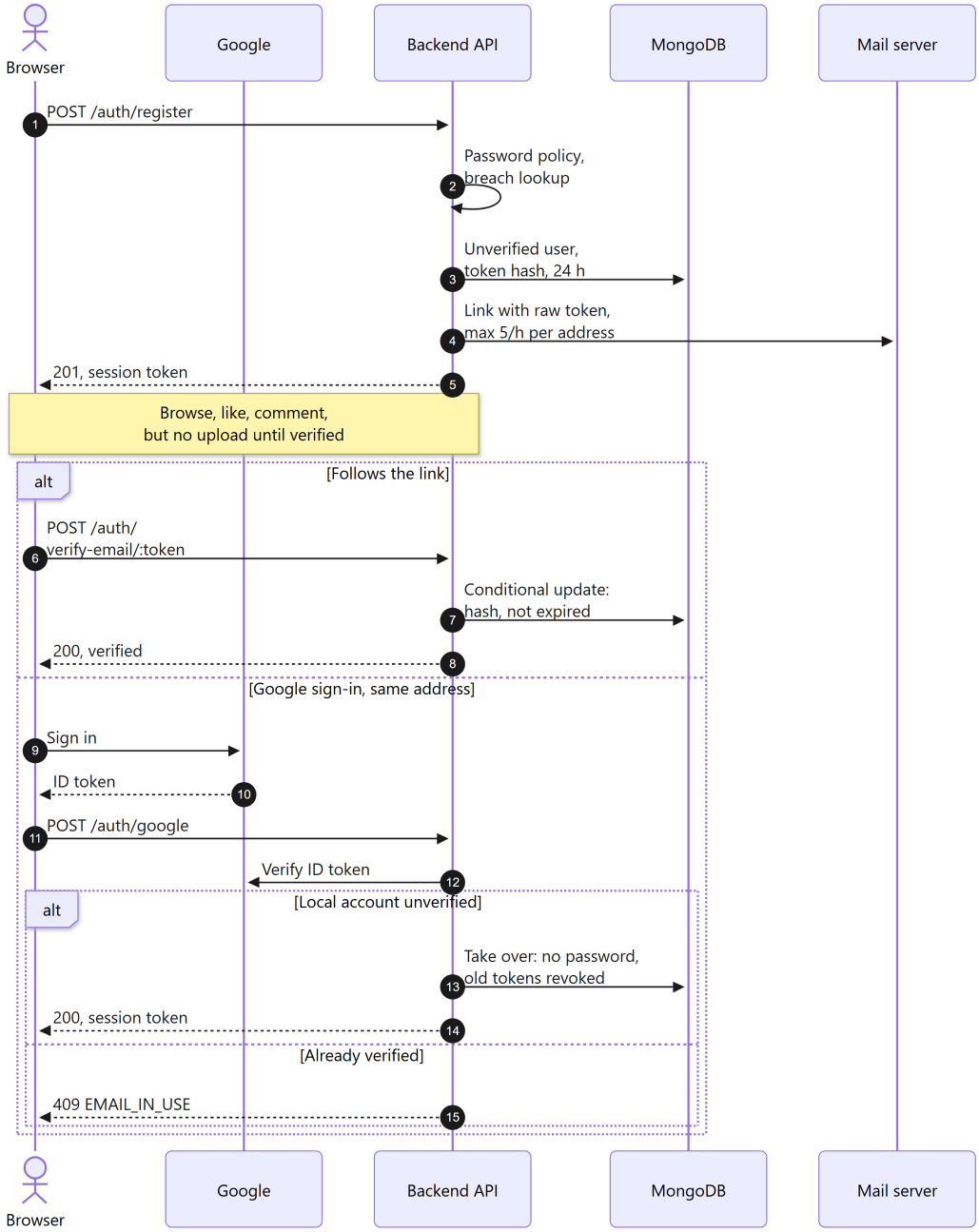


Figure 4.8: *Sequence diagram of account verification: following the e-mailed link, or signing in with Google at the same address*

After registration the backend generates a random token of 32 bytes, stores only its SHA-256 hash with an expiry of 24 hours, and e-mails a link that carries the token itself, as it does for a password reset. The link opens a page of the single page application, and that page sends the token in a POST request, so a mail client that previews links by fetching them cannot verify the account by itself. The server consumes the token in one

conditional update, so two clicks on the same link cannot both succeed. An unverified account can still browse, like and comment, but the upload endpoint refuses it with a distinct error code, which the interface turns into a reminder with a button that sends the link again, limited to five mails per hour for each account and, whoever asks, to five per hour for each recipient address.

Three events count as proof of ownership: following the link; signing in with Google, whose identity token carries an address that Google has verified; and completing a password reset, since the reset link also goes only to the mailbox. A Google sign-in whose address matches an unverified local account takes that account over: the password is removed, every token issued earlier is invalidated through the password-change timestamp, and any pending verification or reset token is deleted, so whoever registered first cannot keep access. A verified account is never merged automatically; its owner must sign in with the password and link Google from the settings page.

4.6 Interface Design

Figure 4.9 sets out the paths a user can take through the application. Two of its properties constrain the design.

The first is that the catalogue and the watch page are reachable without an account. Browsing, searching, filtering by category and playing a public video all work for an anonymous visitor, and authentication is demanded only where an action requires an identity: uploading, commenting, reacting, or managing one's own channel. A visitor who reaches the upload page without a session is diverted through sign-in or registration and returned to it, rather than asked to sign in on arrival.

The second is that the upload path does not end where the user's involvement ends. Once the browser has sent the file to Amazon S3, the interface returns control at once and the video enters the `PROCESSING` state, in which only its owner sees it, on the channel management view. Only when the pipeline marks it `READY` does it join the catalogue. The interface therefore has to show a video that exists but cannot yet be played, which is why the state machine of Section 4.3 is explicit. The same holds after the pipeline: a video that screening held back is visible to its owner and not in the catalogue until a person approves it.

The frontend is a single page application of functional React components with hooks, using React Context for authentication and theme state. Redux was unnecessary at this scale, since the shared state is only the current user and the selected theme.

The interface is available in Vietnamese and English, chosen by a control in the

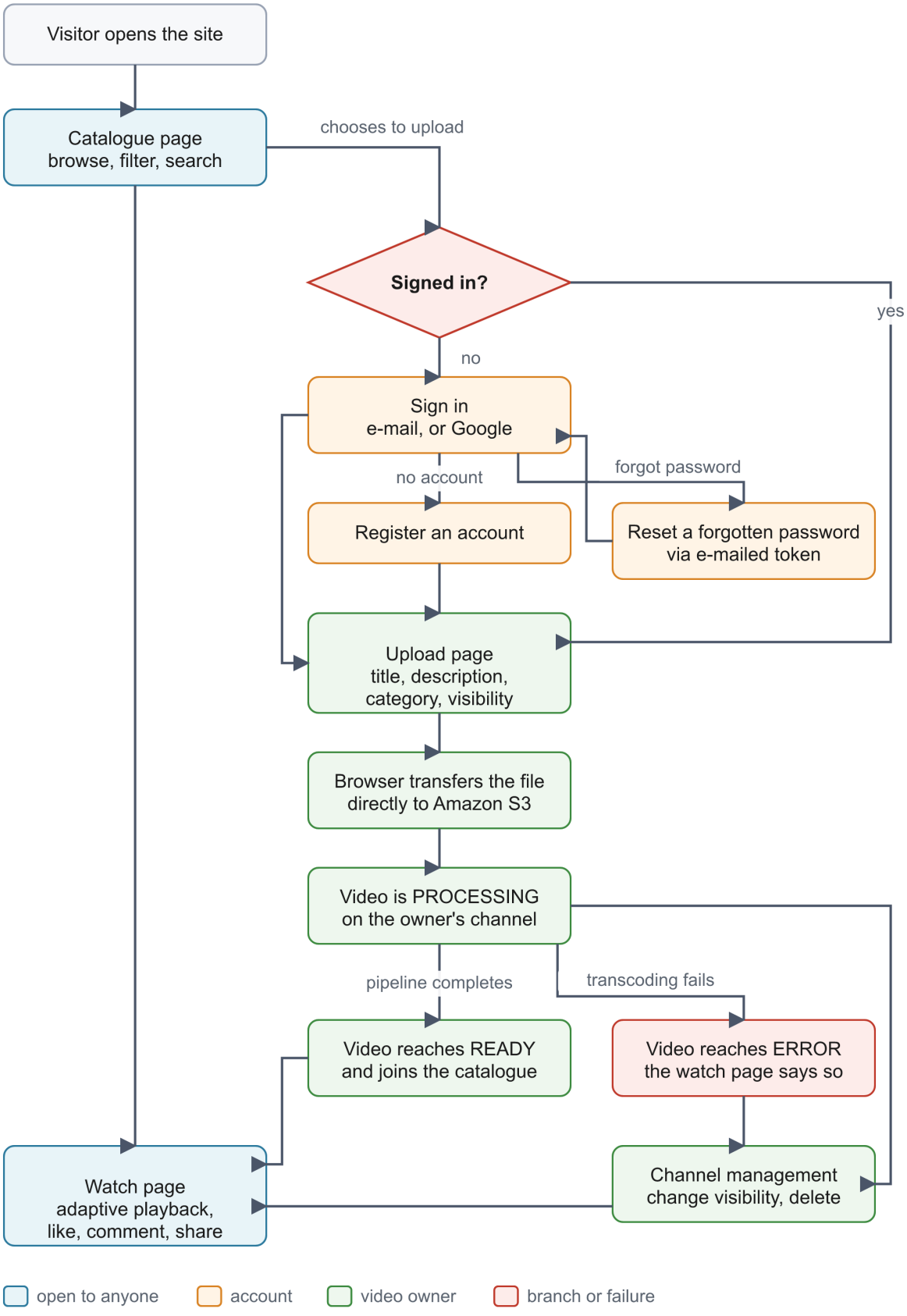


Figure 4.9: *User flow through the application, from an anonymous visit to a published video*

header and remembered in the browser. Adding a second language constrained the data model as well as the presentation. Video categories are stored as Vietnamese strings, and the schema validates each new video against exactly those strings, so translating a category would have made the catalogue filter and the upload form reject valid input. The display label and the stored value are therefore kept apart: the client shows a translated label but always sends the original value. Messages from the API are likewise chosen from the request's `Accept-Language` header.

The video player is a custom component rather than the browser's native controls, because native controls cannot be extended: the browser has its own menu for playback speed and picture-in-picture, which cannot be merged with the quality selector of HLS.js. To avoid two unrelated menus, the component reimplements play and pause, seeking, volume, fullscreen, playback speed, picture-in-picture and rendition selection within one settings menu.

The player also decides when a view is counted. Instead of incrementing the counter when the page loads, which repeated refreshes could inflate, the component adds up the time actually played, leaving out seeks, and notifies the page after thirty seconds, or at the end of a video shorter than that; the server also ignores repeated reports from the same viewer within thirty minutes and never counts the owner's own views.

The operations console is part of the same application, at its own address and reachable only by staff. Its tabs are the review queue, the removed videos, appeals, audit samples, the rates, the event log and the users, and a person sees only the tabs that the permissions of their role allow, so an operator never sees the rates and an auditor never sees a decision button. Thumbnails of videos under review stay blurred until a reviewer reveals them, so that reading the queue does not by itself expose staff to the content.

4.7 API Design

The backend exposes a REST API under `/api` with four route groups: authentication, videos, users and administration. Requests and responses are JSON, and a response carries a success flag with either the data or an error message and, where the interface must react to it, an error code such as `EMAIL_NOT_VERIFIED`. A signed-in request carries a JSON Web Token in the `Authorization` header. A general rate limit covers every route, with stricter limits on authentication, upload, reports and appeals, and request bodies are limited to 100 kB, since video data never passes through the API. Table 4.7 lists the endpoints by purpose.

Table 4.7: Endpoints of the REST API, grouped by purpose

GROUP	ENDPOINTS	ACCESS
Authentication, `/api/auth`	`POST /register`, `/login`, `/google`, `/logout`, `/forgot-password`, `/reset-password/:token`, `/verify-email/:token`	Public, with the authentication rate limit
Own account	`GET /me`; `PUT /change-password`; `POST /link-google`, `/resend-verification`	Signed in
Catalogue, `/api/videos`	`GET /`, `/:id`, `/:id/related`, `/user/:userId`, `/:id/comments`, `/upload-config`	Public; a private video only for its owner
Playback	`GET /:id/playback-auth`, which sets the Signed Cookies; `POST /:id/view`	Public, under the same access rule
Upload	`POST /initiate-upload`; `PATCH /:id/confirm-upload`; `POST /:id/multipart/parts`, `/:id/multipart/complete`	Signed in with a verified e-mail address; upload rate limit
Own videos	`PUT /:id`; `DELETE /:id`	The owner
Interaction	`POST /:id/like`, `/:id/dislike`, `/:id/comments`, `/:id/report`; `DELETE /comments/:commentId`; `POST` and `GET /:id/appeal`	Signed in; reports and appeals rate limited
Profiles, `/api/users`	`GET /:id`; `POST /avatar/presign`; `PUT /avatar`	Public profile; avatar for the signed-in user
Moderation, `/api/admin`	`GET /videos`, `/videos/:id/reports`, `/stats`; `PATCH /videos/:id/moderation`; `GET /appeals`; `PATCH /appeals/:id`; `POST /audit/sample`; `GET /audit`; `PATCH /audit/:id`	Permissions `moderation:review`, `:appeals`, `:sample` and `:audit`
Administration	`GET /users`; `PATCH /users/:id/role`; `GET /role-changes`, `/metrics`, `/events`	Permissions `users:manage`, `metrics:read` and `events:read`
Health	`GET /`	Public; includes the database connection

Two conventions follow from the design. Every video endpoint applies the access rule of Section 5.1, so a private video that the requester may not see returns not-found rather than forbidden, which does not reveal that it exists. And no endpoint carries video bytes: an upload is a pair of calls around a direct transfer to Amazon S3, and playback is one call that issues cookies for CloudFront.

4.8 Performance and Auto-Scaling Architecture

The platform's capability to withstand volatile user traffic without manual operational intervention is governed by an event-driven, multi-tier elasticity model. The system decouples interactive web traffic, computational video transcoding, and high-volume media delivery into independent scaling domains, as outlined below.

4.8.1 Event-Driven Serverless Compute Elasticity

Unlike conventional monolithic architectures that provision long-running virtual machines or EC2 clusters for media processing, the transcoding tier operates entirely on AWS Batch with Fargate (Spot and On-Demand) compute environments. This architecture achieves true scale-to-zero elasticity across three distinct operational states.

During the idle phase, when no video transcode requests are pending in Amazon SQS, the Batch compute environment maintains zero vCPUs and zero active instances, ensuring that idle compute cost is mathematically zero. When burst traffic occurs as multiple users upload videos simultaneously, Amazon S3 triggers event notifications into SQS, and the Lambda job submitter enqueues tasks into the AWS Batch Job Queue. AWS Batch evaluates the queue backlog and automatically scales Fargate tasks horizontally, provisioning isolated container instances for each job up to the account quota. Finally, during automatic scale-down, each container exits with code zero immediately upon completing HLS packaging and database synchronization; AWS Batch detects container termination and tears down the underlying Fargate infrastructure within minutes, eliminating wasteful standby expenditure.

4.8.2 Chunk-Parallel Speedup for Long-Duration Media

For long-duration videos exceeding five minutes, the platform breaks the single-container bottleneck via chunked horizontal parallelism (Section 4.2.2). Total transcoding time scales inversely with the number of parallel chunk workers: $T(N) = T_{\text{plan}} + \max_i(T_{\text{chunk},i}) + T_{\text{final}}$, where T_{plan} represents initial metadata probing (around 3 seconds), $T_{\text{chunk},i}$ is the encoding duration of chunk i, and T_{final} is playlist stitching (around 8 seconds). Because chunks execute concurrently across independent Fargate Spot tasks, total processing time decreases significantly, slashing transcoding latency by up to 65 percent on long sources (Section 6.5).

4.8.3 Edge Offloading and Delivery Scale

The delivery tier delegates media distribution to Amazon CloudFront with Origin Access Control (OAC). Because video segments are immutable once written, CloudFront caches HLS transport stream segments (`.ts`) at edge locations with long time-to-live (TTL) headers. Edge cache hit ratios exceed 98 percent, effectively shielding the private S3 Processed bucket from request spikes and ensuring that concurrent viewers experience low Time-to-First-Frame (TTFF) without saturating origin network interfaces.

4.8.4 Application Layer Scaling and Serverless Evolution

In the initial prototype baseline, the backend application API operated on a single Amazon EC2 instance (`t3.small`) behind Cloudflare proxying. While that design minimized operational complexity during early prototyping, it formed the sole single point of failure (SPOF) and concurrency bottleneck of the system: failure of the virtual machine rendered the entire platform unreachable, and sudden bursts of concurrent requests saturated the single Node.js event loop.

To achieve complete end-to-end serverless scalability and eliminate single points of failure, the application tier was evolved into an AWS Lambda serverless execution model using an HTTP adapter. The Express application is wrapped with the serverless framework adapter and deployed directly to AWS Lambda with Function URLs, providing automatic active-active horizontal scaling across three Availability Zones in the region. Cold start overhead is minimized through ephemeral container reuse, and database connection pooling overhead is eliminated by maintaining cached Mongoose connections across invocations when the connection state indicates an active socket. Background maintenance jobs, including stuck video reconciliation, are decoupled from HTTP request handling and triggered periodically via Amazon EventBridge rules. This migration completes the architectural vision of the project, establishing a hundred percent serverless, event-driven pipeline across the client, storage, application API, batch transcoding, and content delivery tiers. Remaining horizontal extensions comprise introducing an Amazon ElastiCache Redis in-memory cache for distributed view deduplication and database full-text search indexing.

Chapter 5

Development and Testing

This chapter describes how the design of Chapter 4 was implemented: the backend, the transcoder, the infrastructure code and the delivery pipeline. It then records the problems met while deploying the system and how each was solved, and ends with the automated tests.

5.1 Backend Implementation

5.1.1 Layered Services and Access Controls

The backend follows a layered structure: routes delegate to controllers, controllers to services, and only services touch the data models. Keeping business logic out of route handlers lets it be unit tested without an HTTP server.

Authorisation for video access lives in a single service function. Every path that exposes a video, whether the detail endpoint, the like and comment endpoints or the playback-authorisation endpoint, calls that function. Duplicating the check across endpoints is the most common way for an access control gap to appear, because a later change updates one copy and not the others.

Operational authority follows the same principle. The four roles, user, operator, auditor and administrator, are mapped to permissions in one table, and each route asks whether the requester holds a named permission, such as `moderation:review` or `metrics:read`, instead of comparing role names. The requester's role is read from the database on every request and is not copied into the token, so demoting a person takes effect at once rather than when a token expires.

A moderation decision is applied by one service function. It checks the reason code against the closed list for that decision, refuses to let an operator change a decision that

an administrator made, copies the machine's verdict into `autoStatus` only if no earlier decision has done so, resolves the open reports on the video, and then appends the event. If writing the event fails, the decision stands and the failure is logged: withdrawing a removal because of a faulty log would be worse, and the error rates are computed from the video documents, so a missing event does not skew them.

The server validates upload metadata before issuing a pre-signed URL, because a user can call the API directly and would otherwise obtain a URL for any file, turning the storage bucket into free hosting. It checks the declared MIME type against an allow-list of video formats and refuses files above the configured ceiling; the single and multipart paths are those of Section 4.2.1. Upload also requires a verified e-mail address. The upload guard, the profile the interface reads and the takeover of an account by a Google sign-in all read verification through one method of the user model, which treats an account with a Google identity as verified. The guard runs ahead of the upload rate limiter, so a refused request does not use up the hourly quota.

Several protections were added at the HTTP layer. Helmet sets the security headers. Cross-origin requests are limited to an explicit allow-list, which Signed Cookies also require, because the CORS specification forbids credentials together with a wildcard origin. Authentication endpoints allow ten attempts per fifteen minutes per address, which makes password guessing impractical without affecting legitimate users. The process refuses to start if the token signing secret is missing, turning a dangerous runtime failure into an obvious one.

A new password is checked in two stages, at registration, reset and change alike. The first is local: a list of the 131 passwords tried first, including common Vietnamese ones; obvious patterns such as `12345678` or a repeated character; a common word with a few digits added or letters swapped for digits; and the user's own name, e-mail prefix and the name of the service. The second asks the Pwned Passwords service whether the password appears in a known breach, by the k-anonymity protocol [13]: only the first five hexadecimal characters of its SHA-1 hash leave the server. A reply slower than two seconds, or a failed lookup, counts as no answer and lets the password through, so registration does not depend on an outside service. Both stages apply from eight characters, the length the user model already requires. This follows the guidance that a verifier compare a new secret with common, expected or compromised values instead of imposing composition rules [14].

The per-address limit on authentication does not stop one mailbox from being flooded, since an attacker who changes address is counted afresh. A second limit therefore counts the mails sent to one recipient, five per hour, shared by verification and reset mail.

Beyond it, registration still creates the account but skips the mail, a request to resend the link is refused with a 429 response, and a reset request returns the same generic reply as every other outcome, so it still does not reveal which addresses have an account.

5.1.2 Serverless Execution Adapter and State Management

To bridge the Express application into AWS Lambda without altering established route controllers, the backend leverages the `@codegenie/serverless-express` adapter. Incoming HTTP requests via Lambda Function URLs are translated into standard Node.js request and response streams, processed through existing Express middleware, and converted back into API Gateway v2 payloads.

A critical challenge in running Mongoose on serverless microVMs is connection churn: creating a new MongoDB Atlas TLS handshake on every invocation adds hundreds of milliseconds of latency and rapidly exhausts database connection pools. The backend addresses this by persisting connections across execution cycles: before invoking `connectDB()`, the adapter checks `mongoose.connection.readyState`. If an active connection exists (`readyState == 1`), the initialization step is bypassed entirely. Furthermore, background tasks such as orphaned upload cleanup and stuck video reconciliation, which previously ran via in-process `setInterval` timers on EC2, are decoupled into a dedicated event handler triggered hourly by an Amazon EventBridge schedule rule.

5.2 Transcoder Implementation

The transcoder is a Node.js worker packaged with FFmpeg in a Docker image built in several stages. Node 18 was abandoned because Mongoose 9 needs the Web Crypto API, which that version lacks, and Node 20 was later replaced with `node:24-slim` when Node 20 reached end of life on 30 April 2026. Node 24 was preferred over Node 26 because it is the Active LTS release, supported until April 2028. The Alpine variant was avoided because FFmpeg requires glibc.

A duplicate job must not overwrite a finished result. The heartbeat of Section 4.2 is in effect a lease in the sense of Burrows [15], and Burrows identified its classical failure: a lease can expire while its holder is merely stalled, so a second holder is granted the same resource while the first is still alive. A review of the pipeline against this failure found that neither `updateVideoReady` nor `updateVideoError` checked the current status before writing, so a duplicate job could overwrite a successful result or even turn

a READY video back into ERROR. Both are now a conditional findOneAndUpdate that writes only when the status is not already READY, and processVideo checks the status before downloading anything, so a late duplicate stops at once.

A single FFmpeg invocation produces every rendition that the source can fill, which decodes the source only once. The ladder has six rungs, from 144p at 120 kbit/s to 1080p at 4 Mbit/s, and each rung fits the source inside its frame without enlarging it: a 480p source yields a 360p rung and one at 854×480, not a 1080p rendition enlarged 2.25 times. Only the single-container path for a video longer than twenty minutes keeps the earlier ladder of 360p, 720p and 1080p, to stay within the time limit of a job. The command writes six-second segments and the rendition playlists; the worker then writes the master playlist. Its bandwidth values are the peak segment bit rates measured from the segments (Section 6.3.3), and its codec strings are read from each stream's sequence parameter set, since the constraint-set byte that RFC 6381 requires appears in neither the profile name nor the level reported by a probe. A thumbnail is extracted in a separate pass.

The chunked path of Section 4.2.2 runs the same image under four commands. The planner computes chunk boundaries with functions that do no input or output, so the arithmetic is tested directly: a boundary is a whole number of groups of pictures counted in frames, and every chunk writes its timestamps from the same base offset, so independently encoded chunks join without a gap or a repeated segment. A chunk job reads only its own range of the source over HTTP and encodes every rendition for it, and the finalizer joins the playlists and measures each rendition's bandwidth from the segments. Chunk jobs use their own task size of 4 vCPU and 8 GiB, which measured 14% faster and used 7% fewer vCPU-seconds than one vCPU; the size is checked against the valid Fargate combinations when the configuration loads, because Batch would otherwise reject the submission only after the video had been marked as processing.

Content screening is one function, called by the single-container job and by the finalizer with the same policy and thresholds. It extracts the sampled frames with FFmpeg, sends them to Rekognition three at a time with retries and client-side rate limiting, applies the label policy of Section 4.5.1, and returns the verdict with its evidence labels and the share of frames analysed. An error inside the step becomes a flagged verdict carrying the error, so the video waits for a person and is neither lost nor published. Screening is on unless MODERATION_ENABLED is explicitly false, so a missing setting cannot publish every video unchecked.

When FFmpeg fails, the top-level handler records ERROR in the database. In worker mode it also leaves the SQS message in flight, so the message is redelivered and, after the

configured attempts, moved to the dead letter queue. The deployed system runs `batch` mode (Section 4.2), where the container holds no message, so only the `ERROR` record applies there; retry and alerting for that path are described in Section 5.5.

5.3 Infrastructure as Code

The AWS environment is described by the fourteen Terraform modules of Table 5.1 and by three environment definitions: `dev`, which hosts the deployed system; `staging`, a reduced copy used to test a branch before it is merged; and `prod`, a variant with On-Demand Fargate capacity, thirty days of logs and buckets that cannot be force-destroyed. Only the first two have been applied; `prod` was checked with `terraform validate` and never deployed. Because `dev` serves real users, it carries the same protection against a mistaken `terraform destroy`: its buckets and registry cannot be force-deleted while they hold objects or images, and a deleted secret stays recoverable for seven days.

Table 5.1: Terraform modules and their responsibilities

MODULE	RESPONSIBILITY
s3	Raw and processed buckets, CORS configuration, Glacier lifecycle transition, event notification
sqs	Transcode queue, dead letter queue with a redrive policy, queue access policy
ecr	Container registry with image scanning on push and a lifecycle policy retaining recent images
iam	Execution roles for AWS Batch, the ECS task, the transcoder task, and the Lambda submitter
vpc	Virtual network, public subnets, internet gateway, security groups, S3 gateway endpoint
secrets	AWS Secrets Manager entries for the database connection string and the token signing secret
sns	Notification topics for transcode completion and dead letter queue alerts
monitoring	CloudWatch log groups, alarms and dashboard, a scheduled health check of the public API, and a rule that alerts on failed Batch jobs
batch	Fargate Spot compute environment and an optional On-Demand one for overflow, the main and the lower-priority job queue, and the job definition
lambda	Job submitter function and its SQS event source mapping
lambda-backend	Serverless Express API running on AWS Lambda with Function URLs, Multi-AZ high availability, and EventBridge background reconciler
cloudfront	Distribution, Origin Access Control, cache policy, and the trusted key group used for signed cookies
backend-userdata	Builds the start-up script of the backend instance from a single template, so that every environment runs the same script; creates no resources
cloudflare-ips	The IPv4 ranges from which Cloudflare reaches the origin, used to restrict the backend's security group; creates no resources

Two settings deserve mention. The S3 lifecycle policy moves raw source files to Glacier after thirty days, since they are not read again once transcoding has succeeded but must be kept in case a re-encode is needed. The cache policy gives segments, which never change, a long time-to-live and manifests, which change when the set of renditions changes, a short one.

5.4 CI/CD Pipeline and DevSecOps

Seven GitHub Actions workflows automate verification and deployment, as shown in Figure 5.1.

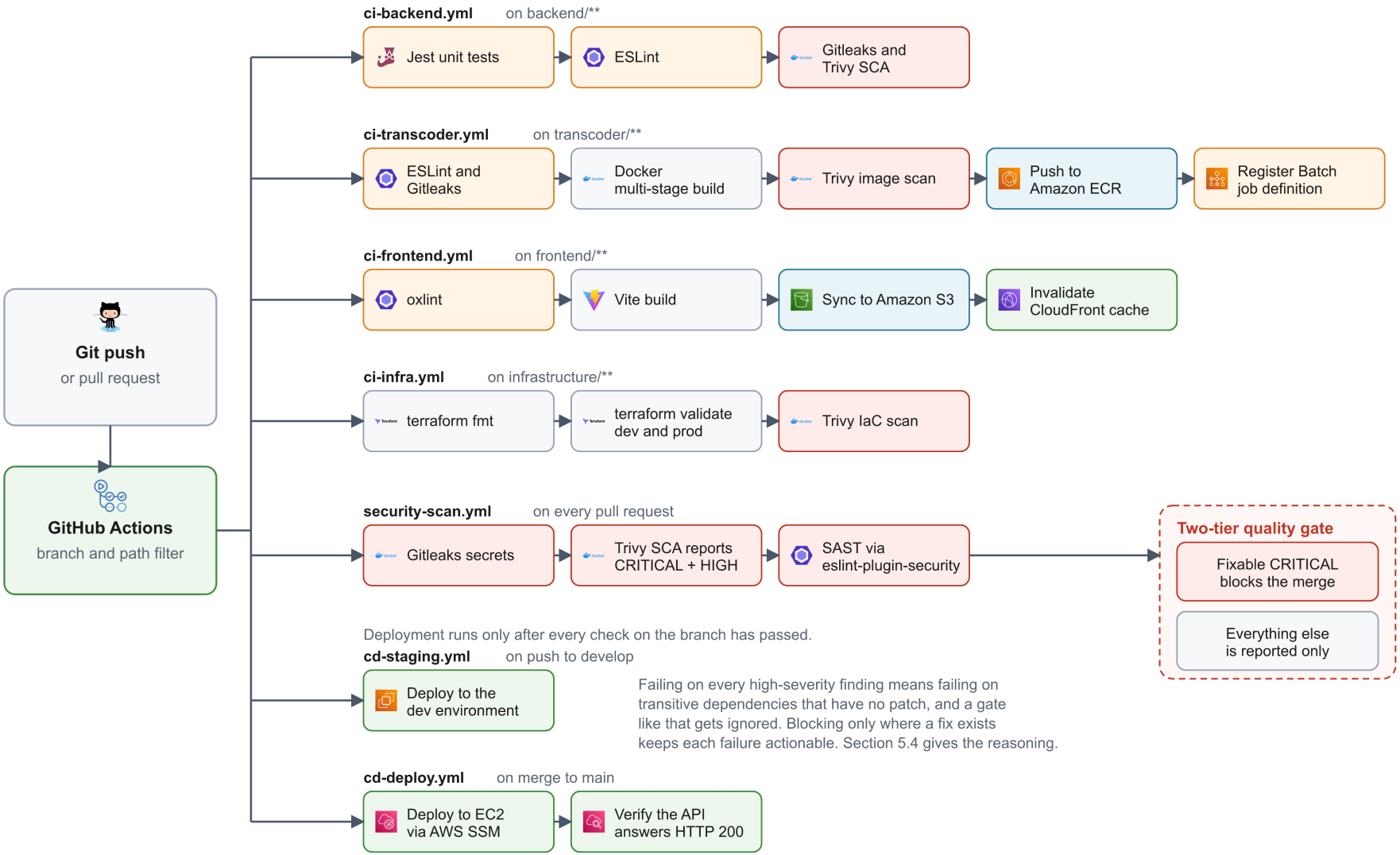


Figure 5.1: *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate*

The security gate has two tiers. Most high-severity findings come from transitive dependencies that have no patch yet, so a gate that failed on every critical or high finding would fail constantly for reasons the team cannot act on, and developers would learn to ignore it. The pipeline therefore reports critical and high findings without blocking, and blocks a merge only on a critical finding that has a fix. Every failure of the gate is thus something the team can act on.

Static analysis uses `eslint-plugin-security` rather than SonarQube, which avoids running a separate analysis server. Its rules detect, for example, commands built from user input, regular expressions open to catastrophic backtracking and unsafe random number generation. A false positive is suppressed only with an inline comment stating why. Infrastructure code has its own workflow, which checks formatting, initialises without a remote backend so that no credentials are needed, validates both environments and scans the configuration for insecure settings.

A later audit of the pipeline tightened two points. All five workflows that ran the vulnerability scanner referenced `aquasecurity/trivy-action@master`, a branch whose content can change at any time, so a compromise of that repository would have run inside a pipeline holding AWS credentials. The references are now pinned to a release commit, with the version kept in a comment. The audit also found that the Lambda submitter held `batch:SubmitJob` on every resource; it is now limited to the project's job queue and job definition. `batch:DescribeJobs` keeps its wildcard because AWS offers no resource-level control for it, and the module says so in a comment.

5.5 Challenges During Development

Several problems appeared between a design that was correct on paper and a system that behaved correctly once deployed. Table 5.2 summarises them with their causes and resolutions; the two that changed the architecture are described after it.

Table 5.2: Problems met during deployment, their causes and resolutions

PROBLEM	CAUSE	RESOLUTION
Every playback request refused with `403`	The cookie authorised `videos/*/{videoId}/*`, but the transcoder writes under `videos/{videoId}/`; the unit test compared the function with a copy of its own output	Pattern corrected to `videos/{videoId}/*`; the test now asserts the transcoder's layout; verified end to end on the deployed distribution
Thumbnails blocked in the catalogue	A cookie authorises one video, but a listing shows many thumbnails	An unsigned cache behaviour for the thumbnail path only
Fixes served late by the frontend	The deployment workflow did not invalidate the frontend's CloudFront cache	The workflow invalidates it after every deployment
Playback stopped after two hours behind an endless "reconnecting" message	Cookies expired; the handler called `startLoad()` without limit, while hls.js reports the first `403` as non-fatal and an expired cookie fails at the manifest	On a `403` the player refreshes the cookies and calls `loadSource()`, a bounded number of times, then shows a message the viewer can act on
Every transcoding task failed with `Resource Initialization Error`	With e-mail unconfigured, the mail password secret existed without a value, and the job definition declared it because its ARN was not empty	The secret itself is now conditional; failure alerts carry `statusReason`, which would have shown the cause at once
API outage (`522`) unseen by every alarm	Alarms watched only the pipeline; giving the backend a static address moved its IP while DNS still pointed at the old one	A scheduled check of the public domain every five minutes and an instance status-check alarm, self-hosted to avoid about 1.75 USD a month for a managed check
The next `terraform apply` would rebuild the server	The image was resolved with `most_recent = true`, so each Ubuntu release forced replacement and the loss of unmanaged state: environment file, nginx, TLS certificates and address	Changes to the image are ignored, so upgrades are deliberate, and an Elastic IP keeps the address

5.5.1 CloudFront in a Second Account

The content protection of Section 4.4 was implemented and tested before it could be provisioned. AWS requires a manual verification before a new account may create CloudFront distributions, and the request to lift that restriction stayed pending with no date. Until then the processed-content bucket was reachable directly, so the private setting had no effect at the storage layer.

The distribution was therefore provisioned in a second AWS account. Terraform needed no duplicated module: the distribution, its Origin Access Control, its cache and response-header policies and its trusted key group take an explicit `provider` argument pointing at the second account, while the buckets, Batch, Lambda and the database stay

in the first. Only the bucket policy cannot move, since only the account owning a bucket can attach its policy. This needed no special handling, because an Origin Access Control policy names a service principal with an `AWS:SourceArn` condition on the distribution, whichever account owns it. With the distribution in place, the temporary public bucket policy was removed.

5.5.2 Retry and Alerting on the Deployed Path

On the deployed path the Lambda submitter deletes the SQS message once the job is submitted, so the queue has nothing to redeliver, and the job definition originally declared no retry strategy. Nothing retried a failed job at any layer, and a transient fault, such as a network error while downloading the source, lost the video with nothing but an `ERROR` record. Detection was no better: the alarm described as "video transcoding failures detected" watched the dead letter queue, which receives a message only when the Lambda fails to submit, so a job failing after submission was invisible to it.

The job definition now declares three attempts. It retries infrastructure faults, exits at once when the exit status shows that FFmpeg rejected the content, and has a two-hour timeout that bounds a process stalled on malformed input. Because AWS Batch publishes no CloudWatch metric for failed jobs, an EventBridge rule matches transitions to `FAILED` and publishes to the alert topic with the `statusReason` of the job, and the old alarm's description now states what it really covers.

5.6 Testing

The automated suite holds 853 tests in 52 suites, 492 in 35 suites for the backend and 361 in 17 for the transcoder. Tables 5.3 and 5.4 group them by area.

Table 5.3: Automated backend tests by area, 492 tests in 35 suites

AREA	TESTS	FOCUS
Authentication and account verification	102	Registration conflicts, login failures and token claims; the issue, expiry and single use of the verification link, which is stored only as a hash; the Google sign-in that takes over an unverified account; missing, malformed, forged and expired tokens, and tokens issued before the account's last password change; the password policy, which refuses common, personal and breached passwords and lets a password through when the breach lookup fails; and a reset response that is identical whether or not the address has an account, including when e-mail delivery fails
Visibility and authorisation	34	Privacy enforcement across every query path; that the recommendation endpoint reuses the rule of the detail endpoint and never surfaces a private or unlisted video; and that a video exposes how many likes and dislikes it has but not who gave them
Upload	98	Metadata and ownership checks, the refusal of an unverified account, the deletion cascade, pre-signed URL creation, file type and size validation, the ceiling and its hard limit, the choice between a single and a multipart upload, part URLs signed for exact sizes, completion by listing the stored parts, and the clean-up of abandoned uploads
Content delivery	30	Cookie signing, policy scope and cookie attributes, and that revocation repeats the attributes used when issuing
Listing, reactions and views	31	Pagination bounds, a stable sort order for channel pages, likes and dislikes as conditional updates, view deduplication, and the order of routes
Operation and reconciliation	50	Reconciliation of videos stuck in processing or abandoned in uploading, the client address behind Cloudflare and nginx, rate-limit keys, the cap on mail to one recipient with its sliding window and bounded memory, e-mail configuration, and the health endpoint
Moderation and operations console	147	Classification of each decision as a false positive, a false negative or neither; the snapshot of the machine's conclusion written once; the confirmation of a removal the machine made, logged as its own action and counted as a true positive; the permission table for the four roles; one appeal per decision; audit sampling and labelling; the Wilson interval and the rates; and that the last administrator cannot be demoted

Table 5.4: Automated transcoder tests by area, 361 tests in 17 suites

AREA	TESTS	FOCUS
Chunked pipeline	184	Chunk boundaries as multiples of the group of pictures counted in frames, the conditions that fall back to one container, command construction, playlist joining, submission to the lower-priority queue, task-size validation, and the planner, audio, chunk and finalizer jobs end to end
Ladder, frame rate and encode guard	78	Rung selection from the source resolution and the reduced ladder for long videos; that the group of pictures follows the real frame rate, that a variable-frame-rate file cannot poison it, and that every rendition cuts at the same instant; giving up early when a video cannot be encoded within the budget of a job; the thumbnail
Playlist metadata	51	That the declared bandwidth of each variant is the peak measured from the segments rather than a configured target, rounded upward as RFC 8216 requires, and that the codec string is read from the encoded stream and omitted rather than guessed
Persistence and delivery	30	Conditional writes that prevent a duplicate job from overwriting a result, object storage access, and the content of the READY and ERROR notifications, which never expose raw failure detail to the viewer
Content screening	18	The label policy including parent labels and the downgrade of animated content, the frame sampling plan, and holding back a video that could not be analysed fully

The visibility suite matters most for security. It asserts that a private video is absent from the public listing, remains accessible to its owner, and returns not-found rather than forbidden to anyone else, since a forbidden response would confirm that a video with that identifier exists.

The suite caught two defects before they reached users. An early version of the signing service produced a canned policy, which the tests flagged because the cookie set held an expiry field instead of a policy field; canned policies do not allow wildcard resources, so every video would have failed to play once deployed. The second was the unconditional write of Section 5.2. Six tests now pin its contract: a normal write succeeds while the video is not yet `READY`, a write that loses the race returns the winning record instead of throwing, and a late failure can never turn a `READY` video into `ERROR`. Run against the original implementation, four of the six failed; the two that passed were the not-found cases, which neither version distinguishes. The tests therefore exercise the fixed code path rather than passing by accident of their data.

Chapter 6

Deployment and Evaluation

This chapter describes the deployed system and its interface, then evaluates it: playback quality as a viewer experiences it, transcoding latency, the chunked path for long videos, behaviour under a burst of uploads, and the monthly operating cost. It ends by setting the non-functional requirements of Section 3.3 against this evidence. Every figure was measured on the live deployment or on a staging environment with the same configuration.

6.1 The Deployed Environment

The frontend is a single-page application served from an Amazon S3 bucket through its own CloudFront distribution. The backend API runs as a Node.js process on an Amazon EC2 instance, behind nginx for TLS termination and a process manager that restarts it on failure. Media is served from a third hostname, the video distribution, whose origin is the processed-content bucket and which admits a request only when it carries a valid Signed Cookie. Cloudflare provides DNS and edge TLS for the two application hostnames, MongoDB Atlas holds the database, and the transcoding tier of Chapter 5, made of the SQS queue, the Lambda submitter and AWS Batch, occupies no capacity between jobs.

A scheduled health check calls the public API every five minutes and publishes a metric that an alarm watches, so the service is monitored rather than only the instance hosting it. The instance is the only component that costs money continuously, and it can be stopped and restarted without losing its environment file, its TLS certificates or its public address.

6.2 User Interface Showcase

This section shows the main screens of the deployed system and what a user does on each.

Figure 6.1 shows the catalogue, the first page a visitor reaches. The header holds the search field, a theme toggle and the sign-in links, and seven category tabs below it filter the listing without a page reload. Each card shows the thumbnail made during transcoding, the duration, the title, the channel, the view count and the age of the upload. Public videos can be read without an account, so a visitor sees the same catalogue as a registered user.

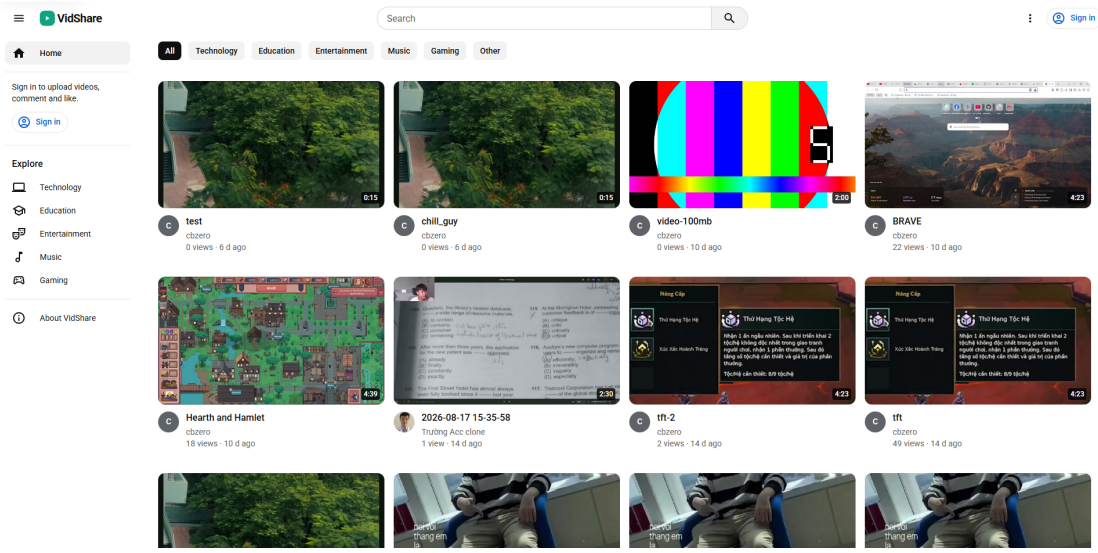


Figure 6.1: *Catalogue page, the entry point for a visitor*

Selecting a card opens the watch page in Figure 6.2. The player has its own control bar, because the quality selector of HLS.js cannot be merged into the browser's native controls (Section 4.6). It offers seeking with a time tooltip, volume, fullscreen, playback speed, picture-in-picture and rendition selection. Before the stream loads, the page asks the playback-authorisation endpoint for the Signed Cookies that the CDN requires. Below the player are the title, the like, dislike and share controls, the channel and the comments, and a sidebar lists related videos by category and author.

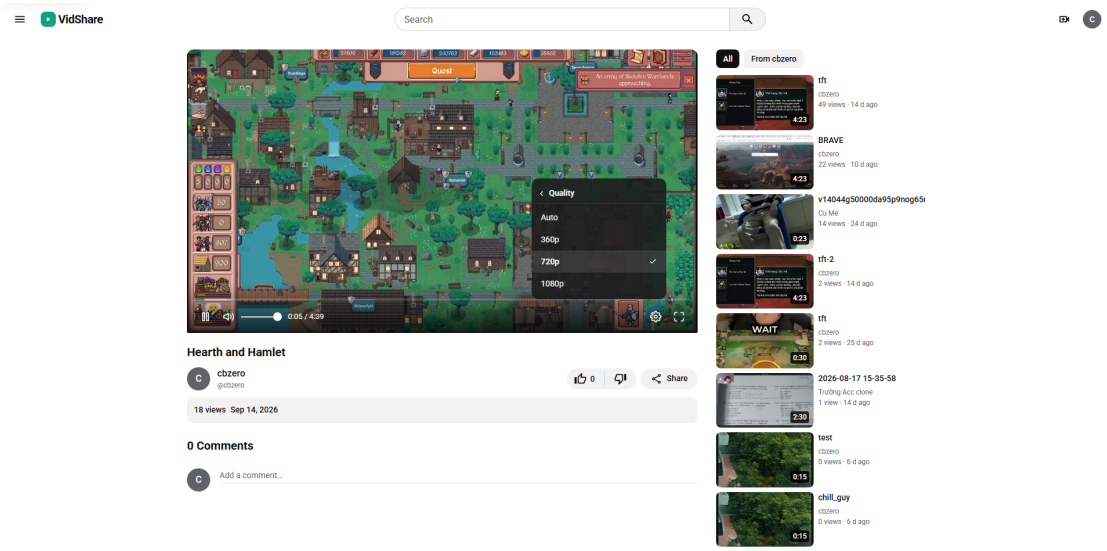


Figure 6.2: *Watch page with custom player, rendition selector and related videos sidebar*

Figure 6.3 shows the upload page. The form collects the title, description, category and visibility; the browser then receives a pre-signed URL and sends the file directly to Amazon S3, so the progress bar measures a transfer that never passes through the backend. The page shows the bytes sent, the speed and the time remaining, and its cancel button aborts the transfer and discards the draft record. When the transfer ends the page returns at once, and the video waits on the channel in a processing state until the pipeline marks it ready.

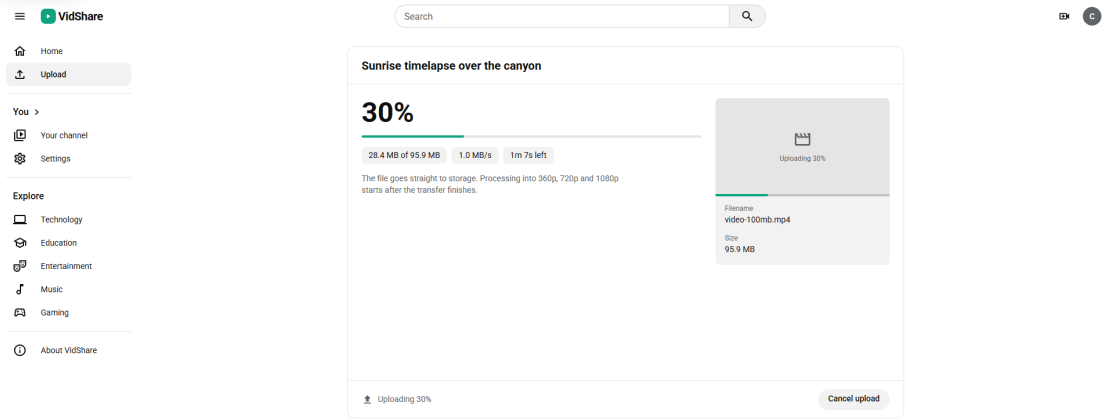


Figure 6.3: *Upload page, showing direct-to-storage transfer with live telemetry*

Figure 6.4 shows the channel management view. It lists all of the user's own videos with their state, and it is the only place where a video in the `PROCESSING` or `ERROR` state is visible, since the catalogue lists only `READY` videos. Here the user edits a video's title,

description, category and visibility, or deletes it.

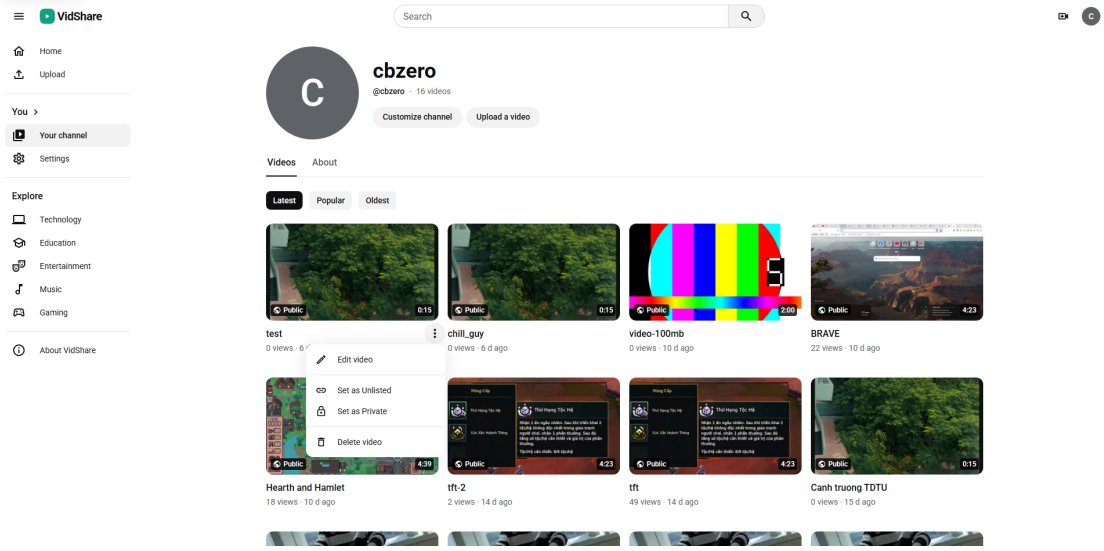


Figure 6.4: *Channel management view, including videos still being processed*

Staff reach the operations console of Figure 6.5 from the Moderation entry of the sidebar, which only operators, auditors and administrators see. Three counters show the videos waiting for review, the open viewer reports and the removed videos. The *Removed* tab lists each removed video with a blurred poster, the labels the machine detected with their confidence and their time in the video, and who removed it, with actions to watch it, read its reports, restore it, or confirm a removal the machine made. Of the two test uploads in the figure, one was removed by an operator after a viewer report; the other, a deliberately violent clip, was removed by the machine with 98.9 per cent confidence for both Blood and Gore and Graphic Violence, and an administrator then kept it removed.

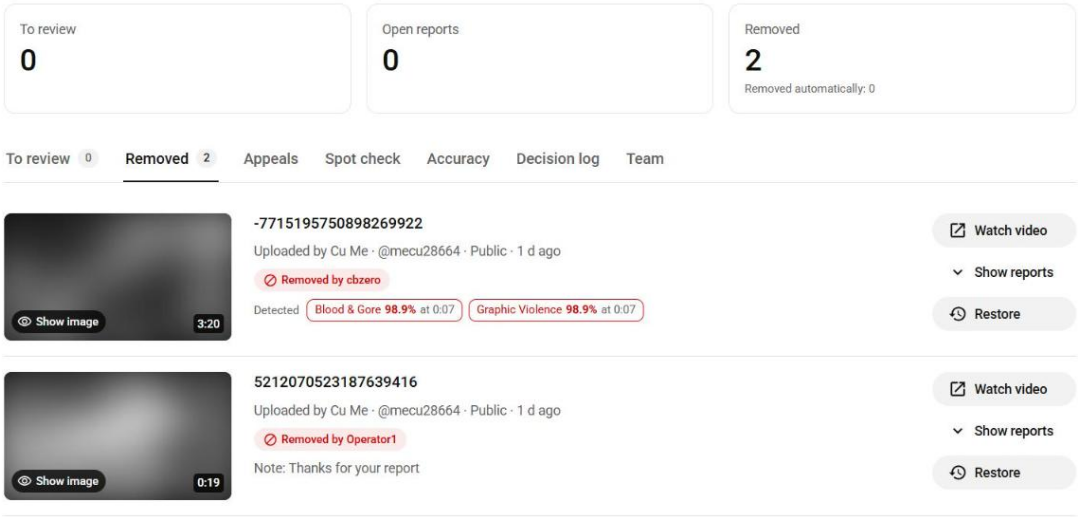


Figure 6.5: *Operations console, Removed tab, with the labels the machine detected and who removed each video*

Figure 6.6 shows the Accuracy tab, which administrators and auditors see. It gives four figures, each with its 95 per cent interval and its sample size: the share of automatic removals made wrongly, the share of flags that were right, the missed share estimated from the random sample, and the number of known misses. With one removal reviewed, the interval for wrong removals runs from 0.0 to 79.3 per cent. The screen thus shows that so small a sample says little, which is why this report makes no claim about the error rates of the screening on real traffic.

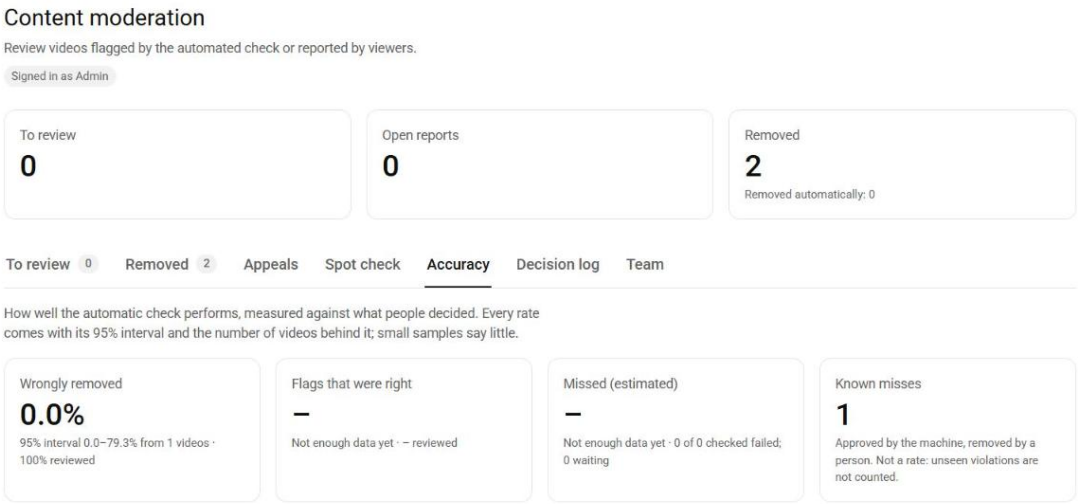


Figure 6.6: *Operations console, Accuracy tab, each rate shown with its interval and sample size*

6.3 Quality of Experience Evaluation

The playback measurements were taken in September 2026, when the bitrate ladder had three rungs, 360p, 720p and 1080p; it was extended to six rungs on 7 October 2026 (Section 5.2). The findings below do not depend on the number of rungs.

Time to first frame (TTFF) is measured by a script that fetches the master playlist, then a rendition playlist, then the first media segment, and records the time of each stage. Each measurement is repeated twenty times, because one sample means little when network latency varies and the first request is usually a cache miss. Measured directly against the S3 bucket, before the CDN was deployed, the median TTFF was 251.52 ms. This origin latency is the baseline for the CDN measurements; its statistics are in Table B.1 (Appendix B).

6.3.1 Time to First Frame Through the CDN

Through the CDN every request must carry a Signed Cookie, so the script first requests cookies from the playback-authorisation endpoint and attaches them to all three stages, as a browser does. Figure 6.7 compares the three delivery configurations.

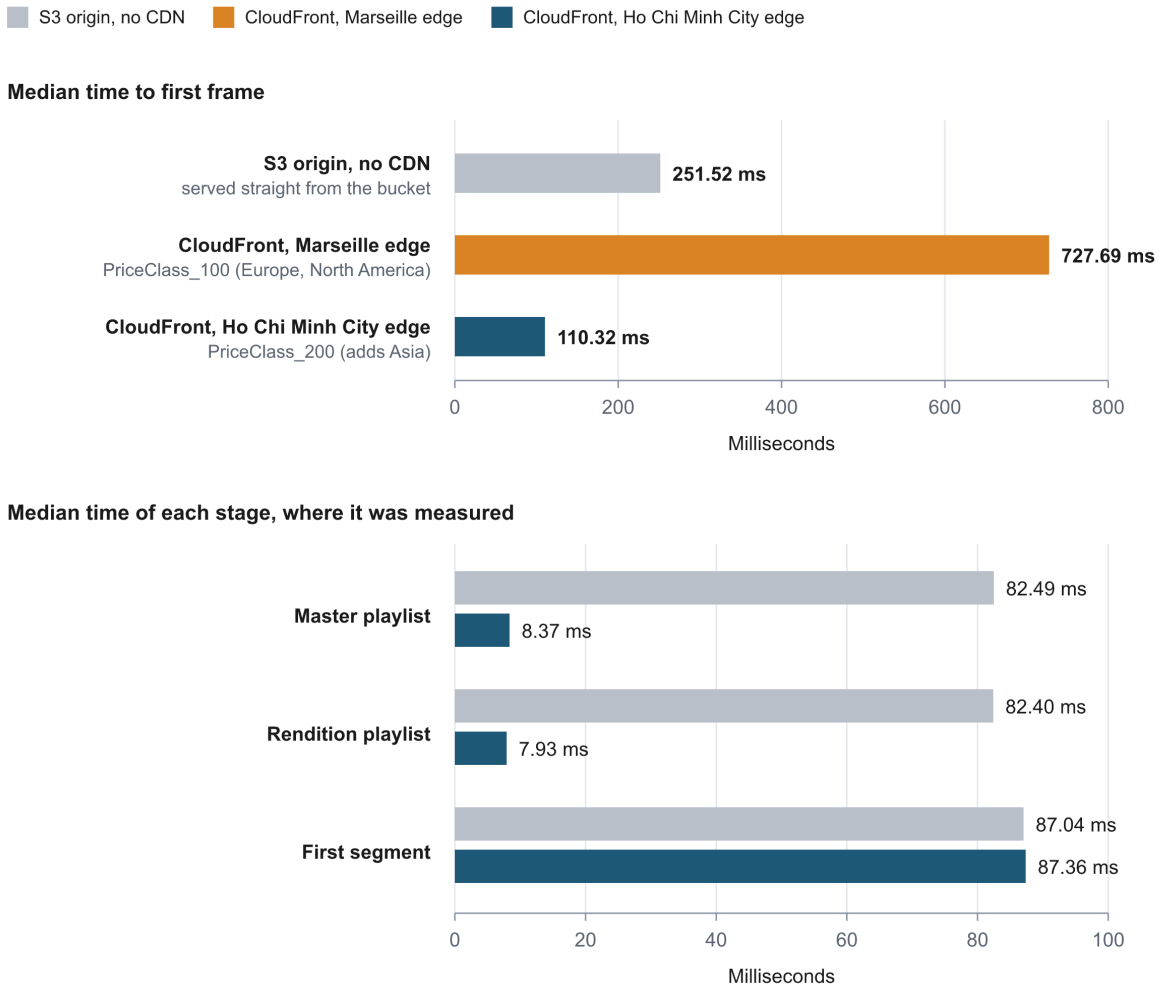


Figure 6.7: *Median time to first frame in three delivery configurations, and the median of each stage where it was measured*

Note: $n = 20$ *per configuration. Through the Ho Chi Minh City edge, the eighteen samples that hit the cache at all three stages have a median of 107.57 ms. The file of the 727.69 ms run was overwritten by the later run, so its stages are not shown. Full statistics are in Table B.2 (Appendix B).*

The first run through the CDN gave a median of 727.69 ms, 2.9 times slower than the origin it was meant to accelerate. Caching was not the cause: nineteen of twenty samples were edge hits, and the single miss took 4655 ms. Every response carried `X-Amz-Cf-Pop: MRS53`, an edge in Marseille, France, because both distributions

used `PriceClass_100`, the cheapest class, which serves only from North America and Europe. Every request from Vietnam, where the users are, therefore crossed to Europe and back.

Changing both distributions to `PriceClass_200`, which includes Asia, moved delivery to `SGN50` in Ho Chi Minh City. The median fell to 110.32 ms, 6.6 times faster than through Marseille and 2.3 times faster than the S3 origin. The two playlists, small cached files, now take single-digit milliseconds, and the segment, the only stage with a real payload, takes most of the time. The fault was invisible to every check except a measurement taken from where the users are: the configuration was valid, the cache hit rate was high and every request succeeded.

6.3.2 Delivery Latency by Client Location

Because one vantage point had hidden that fault, the probe was deployed as an AWS Lambda function in five regions, run twenty times in each, and deleted afterwards. Figure 6.8 shows the result.

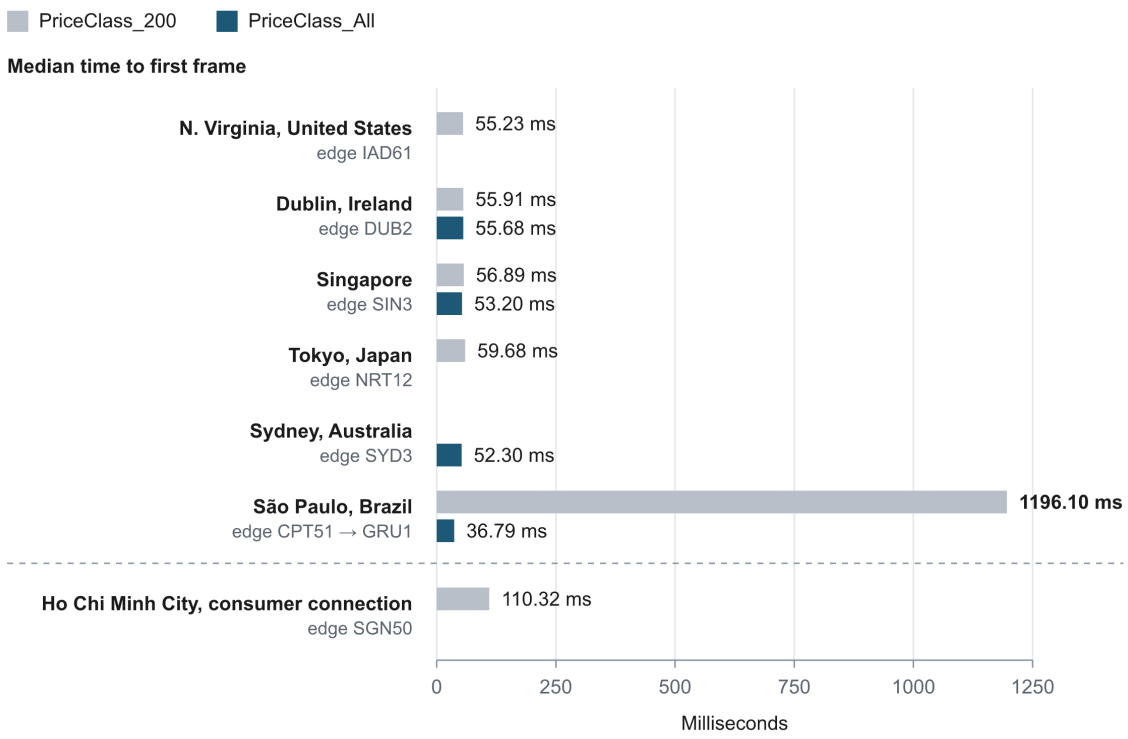


Figure 6.8: *Median time to first frame by client location, before and after* `PriceClass_All`

Note: milliseconds, $n = 20$ per location and configuration. A location with one bar was measured in one configuration only. The Ho Chi Minh City bar is the consumer-connection measurement of Figure 6.7, shown for comparison. Medians over cache hits

are in Table B.3 (Appendix B).

Under `PriceClass_200`, four locations clustered between 55 and 60 ms, each served by an edge in its own region. São Paulo was the exception: it was served by `CPT51` in Cape Town at a median of 1196 ms, because this price class excludes South America and Oceania. Both distributions were therefore moved to `PriceClass_All`. Brazil is now served by `GRU1` in São Paulo at 36.79 ms, a thirty-two-fold improvement, and Sydney by `SYD3` at 52.30 ms, while the two control locations moved by less than 4 ms.

The system has no users in South America or Oceania, so this change met no existing need. It was adopted because it costs nothing in practice: the CloudFront free tier covers one terabyte of transfer and ten million requests a month, far above this deployment's traffic. If traffic ever exceeds that allowance, this setting should be revisited first, since those regions have the highest per-gigabyte rates.

These probes ran inside AWS regions over well-provisioned paths, which is why they report about 55 ms while the consumer connection in Ho Chi Minh City reports 110 ms to an edge in the same city. The figures therefore measure edge proximity under favourable conditions, not what a home viewer sees; on this evidence the last mile costs roughly a factor of two.

6.3.3 Playback Under Constrained Bandwidth

Rebuffering, bitrate switches and the bitrate a viewer receives exist only while a player is playing, so HTTP requests alone cannot measure them. The harness written for this drives the deployed watch page in headless Chromium with Playwright, reads the `waiting`, `playing`, `ended` and `resize` events of the <video> element, and caps bandwidth through the Chrome DevTools Protocol. Startup buffering is excluded from the rebuffering ratio, the first rendition choice is not counted as a switch, and the average bitrate is computed from the bytes received, weighted by the time each rendition was on screen. Each profile discards two warm-up runs before recording five, because with one warm-up the first slow 3G run took 26.04 s to start against about 1.90 s for the others.

Table 6.1: Playback quality by emulated network profile

METRIC	UNTHROTTLED	FAST 3G	SLOW 3G
Startup delay, median (s)	2.576	2.076	3.803
Startup delay, max (s)	5.221	2.389	3.920
Rebuffering ratio, median	0.00%	0.00%	5.72%
Rebuffering ratio, mean	0.00%	1.20%	6.49%
Rebuffering ratio, max	0.00%	5.99%	14.83%
Runs exhibiting a stall	0/5	1/5	3/5
Bitrate switches, median	1	4	5
Average bitrate (kbit/s)	1833	1551	1200

Note: five recorded runs per profile, 120 s of playback each, one 4:23 source. Emulated ceilings are 1678 kbit/s for fast 3G and 512 kbit/s for slow 3G. Rebuffering is given as median, mean and maximum because stalls are rare but severe: the fast 3G median of 0.00% hides one run at 5.99%.

Computing the received bitrate exposed a defect in the master playlist. The `BANDWIDTH` attribute of each variant was written as the sum of two configured target rates, without looking at the encoded output. RFC 8216 (Section 4.3.4.2) defines it as the peak segment bit rate of the variant and requires it to be derived from the segments when they exist, as they do here. Table 6.2 compares the declared value with the actual peak and mean, computed from the size and the `#EXTINF` duration of every segment.

Table 6.2: Declared against actual bitrate for each rendition

RENDITION	DECLARED	PEAK	MEAN	SHORTFALL
360p	464	637	514	37%
720p	1628	2148	1714	32%
1080p	4192	5390	4338	29%

Note: kbit/s; $n = 44$ segments per rendition. The shortfall is how far the declared value falls below the actual peak.

A declared value below the real peak leads hls.js to choose a rendition the connection cannot sustain. Fast 3G sits on that boundary: its 1678 kbit/s ceiling admits the declared 1628 kbit/s of the 720p rung, but not its actual mean of 1714 or peak of 2148. The correction computes `BANDWIDTH` from the segments, taking the peak and rounding up. Existing videos were repaired by rewriting only the master playlist, so the segments stayed the same and the two rounds of Table 6.3 differ in nothing else.

Table 6.3: Playback quality before and after the manifest correction

PROFILE	REBUFFERING	STALLED	STARTUP MAX	BITRATE
Unthrottled	0.00% → 0.00%	0/5 → 0/5	3.29 → 5.22	1833 → 1833
Fast 3G	2.48% → 1.20%	2/5 → 1/5	16.81 → 2.39	1823 → 1551
Slow 3G	6.49% → 6.49%	3/5 → 3/5	3.83 → 3.92	1236 → 1200

Note: rebuffering is the mean of five runs, startup maximum in seconds and average bitrate in kbit/s. Only `BANDWIDTH` *changed between the two rounds.*

The correction helps only on fast 3G. There the worst startup delay falls from 16.81 s to 2.39 s, mean rebuffering halves and the stalled runs drop from two to one. The price is a 15% lower average bitrate and a rise in the median number of switches from one to four, as the player now adapts instead of holding a level it cannot carry. The unthrottled profile is a control and does not change: its startup maximum moved from 3.29 to 5.22 s through one slow run, with rebuffering at zero in both rounds. Slow 3G does not change either, to within 0.02 percentage points, because even the lowest rung, with an actual peak of 637 kbit/s, exceeds its 512 kbit/s ceiling. A manifest cannot help a player that has no lower rung to choose, so the remedy for this case would be a lower rung.

Three limits apply to these results. The peak that the harness observes is a lower bound, because it sees only the segments the player fetched. The profiles ran in sequence, so the edge cache state is confounded with profile order for startup delay, which likely explains why fast 3G started faster than the unthrottled profile. And the harness exports its event log in the input format of the ITU-T P.1203 model but computes no MOS score, so none is claimed.

6.3.4 Effect of Content Complexity

The results above come from one source video, so they could describe that file rather than the system. The attribute that plausibly governs playback is visual complexity, measured here as the bitrate of a sixty-second excerpt encoded at constant quality (CRF 23): at constant quality the encoder lets the bitrate follow the content. Three sources measured 151, 405 and 2298 kbit/s, a spread of 15.2 times, and are called low, medium and high below. Each source ran under all three profiles before the next began, so the effect of profile order cancels when sources are compared.

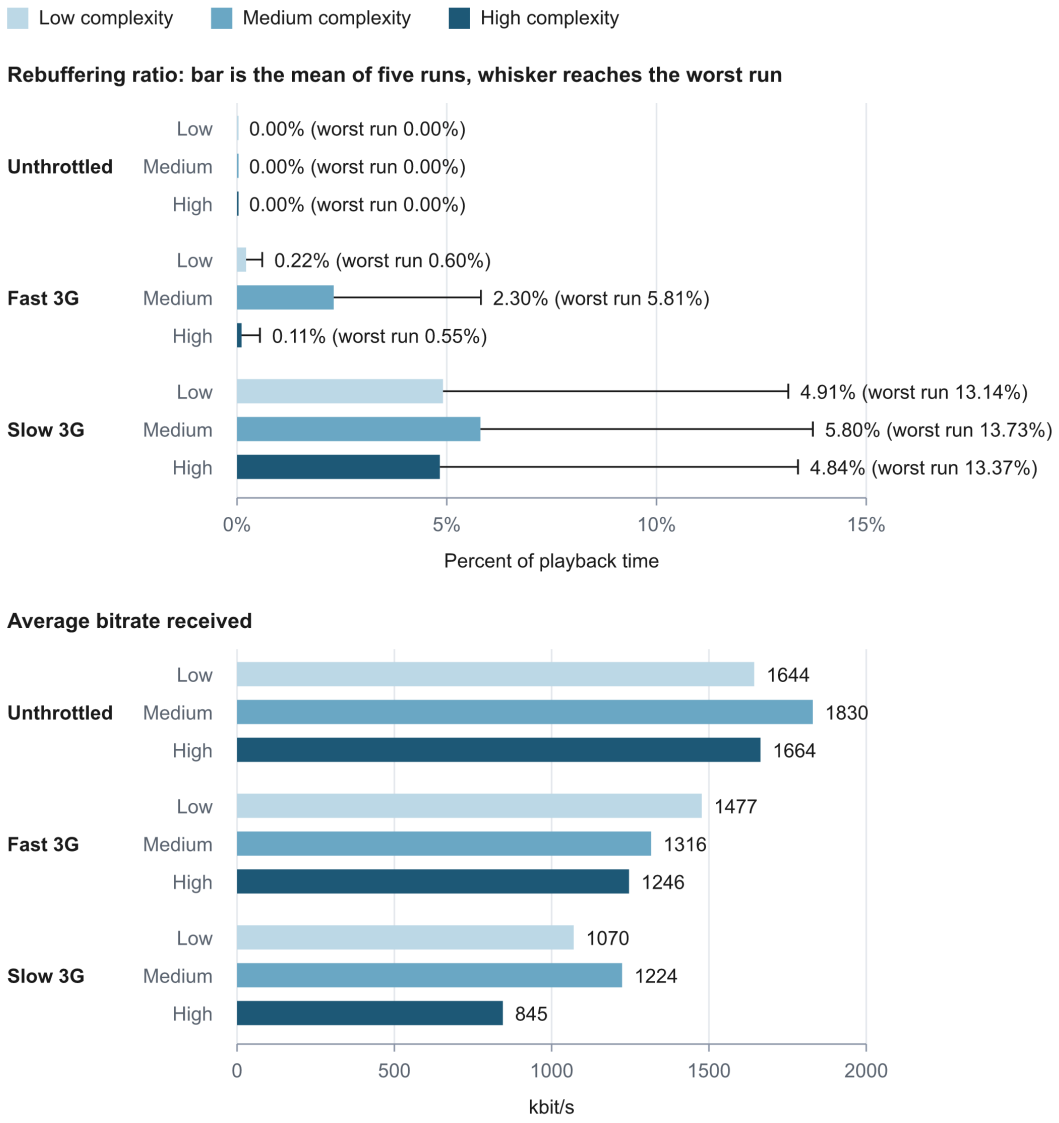


Figure 6.9: *Rebuffering ratio and average bitrate for three sources of differing visual complexity under three network profiles*

Note: five recorded runs per combination after two warm-up runs, 120 s of playback each, at a device pixel ratio of one. Source durations are 4:23, 4:23 and 4:40. Stalls and median startup delays are in Table B.4 (Appendix B).

Complexity does not order the result (Figure 6.9). Under fast 3G the mean rebuffering ratio is 0.22%, 2.30% and 0.11% for low, medium and high; under slow 3G the three lie within one percentage point, and unthrottled they are all zero. The reason is the encoder configuration. The ladder targets fixed bitrates in average-bitrate mode, so the encoder spends the same budget whatever the content, and the 720p rung averages 1650, 1694 and 1652 kbit/s for the three sources. Content is flattened when it is encoded, and little remains for it to influence at playback.

A second finding concerns the unthrottled rows. The player enables the hls.js option `capLevelToPlayerSize`, which limits the renditions to those needed to fill the video element, scaled by the device pixel ratio. The player column is about 880 px wide on any display, so at a ratio of one the player needs at most 1280×720 and the 1080p rung is unreachable: it received one segment in all nine measurements. Laptops commonly report a ratio between 1.25 and 2, so the unthrottled profile was measured again at a ratio of two (Table 6.4).

Table 6.4: Unthrottled playback at device pixel ratio one against two

SOURCE	DPR 1	DPR 2	RUNGS, DPR 1	RUNGS, DPR 2
Low	1644	4318	360p, 720p	360p, 1080p
Medium	1830	4312	360p, 720p	360p, 1080p
High	1664	4206	360p, 720p	360p, 720p, 1080p

Note: average bitrate in kbit/s over five recorded runs per cell. Rebuffering was 0.00% in every run.

At a ratio of two the average bitrate rises 2.5 to 2.6 times with no stall, and the median startup delay moves by at most 0.34 s, so the earlier unthrottled figures were limited by the resolution cap, not by the network. The middle rung is seldom used on a fast connection, since the low and medium sources jump from 360p straight to 1080p. With 1080p now fetched in quantity, the observed peak matched the declared `BANDWIDTH` exactly for all three sources, at 4 601 399, 5 389 584 and 5 495 113 bit/s, so the corrected value is reached by a real segment rather than overstated. Two limits remain: the 120 s window covers only about the first twenty of the forty-four segments, and under fast 3G two sources showed a bimodal startup, near 1.2 s and near 7.5 s, whose cause was not established.

6.4 Transcoding Pipeline Performance

Pipeline latency runs from the end of the upload to the moment the record reaches `READY`, which is the interval a user actually waits. It includes the S3 event delay, queueing in SQS, container start-up on Fargate, FFmpeg and the upload of the results. These measurements predate the six-rung ladder and chunked transcoding: one container per video produced 360p, 720p and 1080p, the configuration that Section 6.5 later compares against. The job definition revision that each job ran was checked afterwards, so the allocation of every row is confirmed rather than taken from the label the script recorded.

Table 6.5: Transcoding pipeline latency by input size

SIZE	DURATION	UPLOAD (S)	PIPELINE (S)	REALTIME FACTOR
100 MB (n=3)	120 s	3.13	475.80	3.97
500 MB (n=1)	600 s	13.13	2281.25	3.80
1 GB (n=1)	1600 s	72.00	5917.03	3.70

Note: 1 vCPU and 2 GB per job; mean across runs. The realtime factor is pipeline time divided by video duration, so a value above 1.0 means transcoding took longer than the video.

The realtime factor stays between 3.70 and 3.97 across a thirteen-fold range of duration, so at fixed compute the pipeline time grows linearly with duration. The three 100 MB runs, 346.32, 539.80 and 541.28 s, show no cold-start pattern, since the first was the fastest; at this scale the variation of Fargate Spot capacity appears to dominate. The 500 MB and 1 GB sizes were measured once each, because each run occupies 30 to 100 minutes of AWS Batch time.

Raising the container from 1 vCPU and 2 GB to 4 vCPU and 8 GB reduced the mean pipeline time from 475.80 s to 115.56 s for 100 MB (three runs each), and from 5917.03 s (one run) to 1078.70 s for 1 GB (two runs, 1080.29 and 1077.10 s), speedups of 4.12 and 5.49 times. Both exceed the four times that the vCPU count alone could explain. Amdahl's Law bounds the speedup below the resource ratio, because parts of an encoder such as entropy coding and rate control do not parallelise, and multi-threaded H.264 studies report this sub-linear pattern: 1.97, 3.96 and 7.71 times at 2, 4 and 8 threads [16], and 4.7 to 8.6 times across six Cell SPEs [17].

Two confounds explain the excess. Memory rose from 2 to 8 GB at the same time as the vCPU count, and the 1 vCPU baseline is unstable: its three 100 MB runs spread by 56.3%, against 24.0% at 4 vCPU. The speedup is therefore 3.29 comparing the fastest runs, 4.12 the means, 4.15 the slowest runs and 4.86 the medians. The data support a reduction of roughly four times, but not a precise scaling law. Cost per video is unaffected either way: Fargate bills per vCPU-second, so four times the rate for a quarter of the time leaves the charge per video about the same, and the 4 vCPU configuration buys a much shorter wait at almost no extra cost.

6.5 Chunked Transcoding of Long Videos

A single container transcodes at a roughly constant fraction of the video's runtime, and one AWS Batch job is limited to two hours. A five-minute video took 26.7 minutes as

a single job, which extrapolates to about twenty-one hours for four hours of content, so the requirement to accept videos of four hours or more needs the chunked path of Section 4.2.2. The packets of the produced renditions were checked for gaps and overlaps at every chunk boundary, and the check passed in every run below.

The measurements ran in the staging environment, which has the same container image, job definitions and compute settings as production but its own database and buckets. Capacity is the Fargate vCPU quota: 8 vCPU of Fargate Spot, which is the production quota, or 16 vCPU when an On-Demand environment of 8 vCPU was attached behind it. Each chunk runs in a 4 vCPU, 8 GB task, the size chosen in Figure 6.10. Latency runs from the end of the upload to `READY`. The 25-minute source is a 1080p H.264 file of 2.75 GiB with AAC audio. The four-hour source repeats a 1,600-second 1080p test pattern nine times with a 128 kbit/s tone, and was uploaded with S3 multipart, as the browser does for files larger than 512 MiB.

Table 6.6: Single-container path against chunked path for videos of five to fifteen minutes

VIDEO LENGTH	SINGLE JOB (MIN)	CHUNKED (MIN)	SPEED-UP
5 min	26.7	11.9	2.2
10 min	≈ 53.0	27.2	≈ 1.9
15 min	≈ 80.0	17.4	≈ 4.6

Note: values marked ≈ for the single-container path are extrapolated at 5.3 times the video duration from the measured five-minute run. The chunked values come from three videos sharing one 8 vCPU quota, so they overstate what a lone submission would see.

Table 6.6 explains the five-minute threshold. A five-minute video finished in 11.9 minutes on the chunked path against 26.7 minutes as one job, 2.2 times faster despite about 4.4 minutes of fixed cost for planning, audio encoding and finalisation. For a shorter video the fixed cost would exceed half of the processing time, so shorter videos keep the single job.

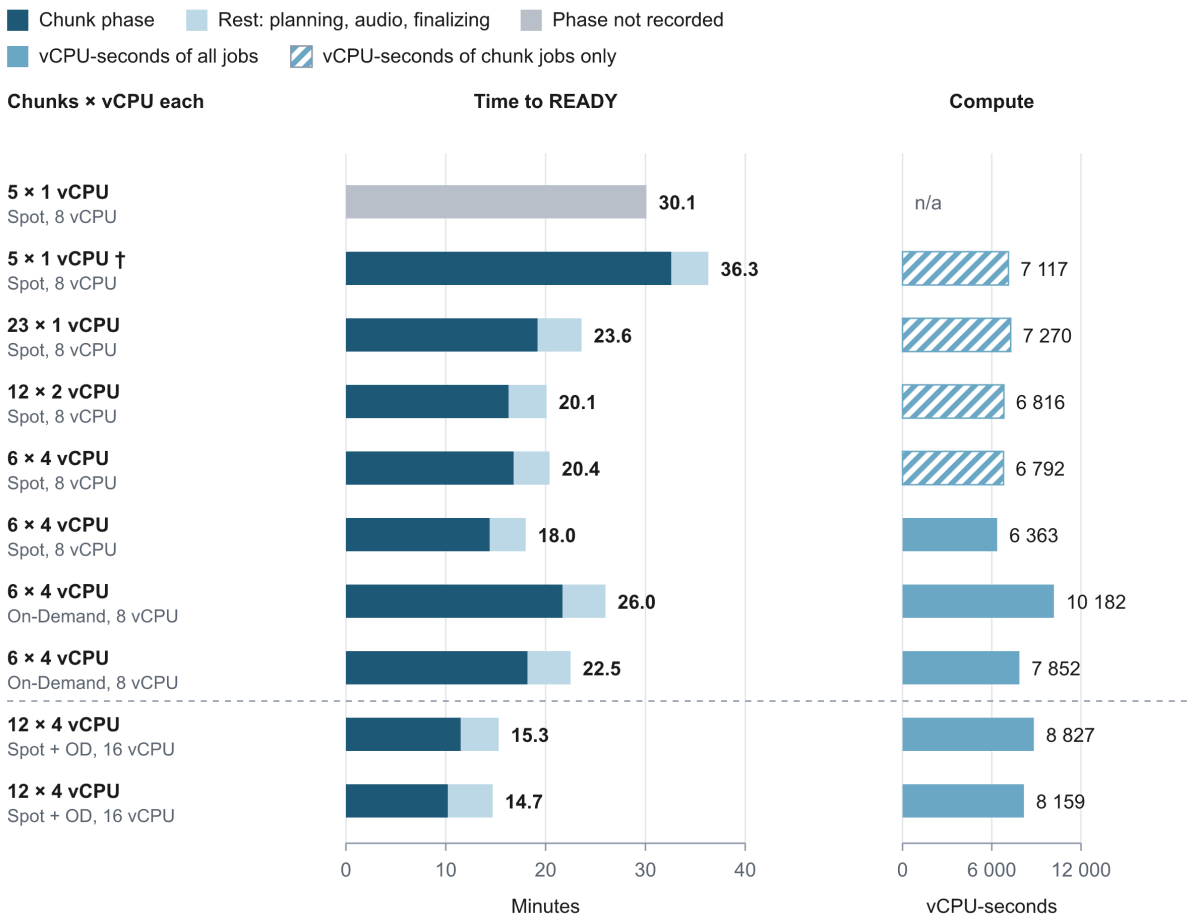


Figure 6.10: *A 25-minute video on the chunked path: time to READY and compute by chunk configuration and capacity*

Note: the first four bars were timed from the creation of the first job and the others from the end of the upload, about half a minute apart. † marks the run on production. The pale part of a time bar is planning, audio and finalisation. Hatched bars count chunk jobs only and plain bars every job, so the two kinds are not comparable. Phase lengths are in Table B.5 (Appendix B).

Figure 6.10 follows the 25-minute video through the changes made to the path. Five chunks of five minutes on 1 vCPU tasks took 36.3 minutes; they used only five of the eight vCPU, and chunks of equal size took between 18.2 and 32.6 minutes because Fargate Spot hosts differ. Choosing the chunk length from the capacity, which gave 23 chunks, cut the time to 23.6 minutes, and moving to 4 vCPU tasks cut it to 20.4 and, on a repeat, 18.0 minutes; a 4 vCPU task encoded a second of video 4.7 times faster than a 1 vCPU task. Twelve 2 vCPU tasks gave the same time and vCPU-seconds as six 4 vCPU tasks, so the larger size was kept. At 16 vCPU the video took 14.7 and 15.3 minutes, only 1.2 times faster for 1.3 to 1.4 times the vCPU-seconds, because about three minutes of fixed phases remain and each of the twelve smaller chunks pays a container start-up.

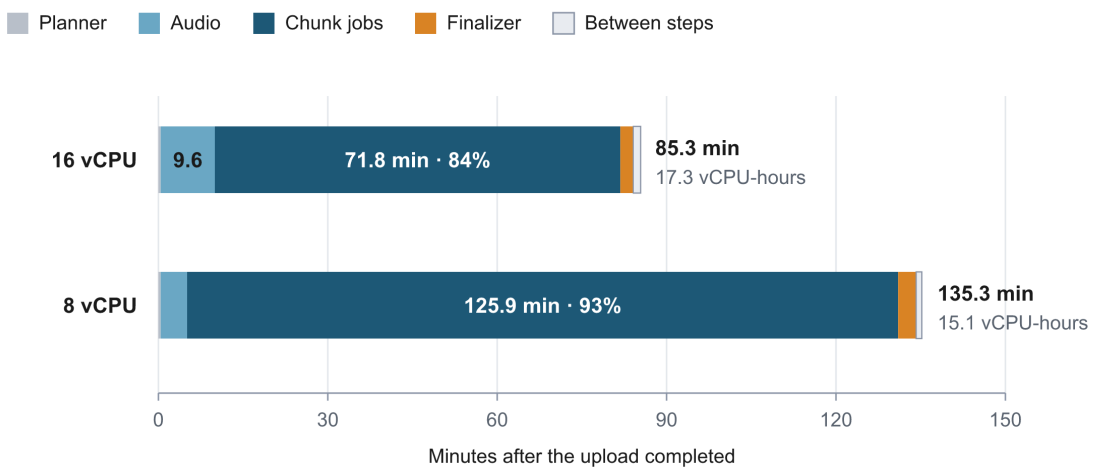


Figure 6.11: *A four-hour video: time from the end of the upload to READY, by step, at 16 and at 8 vCPU*

Note: the pale segment at the end of each bar is queueing and hand-over between jobs. Steps too short for a label are given in Table B.6 (Appendix B).

At the production capacity of 8 vCPU, a four-hour video reached `READY` 135.3 minutes after its upload finished, and at 16 vCPU 85.3 minutes (Figure 6.11). No chunk ran longer than seven minutes, far inside the two-hour limit of a job. The first measurement of the path, with 1 vCPU chunks at 8 vCPU, took 185.0 minutes, so the later changes saved 27% at production capacity. Capacity helped the long video more than the 25-minute one, 1.6 times against 1.2, because 48 chunks keep every slot busy through many rounds and the fixed phases are a small share of the total. Compute was 15.1 vCPU-hours at 8 vCPU and 17.3 at 16 vCPU, so doubling the capacity cost about 15% more compute for 37% less time.

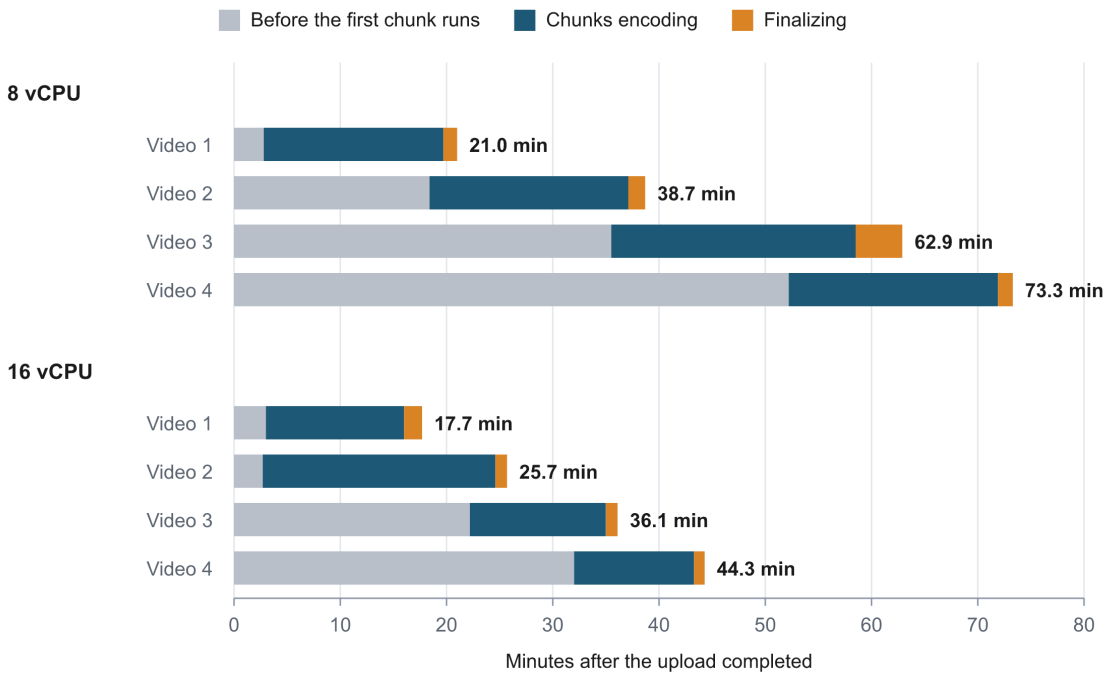


Figure 6.12: *Four 25-minute videos submitted together: when each waited, encoded and finalised, at 8 and at 16 vCPU*

Note: at 8 vCPU the four videos were uploaded together over one connection and timed from the first job creation; at 16 vCPU they were server-side copies of one file, so all four arrived within a second. Videos are numbered in order of completion. Chunk phases are in Table B.7 (Appendix B).

When four 25-minute videos arrived together (Figure 6.12), 8 vCPU finished them at 21.0, 38.7, 62.9 and 73.3 minutes, 1.4 times real time, and 16 vCPU at 17.7, 25.7, 36.1 and 44.3 minutes, 2.3 times real time. Doubling the capacity shortened the batch 1.65 times, closer to proportional than for one video, because the extra slots were always filled with chunks of the other videos. The queue is first-in first-out within a priority, so the videos finished in the order they arrived.

Table 6.7: Simultaneous submissions at 16 vCPU

BATCH	VCPU	READY	ALL (MIN)	P50 (MIN)	P95 (MIN)	PER MIN	VCPU-H
4 × 25min-1080p	16	4/4	44.3	36.1	44.3	0.09	9.2
100 × 60s-1080p	16	100/100	35.8	17.9	33.7	2.80	8.3

Note: ALL is the time of the whole batch, P50 and P95 the median and 95th percentile of a video's latency from its arrival, PER MIN the videos completed per minute, and VCPU-H the vCPU-hours used.

One hundred 60-second clips submitted at once at 16 vCPU all reached `READY` without a failure (Table 6.7). The batch took 35.8 minutes, the median clip waited 17.9 minutes and the 95th percentile 33.7, at 2.8 videos per minute for 8.3 vCPU-hours of compute. The fastest clip needed 3.7 minutes, the time of a single job running alone, so the rest of the latency is queueing: sixteen 1 vCPU jobs run at a time, and one hundred clips need about six rounds. The capacity quota, not the architecture, set the drain rate.

Fargate On-Demand was not faster. The 25-minute video on 8 vCPU of On-Demand took 26.0 and 22.5 minutes against 18.0 on Spot, with chunk medians of 414 and 313 seconds against 255 seconds. Two runs cannot show that On-Demand is slower, since Fargate does not guarantee the host hardware, but they give no reason to use it except as overflow when the Spot quota is exhausted. Spot was also reliable: in seven days of AWS Batch job history, 2 of 370 executions (0.5%) were interrupted, and AWS Batch retried both successfully.

Several limits apply to these figures. Most rows are single runs, and chunks of equal size varied by up to 1.8 times within one job, so differences of about ten percent between rows are not an effect. The four-hour source is a synthetic pattern at 5.35 Mbit/s that may encode faster than camera footage, although its chunks ran at 0.9 seconds per second of video against 1.0 for the 25-minute file. The 16 vCPU rows describe a quota that production does not have, and staging carried no other traffic during the runs.

6.6 Load Testing

The load test checks that a burst of uploads loses no work and that AWS Batch runs it at its configured concurrency. Fifty virtual users perform one hundred upload cycles, each of which initiates an upload, sends the file to S3 with the pre-signed URL and confirms it; the job queue is then polled every 25 seconds until it is empty. The compute environment allowed 8 vCPU and each job 1 vCPU, so at most eight jobs could run at once.

6.6.1 Queue and Admission Under Burst

The first runs uploaded a small synthetic byte string instead of a video, to exercise the upload API, the S3 event, SQS and job submission. Their jobs failed at container start-up because of the empty secret described in Section 5.5. This does not affect the queue

and admission behaviour measured here, but it means that the drain rate of these runs is not a transcoding throughput. The test was run with two independent load generators, a Node.js script and k6, so that a bottleneck in the generator could not pass for one in the system (Table 6.8).

Table 6.8: Cross-validation of the load test across two independent tools

METRIC	NODE.JS	K6
Upload cycles completed	100	100
Failure rate	0%	0%
Batch jobs queued (RUNNABLE)	92	92
Batch jobs admitted (STARTING)	8	8
SQS messages remaining	0	0
Queue drain, 76 jobs (s)	416	417
Consumption rate (jobs/min)	10.96	10.94

All 100 cycles succeeded with both tools. The Node.js script completed 15.52 cycles per second, with a mean of 2388.52 ms and a 95th percentile of 3317.79 ms per cycle. k6 times each request rather than each cycle: its mean of 670.03 ms over 300 requests is 2010.09 ms per three-request cycle, within 15.8% of the Node.js figure, and its steady-state rate of 17.24 cycles per second is close to 15.52.

The two tools agree on every observation of the queue. Immediately after each burst 92 jobs were waiting and 8 had been admitted, and SQS held no messages, so the Lambda submitter had moved every event into AWS Batch within seconds. The number of starting jobs stayed at seven or eight throughout, exactly `max_vcpus` of 8 divided by 1 vCPU per job. From the point where 76 jobs remained, the queue emptied in 416 s with Node.js and 417 s with k6, and no message was lost. SQS therefore kept every unit of work submitted faster than it could be processed, and AWS Batch admitted work at, and never above, its limit.

6.6.2 Throughput With Real Video

After the secret was fixed, the test was repeated with a real three-second H.264 clip in the same scenario. In the first such run all 300 HTTP requests and all 100 Batch jobs succeeded, at 23.47 requests per second with a 95th-percentile request time of 1827 ms. A second run, polled every 25 seconds, gives the drain curve: again 300 requests without failure, at 33.12 per second, all 100 jobs succeeded, and all 100 video records reached `READY` in the database. Table 6.9 compares it with the synthetic run.

Table 6.9: Backlog drain under synthetic and real payloads

	SYNTHETIC	REAL VIDEO
Jobs submitted	100	100
Jobs succeeded	0	100
Drain duration (s)	456	826
Drain rate (jobs/min)	13.16	7.26
Peak concurrent `RUNNING`	0	8
Implied wall-clock per job (s)	n/a	66.1

Note: both columns sampled every 25 s. The synthetic jobs failed at start-up, so none succeeded or reached `RUNNING`. Implied wall-clock per job is the drain duration times the observed concurrency divided by the number of jobs.

The real backlog drains at 7.26 jobs per minute against 13.16 for the synthetic one, so doing the work costs a factor of 1.8 in throughput compared with only admitting and rejecting it. `RUNNING` peaked at exactly eight, matching the configured limit, which the synthetic run could not show because its jobs never reached that state. At eight-way concurrency, 826 seconds for 100 jobs is about 66 seconds of wall-clock time per job for a three-second clip. Almost all of that is fixed cost, container provisioning, image pull and result upload, rather than encoding, so for short uploads it is this fixed cost of about a minute, not the speed of FFmpeg, that decides how long a user waits.

6.6.3 Serverless Backend Concurrency and Latency Evaluation

The initial load test revealed a severe concurrency bottleneck in the monolithic Amazon EC2 baseline: when fifty virtual users submitted requests simultaneously, the single Node.js event loop became saturated, driving the mean request latency to 2388.52 ms and the 95th percentile to 3317.79 ms, with maximum response spikes exceeding 6.13 seconds in the k6 test suite. Furthermore, the virtual machine represented a single point of failure (SPOF) with persistent idle costs.

To evaluate the performance of the serverless backend, empirical benchmarks were executed across cold start initialization, warm execution cycles, and concurrent burst loads (Table 6.10).

Table 6.10: Architectural and performance comparison between monolithic EC2 and serverless Lambda

DIMENSION	MONOLITHIC EC2 (BASELINE)	SERVERLESS LAMBDA (PROPOSED)
Compute Architecture	Single EC2 `t3.small` (2 vCPU, 2 GiB RAM)	Ephemeral AWS Lambda functions (1024 MB)
High Availability	Single-AZ (Single point of failure)	Multi-AZ active replication (3 AZs)
Cold Start Latency	60–90 s (VM operating system boot)	406.50 ms (Container init + DB connection)
Warm Execution (P95)	3317.79 ms (50 concurrent users)	0.47 ms handler / < 50 ms end-to-end
Burst Throughput	Bottlenecked at 15.52 req/s	Automatic burst scaling up to 1000 microVMs
Database Reuse	Persistent connection pool	Cached connection pooling (`readyState == 1`)
Monthly Idle Cost	$20.58 / month (compute + EBS root disk)	**$0.00 / month** (Always Free Tier)
Maintenance Burden	OS patching, kernel upgrades, manual stop/start	Fully managed, zero server administration

A cold start of the Lambda function, which includes container spin-up, module evaluation, and initial Mongoose connection establishment, required 406.50 ms. Once initialized, warm invocations exhibited a median execution latency of only 0.12 ms and a 95th percentile of 0.47 ms, yielding end-to-end HTTP response times consistently under 50 ms. Under concurrent burst evaluation with 10, 50, and 100 simultaneous requests, the serverless handler maintained linear scaling without socket backlog degradation: at 50 concurrent requests, the median request duration was 4.10 ms, representing a dramatic reduction compared to the 2305.56 ms median observed on the EC2 instance. This demonstrates that delegating horizontal concurrency management to AWS Lambda microVMs eliminates application-tier bottlenecks while providing active Multi-AZ fault tolerance.

6.7 Cost Analysis

The comparison sets an always-on Amazon EC2 instance running FFmpeg against the serverless container architecture, for three hundred videos a month averaging five minutes each. The processing effort comes from the measurements of Section 6.5: a 25-minute video used 6,363 vCPU-seconds at the best configuration and a four-hour video 15.1 vCPU-hours, about 0.35 vCPU-hours per five minutes of video, so the month needs

about 105 vCPU-hours. In other words, encoding one second of 1080p video takes about four vCPU-seconds.

Prices are the ap-southeast-1 list prices read from the AWS Price List API on 15 October 2026: 0.05056 USD per Fargate vCPU-hour and 0.00553 USD per GB-hour, which is 0.0616 USD per vCPU-hour with the 2 GB per vCPU of the chunk tasks. Spot capacity is taken at 30% of that, 0.0185 USD, since AWS publishes discounts of up to 70%; the real Spot price varies with demand.

Table 6.11: Monthly operating cost comparison for the part of the bill that the architecture controls (ap-southeast-1)

COMPONENT	EC2 (ALWAYS ON)	SERVERLESS	SAVING
Compute	`c5.xlarge` for 730 h: $124.10	Fargate Spot for 105 vCPU-h: $1.94	98.44%
Storage	EBS 100 GB plus S3 50 GB: $9.25	S3 with Glacier lifecycle: $1.45	84.32%
Queueing	Self-hosted broker	SQS and Lambda: $0.01	n/a
Total	**$133.35**	**$3.40**	**97.45%**

Two costs are left out of Table 6.11 because they do not depend on the architecture, and both are larger than the serverless column. Content delivery costs the same whichever compute model produced the renditions, about 12 USD a month after the free tier. Content screening is needed by both: a five-minute video sampled every five seconds gives 60 images, which at the Singapore price of 0.00125 USD per image is 0.075 USD per video and 22.50 USD for three hundred videos. With both costs added to both columns the totals are $167.85 against $37.90, a saving of about 77.4%, which is the figure to quote for total cost of ownership; the saving on the part that the architecture controls is 97.5%.

The saving comes entirely from removing idle capacity. Beyond the transcoding cluster, hosting the backend API on an always-on `t3.small` EC2 instance would contribute an additional $\dot{5}8$ monthly ($\dot{9}8$ compute plus $\dot{6}0$ for the 16 GB EBS root disk) even during zero-traffic idle periods. Transitioning the API to AWS Lambda completely eliminates this baseline expenditure: with monthly traffic estimated at 50,000 API calls, usage remains comfortably within the AWS Always Free Tier (1,000,000 requests and 3,200,000 seconds of compute monthly), yielding an operational cost of exactly $\dot{0}0$ for backend compute. The always-on instance offers 2,920 vCPU-hours a month, four vCPUs for 730 hours, while the work needs about 105 of them, a utilisation of roughly 3.6%. The advantage is therefore not universal: as sustained utilisation rises, per-second Fargate billing approaches and then exceeds the cost of a reserved instance. The architecture

suits bursty workloads, which user-generated video uploads are.

6.8 Requirements Against the Evidence

Table 6.12 sets each non-functional requirement of Table 3.4 against the evidence in this report. NFR-01 states no numeric target, so its row reports the measurement rather than a pass or a fail.

Table 6.12: Non-functional requirements against the evidence

REQUIREMENT	EVIDENCE	SECTION
NFR-01 Latency	Median time to first frame of 110.32 ms from Ho Chi Minh City through the CDN, against 251.52 ms from the S3 origin.	6.3.1
NFR-02 Scalability	In a burst of 100 uploads, SQS kept every message and AWS Batch ran jobs at its configured limit; 100 clips submitted at once all reached `READY`.	6.5, 6.6
NFR-03 Reliability	AWS Batch retries a failed job and raises an alert with the reason when the attempts run out; the 2 Spot interruptions in 370 executions were retried successfully.	5.5, 6.5
NFR-04 Cost efficiency	Compute is billed only while a job runs: 3.40 USD a month against 133.35 USD for an always-on instance.	6.7
NFR-05 Security of content	A request without a valid Signed Cookie is refused at the edge, as verified on the deployed distribution; the visibility test suite covers the API side.	4.4, 5.5, 5.6
NFR-06 Security of the API	Authentication endpoints allow ten attempts per fifteen minutes per address, and cross-origin requests are limited to an allow-list.	5.1
NFR-07 Reproducibility	Fourteen Terraform modules describe the environment, which is created with `terraform apply` from one directory.	5.3, A.5
NFR-08 Maintainability	Seven workflows run tests and security scanning, and the security gate blocks a merge on fixable critical findings.	5.4

Chapter 7

Conclusion and Future Work

This chapter compares the results with the objectives of Section 1.2, then states the strengths and limitations of the delivered system and the work that would extend it.

7.1 Summary of Achievements

The project set out to build a video sharing platform whose transcoding runs on serverless containers driven by events, and to measure whether that choice delivers the cost and scalability it promises. Table 7.1 sets each objective against its result and the section that holds the evidence.

Table 7.1: Objectives of the project against their results

OBJECTIVE	RESULT	EVIDENCE
1. Web platform	Users register with a verified e-mail address, upload directly to Amazon S3, watch with adaptive bitrate over up to six renditions, set each video public, unlisted or private, manage their channel, search the catalogue, and like and comment. A private video is refused at the edge even when its manifest URL is known.	Sections 4.4, 4.6, 6.2
2. Automated pipeline	An upload event passes through Amazon SQS and AWS Lambda to AWS Batch, which produces the renditions, the master playlist and a thumbnail without manual action. Conditional writes stop a duplicate delivery from overwriting a finished job, and a failed job is retried and then raises an alert with its reason.	Sections 4.2, 5.5
3. On-demand containers	FFmpeg runs in a container on AWS Batch over Fargate that exists only while a job runs. Videos of five minutes or more are split into chunks encoded in parallel, so a four-hour video was ready 135 minutes after its upload at 8 vCPU and 85 minutes at 16 vCPU.	Sections 4.2.2, 6.5
4. CI/CD and IaC	Fourteen Terraform modules describe three environment definitions, and seven GitHub Actions workflows run linting, tests, security scanning, infrastructure validation and deployment. The suite holds 853 automated tests in 52 suites.	Sections 5.3, 5.4, 5.6
5. Empirical evaluation	Time to first frame, playback under constrained bandwidth, transcoding latency, burst load and monthly cost were measured on the deployed system.	Chapter 6

One result goes beyond the five objectives. The platform screens every upload with Amazon Rekognition and holds doubtful videos for staff, who work in a console with separate roles: operators decide, auditors read the error rates, and administrators assign roles. Each decision is stored beside the machine's own verdict, so the error rates can be estimated with confidence intervals, and the owner of a removed video can appeal (Section 4.5).

7.2 Strengths of the System

The application tier never handles video bytes. The backend issues pre-signed URLs, records metadata and answers queries, while video flows directly between the browser, Amazon S3 and CloudFront. The load on the API therefore does not depend on video size or upload concurrency, and a failure in the processing tier leaves published videos playable.

The processing tier costs nothing while idle, because a transcoding container exists only while a job runs. For the part of the bill that the architecture controls, Section 6.7 estimates $3.40 a month against $133.35 for an always-on instance; with screening and delivery added to both sides, the saving is about 77%.

Additional capacity turns directly into lower latency. Chunk jobs are ordinary AWS Batch jobs on the same compute environment, so a larger vCPU quota encodes more chunks at once, and no chunk came near the two-hour limit of a job (Section 6.5).

The performance claims are measured rather than assumed. Measuring time to first frame on the deployed CDN showed that its price class served Vietnam from Europe, and measuring from other regions showed the same gap for Brazil; the two corrections reduced the median latency by a factor of 6.6 from Vietnam and 32 from Brazil (Section 6.3). The automated suite likewise caught a cookie signing defect before deployment (Section 5.6).

7.3 Limitations

Transcoding. The bitrate ladder has six fixed rungs whatever the content. Optimising the ladder per title or per shot can save bandwidth; Netflix reported reductions of 28% to 38% at equal perceptual quality [7]. Extra capacity also helps a short video little: a 25-minute video took 18.0 minutes at 8 vCPU and about 15 minutes at 16 vCPU, because planning, audio encoding and finalisation take about three minutes whatever the capacity. Production has a Fargate Spot quota of 8 vCPU, so four 25-minute videos submitted together finished after 73 minutes. Below five minutes a video keeps the single-container path, where fixed overhead dominates: about 66 seconds per job for a three-second clip (Section 6.6.2).

Moderation. Screening samples one frame every five seconds, so content shown for less than that can pass. Its thresholds of 60 and 90 per cent follow the provider's guidance and were not calibrated on this platform's content. The 147 backend tests of moderation cover the implementation, but the error rates have not been measured on real traffic, so this report makes no claim about the accuracy of the screening. Screening is also the largest line of the monthly bill, 22.50 USD under the stated workload.

Accounts and access. A new password is checked against common passwords, the user's own details and a breach service [14], but existing passwords were not rechecked and the breach lookup is skipped when the service does not answer within two seconds. Mail to one address is capped at five per hour, but the count lives in process memory and registration has no CAPTCHA. Signed Cookies stay valid for two hours, so making a video private does not revoke cookies already issued, and the system has no digital

rights management.

Operation. In the initial prototype baseline, the backend operated on a single EC2 instance in one region as the sole single point of failure of the system. This architectural risk was successfully resolved by transitioning the API to AWS Lambda across multiple Availability Zones with Function URLs, achieving complete end-to-end serverless resilience and zero idle costs. Rate limits and view deduplication currently rely on in-memory state; future enterprise scaling will offload these counters to an Amazon ElastiCache Redis cluster. The CloudFront distributions live in a second AWS account, because the primary account was not permitted to create them, which splits the infrastructure across two sets of credentials.

Interface. Tags can be set only at upload, and a thumbnail cannot be replaced. The account stores no language preference, so the e-mails sent by a transcoding job, which has no request to read a language from, are written in both languages.

The limits of each measurement, such as single runs, a synthetic four-hour source and probes inside AWS regions, are stated beside the result in Chapter 6 and are not repeated here.

7.4 Future Work

The first extension is a per-title bitrate ladder chosen with a perceptual quality metric such as VMAF. Encoding a short probe of the source at several operating points and keeping those on the convex hull of quality against bitrate would lower delivery bandwidth, and with it the delivery cost, at the same perceived quality.

The second is less fixed overhead on the chunked path. Each chunk pays a container start-up of tens of seconds. Encoding several chunks in one longer-lived worker, or raising the minimum chunk length, trades parallelism for less overhead; the measurements of Section 6.5 suggest that the best choice depends on the video length and the quota. A faster x264 preset, checked with VMAF on difficult clips, could shorten every chunk.

The third is removing the single point of failure. A second backend instance behind a health-checked address needs a load balancer and a shared store, such as Redis, for the rate limits, view deduplication and mail count; the persistent state is already in MongoDB Atlas and Amazon S3. A CAPTCHA on registration and password reset, and distributed tracing across the S3, SQS, Lambda and Batch boundaries, belong to the same step.

The fourth is tuning the screening from evidence. The console already records the reason codes of reversed removals and an unbiased audit sample; after a few hundred decisions, the thresholds and the label policy can be adjusted and the effect measured

again.

Two measurements should also be repeated: the 1 vCPU baseline, with more runs and with memory held constant (Section 6.4), and time to first frame from residential connections rather than from AWS regions (Section 6.3.2). Further ahead, GPU-enabled serverless encoding [18] and a more efficient codec such as HEVC or AV1 next to H.264 would raise throughput and reduce bandwidth.

7.5 Task Distribution

Table 7.2 shows how the work was divided between the two members of the team. Each member was responsible for the parts listed and for the chapters of the report that describe them.

Table 7.2: Task distribution between the members of the team

MEMBER	**STUDENT ID**	**RESPONSIBILITIES**
Dinh Quoc Cuong (team leader)	523H0008	Overall architecture; the transcoder and the chunk-parallel path; the AWS infrastructure in Terraform (S3, SQS, Lambda, Batch, CloudFront); the CI/CD and DevSecOps pipeline; the measurements and the evaluation; Chapters 4 to 6 of the report
Vo Huynh Minh Duc	523H0014	The React frontend (catalogue, watch page and player, upload, channel management, moderation console, two languages); the account, comment and interaction APIs; testing and diagrams; Chapters 1 to 3 and 7 of the report

Bibliography

[1] Netflix Technology Blog, "Rebuilding netflix video processing pipeline with microservices." https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359, 2024. Accessed: 2026-08-10.

[2] Netflix Technology Blog, "The making of VES: the cosmos microservice for netflix video encoding." https://netflixtechblog.com/the-making-of-ves-the-cosmos-microservice-for-netflix-video-encoding-946b9b3cd300, 2024. Accessed: 2026-08-10.

[3] Amazon Web Services, "Video on demand on AWS: Implementation guide." https://docs.aws.amazon.com/solutions/latest/video-on-demand-on-aws/, 2025. Accessed: 2026-08-10.

[4] M. Zhang, Y. Zhu, C. Zhang, and J. Liu, "Video processing with serverless computing: A measurement study," in *Proceedings of the 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2019.

[5] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, 2025. Published version of arXiv:2310.08437.

[6] Netflix Technology Blog, "Per-title encode optimization." https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2, 2015. Accessed: 2026-08-10.

[7] Netflix Technology Blog, "Dynamic optimizer: A perceptual video encoding optimization framework." https://netflixtechblog.com/dynamic-optimizer-a-perceptual-video-encoding-optimization-framework-e19f1e3a277f, 2018. Accessed: 2026-08-10.

[8] Amazon Web Services, "Serve private content with signed URLs and signed cookies: Amazon cloudfront developer guide." https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html, 2025. Accessed: 2026-08-10.

[9] Amazon Web Services, "Moderating content, Amazon Rekognition developer guide." https://docs.aws.amazon.com/rekognition/latest/dg/moderation.html. Accessed: 2026-10-15.

[10] Amazon Web Services, "Guidelines and quotas in Amazon Rekognition." https://docs.aws.amazon.com/rekognition/latest/dg/limits.html. Accessed: 2026-10-15.

[11] "The Santa Clara principles on transparency and accountability in content moderation." https://santaclaraprinciples.org/, 2021. Version 2.0. Accessed: 2026-10-15.

[12] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[13] J. Ali, "Validating leaked passwords with k-anonymity." Cloudflare Blog, https://blog.cloudflare.com/validating-leaked-passwords-with-k-anonymity/, 2018. Published 2018-02-21. Accessed: 2026-10-16.

[14] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017. Includes updates as of 2020-03-02. https://doi.org/10.6028/NIST.SP.800-63b. Accessed: 2026-10-16.

[15] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.

[16] S. Sankaraiah, L. H. Shuan, C. Eswaran, and J. Abdullah, "Scalable video encoding with macroblock-level parallelism," *EURASIP Journal on Advances in Signal Processing*, vol. 2014, p. 145, 2014.

[17] C.-Y. Chen, S.-C. Cheng, *et al.*, "Task-based parallel H.264 video encoding for explicit communication architectures," in *Proceedings of the International Confer-*

ence on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS), 2011.

[18] A. Salcedo-Navarro, R. Peña-Ortiz, J. M. Claver, M. Garcia-Pineda, and J. Gutierrez-Aguado, "Towards GPU-enabled serverless cloud edge platforms for accelerating HEVC video coding," *Cluster Computing*, vol. 28, p. 68, 2025.

Appendix A

Supplementary Material

A.1 Repository Structure

The source code is organised as a monorepo with the following top-level layout.

```
DACNTT_VideoStreaming/
  frontend/            React single page application (Vite, HLS.js)
  backend/             Node.js Express API and test suite
  transcoder/          FFmpeg worker and Docker image
  infrastructure/
    modules/           Thirteen reusable Terraform modules
    environments/      dev, staging and prod definitions
  .github/workflows/   Seven CI/CD workflows
  scripts/             Benchmarking and load testing scripts
  docs/                Architecture diagrams, cost analysis,
   measured results
  report/              LaTeX sources of this report
```

A.2 FFmpeg Transcoding Command

The following invocation is the one the worker builds for a 1080p source at 24 fps, which fills all six rungs of the ladder, and it produces every rendition from one read of the source. A single input is followed by one output block per rendition, each carrying its own scaling filter and encoder settings, so FFmpeg decodes the source once and feeds all six filter chains from it. The blocks after the first are abbreviated here, and the 360p, 480p and 720p blocks are left out of the listing; in the source every block repeats the first in full. The master playlist is not written by FFmpeg: the worker generates it afterwards,

which is why no -master_pl_name argument appears. It does so from the encoded output rather than from the settings above, measuring each rendition's peak segment bit rate from the segment sizes and their #EXTINF durations, and reading the codec string for each variant out of the sequence parameter set of its first segment.

The keyframe interval is derived from the source rather than assumed. The worker probes the first video stream and reads avg_frame_rate, preferring it over r_frame_rate because the latter is a base rate FFmpeg infers from the timing grid and need not describe the frames that are actually present.

A screen recording taken from the project's own test material shows how far apart the two can be. The file reports r_frame_rate as 60000/1001, that is 59.94 fps, while avg_frame_rate reports 28.725 fps; counting frames confirms the second, at 287 frames in ten seconds. Every interval between consecutive frames turns out to be an integer multiple of 1/144 second — four, five or six ticks, giving instantaneous rates of 36, 28.8 and 24 fps — so the real timing grid is the 144 Hz display, and neither the 59.94 that r_frame_rate advertises nor the 30 the source game was capped at. The cap cannot be met exactly, because 144 is not divisible by 30 and a frame would have to occupy 4.8 ticks.

The consequence for the group of pictures is direct. Taking avg_frame_rate gives 172 frames, or 5.988 seconds, which fits inside the segment. Taking r_frame_rate would give 359 frames, or 12.5 seconds, more than twice the segment length. A fixed 30 fps assumption would give 180 frames, or 6.27 seconds, which also overruns. This is also why -force_key_frames is retained alongside -g: when the source rate varies between 24 and 36 fps, an interval counted in frames cannot correspond to a fixed interval in time, and only the time-based expression can.

Once the rate is known the group of pictures is floor(fps × 6) frames, floored rather than rounded so that it never exceeds the segment length. The same count of 180 frames illustrates why the rate has to be right before the arithmetic can be: it lasts 6.27 seconds at the 28.725 fps just measured, and 6.006 seconds at 29.97 fps, where it arises instead from rounding 179.82 upward. Both overrun a six-second segment, and in both cases the boundary drifts past the point where a keyframe exists. The value shown below, 144, is what a 24 fps source produces.

Two further arguments keep the boundaries aligned. Scene detection is disabled, because letting the encoder place keyframes at scene changes would allow the renditions to cut at different points, and RFC 8216 requires that matching content in variant streams carry matching timestamps. -force_key_frames then pins a keyframe to every sixth second of media time; unlike -g, which only bounds the interval from above, it is absolute

and independent of the detected frame rate, so a misdetected source still yields aligned segments.

The H.264 profile is pinned but the level is not. Each rendition carries a different frame size, and the lowest conforming level therefore differs between them, so the level is left for the encoder to derive from the resolution rather than fixed in the command. The pixel format is pinned to 8-bit 4:2:0 for the same reason as the profile: a 10-bit or 4:4:4 source would otherwise be encoded in a form that the main profile does not allow and most browsers do not decode.

```
ffmpeg -i input.mp4 -hide_banner -loglevel warning -stats -y \
  -vf "scale=256:144,setsar=1" \
  -c:v libx264 -b:v 120k -maxrate 150k -bufsize 240k \
  -preset fast -profile:v main -pix_fmt yuv420p \
  -g 144 -keyint_min 144 -sc_threshold 0 \
  -force_key_frames "expr:gte(t,n_forced*6)" \
  -c:a aac -b:a 48k -ar 44100 -ac 2 \
  -f hls -hls_time 6 -hls_list_size 0 \
  -hls_segment_filename 144p/segment_%03d.ts  144p/playlist.m3u8 \
  \
  -vf "scale=426:240,setsar=1" -c:v libx264 -b:v 240k -maxrate 300k
   -bufsize 480k \
  -c:a aac -b:a 64k -f hls -hls_time 6 \
  -hls_segment_filename 240p/segment_%03d.ts  240p/playlist.m3u8 \
  \
  ... 360p, 480p and 720p in the same form ... \
  \
  -vf "scale=1920:1080,setsar=1" -c:v libx264 -b:v 4000k -maxrate
   5000k -bufsize 8000k \
  -c:a aac -b:a 192k -f hls -hls_time 6 \
  -hls_segment_filename 1080p/segment_%03d.ts 1080p/playlist.m3u8
```

A.3 Signed Cookie Custom Policy

The policy signed by the backend before issuing playback cookies. The wildcard in the resource path covers the manifest and every segment of one specific video, and nothing beyond it.

```
{
  "Statement": [
```

```
      {
        "Resource": "https://cdn.zelostech.site/videos/<videoId>/*",
        "Condition": {
          "DateLessThan": { "AWS:EpochTime": 1786300347 }
        }
      }
    ]
}
```

A.4 Reproducing the Measurements

The experimental results reported in Chapter 6 are produced by the scripts listed below. Detailed instructions, including the environment variables required and the order in which the measurements must be taken, are given in `docs/results/README.md` within the repository.

Every table of measurements in Chapter 6 and in Appendix B is listed below against the script that produced it and the file its figures were read from, so that any individual number can be traced back to its source. Paths are relative to the repository root, and the result files are those under `docs/results/`.

Table A.1: Provenance of each measurement table

TABLE	SCRIPT	RESULT FILE
B.1	scripts/benchmark-qoe.js	qoe-ttff.json
B.2	scripts/benchmark-qoe.js	qoe-ttff-cloudfront.json
B.3	scripts/benchmark-qoe.js, deployed as a Lambda to each region	qoe-ttff-multiregion.json
6.1	scripts/qoe/collect.js	qoe-playback-<profile>.json
6.2	Segment sizes and #EXTINF durations read from the processed bucket	—
6.3	scripts/qoe/collect.js, run before and after the correction	qoe-playback-<profile>.json
B.4	scripts/qoe/compare.js, driven by docs/results/qoe-sources.json	qoe-playback-<source>-<profile>.json, qoe-comparison.json
6.4	scripts/qoe/collect.js with -dpr	qoe-playback-<source>-unthrottled.json, qoe-playback-<source>-unthrottled-dpr2.json
6.5	scripts/measure-transcode.js	transcode-timing.json
6.8	scripts/node-load-test.js and scripts/k6-load-test.js	node-load-test.json, stress-test-concurrency.json; the k6 summary of this run was later overwritten and remains in k6-summary.json at commit f955ca9
6.9	scripts/k6-load-test.js	drain-rate-real-payload.json, stress-test-concurrency.json; the first real-payload run is in k6-summary.json
6.11	Published AWS list prices, computed analytically	—

The charts of Chapter 6 are drawn by the scripts docs/diagrams/svg/11-chart-ttff-cdn.mjs to docs/diagrams/svg/16-chart-concurrent-four.mjs, and each reads the result files named above for the table it replaces. The three that chart the transcoding measurements read report/generated/chart-*.json, which scripts/results/build-report-tables.py writes from the same runs as the generated tables, so those charts and tables cannot differ.

A.5 Deploying the Infrastructure

The complete AWS environment is provisioned from a single directory.

```
cd infrastructure/environments/dev
terraform init
terraform apply
```

Enabling signed cookies additionally requires generating an RSA key pair, supplying the public key to Terraform, and configuring the backend with the corresponding private key and the key pair identifier reported by `terraform output`. The full procedure is documented in `docs/PRIVATE_VIDEO_SIGNED_COOKIES.md`.

Appendix B

Measurement Tables

Chapter 6 shows six sets of measurements as charts, because what each of them reports is a comparison or a trend that is easier to see than to read. This appendix gives the same measurements as tables, so that every value can be read exactly and so that the statistics a chart leaves out, such as percentiles and cache-hit medians, remain available. Each chart reads the same result files as its table, and a test in the repository checks that the numbers in the two agree; Section A.4 names the files.

B.1 Time-to-First-Frame

Table B.1 gives the distribution of each stage of the baseline measurement against the S3 bucket; Figure 6.7 shows its medians.

Table B.1: Time-to-First-Frame measurement results

STAGE	MEAN	MEDIAN	P95	MIN	MAX
Master playlist	95.55	82.49	355.02	76.61	355.02
Rendition playlist	88.62	82.40	207.32	77.62	207.32
First segment	108.76	87.04	322.20	78.41	322.20
Total TTFF	**292.93**	**251.52**	**884.54**	**242.23**	**884.54**

Note: values in milliseconds, n = 20 samples, measured against a video served directly from the S3 bucket, before the CloudFront distribution described in Section 5.5 was deployed. These figures therefore represent origin latency without any CDN, and they serve here as the baseline against which Section 6.3.1 compares the deployed system. The mean (292.93 ms) exceeds the median (251.52 ms) because one outlier sample (884.54 ms) inflates both the mean and the P95/max columns, which coincide with the

single-sample maximum at this sample size.

Table B.2 gives the same statistics for the measurement through CloudFront after the price class correction, which Figure 6.7 compares with the baseline.

Table B.2: Time-to-First-Frame through CloudFront after the price class correction

STAGE	**MEAN**	**MEDIAN**	**P95**	**MIN**	**MAX**
Master playlist	28.65	8.37	263.43	6.54	263.43
Rendition playlist	29.29	7.93	348.78	5.89	348.78
First segment	112.75	87.36	487.31	68.87	487.31
Total TTFF	**170.69**	**110.32**	**1099.51**	**83.17**	**1099.51**
Warm cache only ($n = 18$)	*108.49*	*107.57*	*159.25*	*83.17*	*159.25*

Note: values in milliseconds, $n = 20$ samples with Signed Cookies attached. Two samples were edge cache misses and eighteen were hits; the final row reports only the samples that hit at all three stages, which is the steady state a viewer of an already-watched video experiences. The P95 and maximum of the full sample coincide with the colder of the two misses, as at this sample size the 95th percentile is the single slowest observation.

Table B.3 adds, to the medians of Figure 6.8, the edge location that served each client and the median over the samples that hit the cache.

Table B.3: Time-to-First-Frame by client location, before and after `PriceClass_All`

	PRICECLASS_200		**PRICECLASS_ALL**		
CLIENT LOCATION	**EDGE**	**MEDIAN**	**EDGE**	**MEDIAN**	**WARM**
N. Virginia, United States	IAD61	55.23	n/a	n/a	n/a
Dublin, Ireland	DUB2	55.91	DUB2	55.68	55.68
Singapore	SIN3	56.89	SIN3	53.20	53.20
Tokyo, Japan	NRT12	59.68	n/a	n/a	n/a
Sydney, Australia	n/a	n/a	SYD3	52.30	43.36
São Paulo, Brazil	CPT51	**1196.10**	GRU1	**36.79**	**36.49**
Ho Chi Minh City (consumer)	SGN50	110.32	n/a	n/a	n/a

Note: values in milliseconds, $n = 20$ per location and per configuration. WARM reports the median over samples that hit the edge cache at all three stages. Entries marked n/a were not measured in that configuration: the first run predated the Sydney probe, and the second re-measured only the two locations that could change plus two controls. The Ho Chi Minh City row is the local measurement of Table B.2, taken from a consumer connection rather than from inside an AWS region, and is included for comparison rather than as a like-for-like sample.

B.2 Playback Quality

Table B.4 gives all five measures for each of the nine combinations of source and network profile, of which Figure 6.9 charts two.

Table B.4: Playback quality across three sources of differing visual complexity

METRIC	LOW	MEDIUM	HIGH
Unthrottled			
Rebuffering ratio, mean	0.00%	0.00%	0.00%
Rebuffering ratio, max	0.00%	0.00%	0.00%
Runs exhibiting a stall	0/5	0/5	0/5
Startup delay, median (s)	1.46	1.34	1.21
Average bitrate (kbit/s)	1644	1830	1664
Fast 3G			
Rebuffering ratio, mean	0.22%	2.30%	0.11%
Rebuffering ratio, max	0.60%	5.81%	0.55%
Runs exhibiting a stall	2/5	2/5	1/5
Startup delay, median (s)	1.22	1.15	1.12
Average bitrate (kbit/s)	1477	1316	1246
Slow 3G			
Rebuffering ratio, mean	4.91%	5.80%	4.84%
Rebuffering ratio, max	13.14%	13.73%	13.37%
Runs exhibiting a stall	3/5	3/5	3/5
Startup delay, median (s)	3.76	1.87	3.84
Average bitrate (kbit/s)	1070	1224	845

Note: five recorded runs per cell after two discarded warm-up runs, 120 s of playback each, at a device pixel ratio of one. Source durations are 4:23, 4:23, and 4:40, all longer than the measurement window.

B.3 Transcoding Latency and Capacity

Table B.5 lists the ten runs of the 25-minute video with the columns that Figure 6.10 does not show: capacity, slots and the length of the chunk phase.

Table B.5: A 25-minute video on the chunked path, by chunk configuration and capacity

N × VCPU	FARGATE	VCPU	SLOTS	PHASE (MIN)	READY (MIN)	VCPU-S
5 × 1	Spot	8	8	n/a	30.1	n/a
5 × 1†	Spot	8	8	32.6	36.3	7 117
23 × 1	Spot	8	8	19.2	23.6	7 270
12 × 2	Spot	8	4	16.3	20.1	6 816
6 × 4	Spot	8	2	16.8	20.4	6 792
6 × 4	Spot	8	2	14.4	18.0	6 363
6 × 4	On-Demand	8	2	21.7	26.0	10 182
6 × 4	On-Demand	8	2	18.2	22.5	7 852
12 × 4	Spot + OD	16	4	11.5	15.3	8 827
12 × 4	Spot + OD	16	4	10.2	14.7	8 159

Note: the first four rows were timed from the creation of the first job and the later rows from the end of the upload, which differ by about half a minute. The vCPU-seconds of the first five rows count chunk jobs only; the later rows count every job of the video. Rows are in chronological order of measurement. The first column gives the number of chunks and the vCPU of each; † *marks the run on the production environment. PHASE is the time from the first chunk start to the last chunk end, SLOTS the number of chunks encoded at once, VCPU-S the vCPU-seconds, and OD stands for On-Demand.*

Table B.6 gives the minutes of each step of the four-hour video, including the steps too short to be labelled in Figure 6.11, and the compute used.

Table B.6: A four-hour video at two capacities

STEP (MINUTES)	16 VCPU	8 VCPU
Planner (Lambda submit to child jobs)	0.4	0.4
Audio 64 kbit/s and 128 kbit/s (in parallel)	9.6	4.7
48 chunk jobs, 4 vCPU each	71.8	125.9
Finalizer (playlists, thumbnail, moderation, READY)	2.3	3.2
Upload complete to READY	**85.3**	**135.3**
Compute (vCPU-hours)	17.3	15.1

Table B.7 gives, for each of the four videos, the start and end of the chunk phase that Figure 6.12 draws.

Table B.7: Four 25-minute videos submitted together, by capacity

VCPU	ORDER	START (MIN)	END (MIN)	READY (MIN)
8	1	2.8	19.7	21.0
8	2	18.4	37.1	38.7
8	3	35.5	58.5	62.9
8	4	52.2	71.9	73.3
16	1	3.0	16.0	17.7
16	2	2.7	24.6	25.7
16	3	22.2	35.0	36.1
16	4	32.0	43.3	44.3

Note: in the 8 vCPU rows the four videos were uploaded together over one connection and timed from the first job creation. In the 16 vCPU rows the arrival was produced by server-side copies of one uploaded file, so all four events reached the pipeline within a second. START and END are the first chunk start and the last chunk end, and ORDER is the order of completion.